# GARBAGE-GUARD: DEVELOPMENT OF COMPUTER VISION FOR CREEKS GARBAGE DETECTION USING CONVOLUTIONAL NEURAL NETWORK

**A Capstone Study**

Presented to the Faculty of the

**ELECTRONICS ENGINEERING TECHNOLOGY DEPARTMENT**

College of Industrial Technology

Technological University of the Philippines

Ayala Boulevard, Ermita, Manila

by

**BRIAN JAY CELESTE**

**GENESIS WALTER D. REYES II**

**JADE C. GUNAYAN**

**KING PERTH G. PEPITO**

**RODEL S. BANOGBANOG**

In Partial Fulfillment for the Requirements of the Degree

**Bachelor of Engineering Technology major in Computer Engineering Technology**

November, 2026

Technological University of the Philippines
**COLLEGE OF INDUSTRIAL TECHNOLOGY**
Ayala Boulevard, Ermita, Manila 1000
cit@tup.edu.ph

## APPROVAL SHEET

The capstone study titled **"GARBAGE-GUARD: DEVELOPMENT OF COMPUTER VISION FOR CREEKS GARBAGE DETECTION USING CONVOLUTIONAL NEURAL NETWORK"**, prepared and submitted by **BRIAN JAY CELESTE, GENESIS WALTER D. REYES II, JADE C. GUNAYAN, KING PERTH G. PEPITO and RODEL S. BANOGBANOG**, in partial fulfillment of the requirements for the degree **Bachelor of Engineering Technology major in Computer Engineering Technology**, is hereby approved and accepted.

**JONEL R. MACALISANG**
Adviser

**TRISTAN R. MOPAS**
Chairperson

**MARC ARDIE V. ARDIENTE**
Member

**MA. IAN P. DE LOS TRINOS**
Member

**MINABELLE D. VILLAFUERTE**
Member

**GHAZALI R. SISON**
Dean, College of Industrial Technology

Date: _____

# TABLE OF CONTENTS


| Section | Page |
|---|---|
| **PRELIMINARIES** | |
| Title Page | i |
| Approval Sheet | ii |
| Table of Contents | iii |
| List of Tables | v |
| List of Figures | vii |
| List of Abbreviations | xiii |
| Abstract with Keywords | xiv |
| **I. INTRODUCTION** | **1** |
| 1.1 Background of the Study | 1 |
| 1.2 Objectives of the Study | 3 |
| 1.3 Scope and Limitations of the Study | 4 |
| 1.4 Significance of the Study | 6 |
| **II. LITERATURE REVIEW** | **7** |
| 2.1 Conceptual Framework of the Study | 7 |
| 2.2 Review of Related Literature and Studies | 9 |
| 2.3 Review of Related Patents | 28 |
| 2.4 Synthesis of the Review | 40 |
| **III. METHODOLOGY** | **42** |
| 3.1 System Design | 42 |
| 3.2 System Development | 64 |
| 3.3 Testing Procedure | 75 |
| **IV. RESULTS AND DISCUSSION** | **85** |
| 4.1 System Description and Structure | 85 |

4.2 Test Results 96

**V. CONCLUSIONS 122**

**VI. ACKNOWLEDGMENT 126**

**VII. DECLARATION OF CONFLICT OF INTEREST 127**

**VIII. REFERENCES 128**

**IX. APPENDICES 2**

Appendix A. Gantt Chart 2

Appendix B. Bill of Materials 3

Appendix C. Testing Permit 4

Appendix D. Deployment Permit 6

Appendix E. Documentation of the Study 9

Appendix F. Usability Survey Form 25

Appendix G. End-User Usability Survey Responses 26

Appendix H. User Interface of the System 27

Appendix I. User Manual 48

**X. STUDENTS' BIONOTE 58**

# LIST OF TABLES


| Table | Title | Page |
|---|---|---|
| 1 | Table of Related Patents | 29 |
| 2 | Proposed Solution Matrix | 41 |
| 3 | ISO/IEC 25010:2023-Based Metrics Descriptions and Scale of Measurements for Garbage-Guard | 75 |
| 4 | Unit Testing Cases for Controlled Environment Evaluation | 79 |
| 5 | System Testing Cases for Real-World Environmental Conditions | 80 |
| 6 | Acceptance Testing Evaluation Test Cases | 82 |
| 7 | Developer-Side Internal Acceptance Indicators | 83 |
| 8 | End User Side Acceptance Survey Indicators | 83 |
| 9 | Five-Point Likert Scale and Interpretation of the Weighted Mean | 84 |
| 10 | Training Runs of the YOLO26n Model and Their Test Results | 96 |
| 11 | Per-Class Detection Results on the Frozen Test Set and on the Creek Images | 100 |
| 12 | mAP@0.5 of v7 to v10 on the Frozen Test Set and the Creek-Only Photographs | 103 |
| 13 | mAP@0.5 on Pictures from the Prototype's Camera and on the Third-Session Test Half | 104 |
| 14 | Unit Testing Results | 106 |
| 15 | System Testing Results | 106 |
| 16 | Field Deployment Log | 113 |
| 17 | Threshold Calibration Record | 114 |

| Table | Title | Page |
|---|---|---|
| 18 | Developer-Side Internal Acceptance Results | 117 |
| 19 | Acceptance Testing Results | 117 |
| 20 | End-User Usability Survey Results | 119 |
| G1 | Ratings and Comments of the Respondents in the End-User Usability Survey | 26 |

# LIST OF FIGURES


| Figure | Title | Page |
|---|---|---|
| 1 | Conceptual Framework of Garbage-Guard | 7 |
| 2 | Cumulative Contribution of Rivers to Global Plastic Emission | 10 |
| 3 | Municipal Solid Waste against Gross Domestic Product per Capita | 12 |
| 4 | Weathering and Fragmentation of Plastic in the Marine Environment | 13 |
| 5 | Floating Objects Detected on a Water Surface | 14 |
| 6 | Success Rates of Visual Tracking Algorithms on a Benchmark Set | 15 |
| 7 | Detection and Tracking Pipeline for River-Floating Waste | 16 |
| 8 | Tracking Accuracy and Speed of Four Association Methods on the MOT17 Validation Set | 17 |
| 9 | Detection Accuracy in the Laboratory Compared with the Field | 18 |
| 10 | Floating Litter Images with True and Predicted Labels | 19 |
| 11 | Ground Truth Compared with Model Output for Three Waste Classes | 20 |
| 12 | Overhead Imagery of the Kind Used for Object Detection | 22 |
| 13 | Architecture of the YOLOv8 Detector | 23 |
| 14 | Architecture of a Convolutional Neural Network for Image Classification | 23 |
| 15 | Selection of Studies for a Systematic Review of Air Quality Monitoring | 25 |
| 16 | Water Level and Pressure Recorded during a Rainfall Event | 27 |
| 17 | Neuro-Fuzzy Network for Flood Forecasting | 28 |
| 18 | Garbage Detection Method, Related Equipment and Device | 30 |
| 19 | Vision Garbage Classification System and Classification Method Based on TensorFlow | 31 |

| Figure | Title | Page |
|---|---|---|
| 20 | WCM-Smart City: Waste Collection and Management under Smart City Using IoT | 32 |
| 21 | Cleaning Unit Using Deep Learning for Floating Trash Detection | 33 |
| 22 | System for Monitoring Water Drainage | 34 |
| 23 | Drainage Monitoring Apparatus and System | 35 |
| 24 | Detection and Monitoring of Waste Disposal Using Artificial Intelligence | 36 |
| 25 | Flood Warning System | 37 |
| 26 | Method and System for Flood Early Warning in Smart Cities Based on the Internet of Things | 38 |
| 27 | Apparatus and Method for Waste Monitoring and Analysis | 39 |
| 28 | Garbage-Guard's System Architecture | 42 |
| 29 | Front and Side Views of the Garbage-Guard Enclosure with Dimensions | 43 |
| 30 | System Flow of the Waste Detection System | 44 |
| 31 | Garbage-Guard's System Flowchart | 45 |
| 32 | Raspberry Pi 5 Single-Board Computer | 47 |
| 33 | Hikvision HiWatch HWI-B140H Bullet Network Camera | 48 |
| 34 | Waveshare UPS HAT (E) with Four Battery Bays | 49 |
| 35 | 5V to 12V USB-A to DC Barrel Step-up Cable | 50 |
| 36 | Erlvsoiis WF-ER1800M Dual-Band USB Wi-Fi Adapter | 51 |
| 37 | Vention USB 3.0 Type A Male-to-Female Extension Cable | 51 |
| 38 | 7-inch IPS HDMI Touchscreen | 52 |
| 39 | Raspberry Pi Active Cooler Fitted to a Raspberry Pi 5 | 53 |
| 40 | SanDisk High Endurance 64GB microSD Card | 54 |
| 41 | KingSpec M.2 2242 NVMe Solid-State Drive | 54 |

| Figure | Title | Page |
|---|---|---|
| 42 | Waveshare PCIe to 2-Channel M.2 Adapter | 55 |
| 43 | MH-FMD Passive Buzzer Module | 56 |
| 44 | Metal Waterproof Junction Box Used for the Enclosure, Before Modification | 57 |
| 45 | Mitsushi 2500W Power Extension with Three Universal Outlets and Six USB Ports | 58 |
| 46 | 5V USB Case Fan | 59 |
| 47 | V-Model and Scrum Hybrid Development Model | 64 |
| 48 | Model Training and Deployment Pipeline | 67 |
| 49 | Annotating a Food Container in the Roboflow Annotation Editor | 70 |
| 50 | A Fully Annotated Creek Frame in the Garbage-Guard Dataset | 71 |
| 51 | The Assembled Garbage-Guard Unit with the Hikvision HiWatch HWI-B140H Camera | 86 |
| 52 | Side View of the Enclosure Showing the Case Fan and the USB Wi-Fi Adapter | 87 |
| 53 | Inside the Enclosure with the Main Parts Labelled | 88 |
| 54 | The Touchscreen Dashboard During the Field Session at the Creek | 89 |
| 55 | Monitor Tab of the Touchscreen Dashboard with Live Detections (TC_SYS_05) | 93 |
| 56 | Dashboard During an Accumulation Alert (TC_SYS_02, TC_SYS_09) | 93 |
| 57 | Drawing the Monitored Area on the Dashboard (TC_SYS_06) | 94 |
| 58 | History Tab with the Stored Alerts and Snapshots (TC_SYS_08) | 94 |
| 59 | Settings Tab with the Threshold, Camera Stream and Model Options (TC_SYS_10) | 95 |

| Figure | Title | Page |
|---|---|---|
| 60 | Accumulation Alert Notification on the Android Application (TC_SYS_09) | 95 |
| 61 | Training and Validation Curves of the v10 Model | 101 |
| 62 | Precision-Recall Curve of the v10 Model on the Validation Set | 102 |
| 63 | Normalized Confusion Matrix of the v10 Model on the Validation Set | 102 |
| 64 | Switching On the Unit with the UPS HAT Power Switch | 108 |
| 65 | Raspberry Pi Desktop After Start-Up | 108 |
| 66 | Monitor Tab with the Live Camera Feed and No Items in the Area | 109 |
| 67 | Accumulation Alert on the Touchscreen with the Limit Set to 3 | 110 |
| 68 | Buzzer Module Inside the Enclosure | 110 |
| 69 | History Tab with the Saved Limit-Reached Record | 111 |
| 70 | Accumulation Alert Notification on the Android Phone | 112 |
| 71 | Monitor Tab of the Android Application During the Alert | 112 |
| 72 | Hand Count and Dashboard Count at Each Calibration Reading | 116 |
| 73 | Live Classification Accuracy by Deployment Day | 118 |
| E1 | Topic Approval at the Proposal Defense | 9 |
| E2 | First Day of Prototype Assembly | 10 |
| E3 | Second Day of Prototype Assembly | 11 |
| E4 | Third Day of Prototype Assembly | 12 |
| E5 | First Independent Test for Data Gathering | 13 |
| E6 | Researchers with the Barangay Officials | 14 |
| E7 | Field Test at Barangay 870, Pandacan, Manila | 15 |
| E8 | Second Independent Test and First Accumulation Alert | 16 |
| E9 | Third Independent Test and Image Collection for Model v10 | 17 |
| E10 | Measuring the Camera Cable from the Barangay Building to the Bridge | 18 |

| Figure | Title | Page |
|---|---|---|
| E11 | First Day of Deployment Test | 19 |
| E12 | Usability Survey with the Barangay Personnel and the Estero Rangers | 20 |
| E13 | Second Day of Deployment Test | 21 |
| E14 | Third Day of Deployment Test | 22 |
| E15 | Final Defense | 23 |
| E16 | Completion of the Final Defense at the TUP Plaza | 24 |
| H1 | Web Dashboard Monitor Tab | 27 |
| H2 | Web Dashboard During an Accumulation Alert | 28 |
| H3 | Drawing the Monitored Area on the Web Dashboard | 29 |
| H4 | Web Dashboard Fullscreen View | 30 |
| H5 | Web Dashboard in Dark Mode | 31 |
| H6 | Web Dashboard History Tab | 32 |
| H7 | Snapshot Viewer on the Web Dashboard | 33 |
| H8 | Delete Confirmation on the Web Dashboard | 34 |
| H9 | Web Dashboard Settings Tab | 35 |
| H10 | Battery Details on the Web Dashboard | 36 |
| H11 | Connected Phones Panel on the Web Dashboard | 37 |
| H12 | Network Setup Page of the Raspberry Pi | 38 |
| H13 | Mobile App Connection Screens | 39 |
| H14 | Mobile App Monitor Screen | 40 |
| H15 | Mobile App Fullscreen View | 41 |
| H16 | Mobile App History Screen | 42 |
| H17 | Mobile App Settings for Detection, Pi Screen and Buzzer | 43 |
| H18 | Mobile App Settings for Alerts, Appearance and Connection | 44 |
| H19 | Mobile App Detail Panels | 45 |
| H20 | Mobile App Alert Notification | 46 |

| Figure | Title | Page |
|---|---|---|
| H21 | Mobile App in Dark Mode | 47 |
| I1 | Cover and Contents of the User Manual | 48 |
| I2 | Problem Finder and Overview of the System | 49 |
| I3 | Outside and Inside of the Unit | 50 |
| I4 | Setting Up at the Creek and Safe Shutdown | 51 |
| I5 | Monitor Tab and Alert Controls | 52 |
| I6 | Monitored Area and History Tab | 53 |
| I7 | Settings Tab and Connecting a Phone | 54 |
| I8 | Phone App Screens and Themes | 55 |
| I9 | Troubleshooting the Unit, Camera and Alerts | 56 |
| I10 | Care, Maintenance and Quick Reference | 57 |

# LIST OF ABBREVIATIONS

| Abbreviation | Definition |
|---|---|
| AI | Artificial Intelligence |
| CCTV | Closed-Circuit Television |
| CNN | Convolutional Neural Network |
| GPIO | General-Purpose Input/Output |
| HAT | Hardware Attached on Top |
| IEC | International Electrotechnical Commission |
| IoT | Internet of Things |
| IPO | Input-Process-Output |
| ISO | International Organization for Standardization |
| mAP | Mean Average Precision |
| NCNN | Neural network inference framework for mobile and embedded devices |
| NVMe | Non-Volatile Memory Express |
| PCIe | Peripheral Component Interconnect Express |
| ROI | Region of Interest |
| RRL | Review of Related Literature |
| RTSP | Real-Time Streaming Protocol |
| SDLC | Software Development Life Cycle |
| SSD | Solid-State Drive |
| UNEP | United Nations Environment Programme |
| UN-SDG | United Nations - Sustainable Development Goals |
| UPS | Uninterruptible Power Supply |
| YOLO | You Only Look Once |
| YOLO26n | You Only Look Once version 26 nano |

## ABSTRACT

Floating garbage that builds up in urban creeks blocks drainage and worsens flooding, yet barangays still rely on walks along the bank to know when a stretch needs clearing. Existing studies detect waste in single images or monitor water levels with sensors, but none counts floating garbage over time within an operator-defined area. This study developed Garbage-Guard, a fixed unit that detects bottles, bags and food containers with a YOLO26n convolutional neural network on a Raspberry Pi 5. It crops the camera feed to a monitored area over the water, tracks each item with ByteTrack, smooths the count over 15 frames and raises an Accumulation Alert on a touchscreen dashboard, a buzzer and an Android application when the count reaches a set threshold. The model was trained over ten runs on a custom dataset, and the system was evaluated under ISO/IEC 25010:2023. On a frozen test set of 807 images, the final model reached a precision of 84.6%, a recall of 67.2% and a mAP@0.5 of 0.744. At a creek in Barangay 870, Pandacan, Manila, the unit ran for 24 hours over six days without a crash, processed each frame in 275ms to 350ms and raised the alert in 10 of 10 trials. Live classification averaged 74.7% and reached 85.0% after the camera was lowered closer to the water, and four end users rated usability 4.40 (Excellent). With its threshold calibrated on site, Garbage-Guard can serve barangays as an early-warning tool, supporting UN Sustainable Development Goals 6, 9 and 11.

**Keywords:** computer vision, floating garbage detection, YOLO26n, edge computing, Raspberry Pi 5

# I. INTRODUCTION

## 1.1 Background of the Study

Urban waterways such as rivers and creeks act as natural drainage, habitat for wildlife, and channels that carry water through cities. Rapid urban growth and rising populations, however, have sharply increased solid waste generation in many countries, the Philippines among them. Tons of solid waste are produced across the country each day, and part of it reaches waterways through poor disposal practices and inadequate collection services (Development Asia, 2024; UNEP, 2023; World Bank, 2022). Creeks and rivers are also major sinks and carriers of pollution from land to sea, which makes them key sites for pollution monitoring (UNEP, 2023; Acot et al., 2022; World Bank, 2022).

A major environmental problem for urban creeks is the steady dumping of solid waste, especially plastics and other non-biodegradable items. In many urban areas, discarded garbage builds up in waterways and blocks the natural flow of water. Rivers and riverbanks hold the highest concentration of litter among the environments studied, since they act as collection points for debris (UNEP, 2023; Acot et al., 2022; Meijer et al., 2021; World Bank, 2022). Creeks and rivers in the Philippines regularly receive waste from nearby communities, which pollutes the water and degrades the environment (Meijer et al., 2021; Robles & Monjardin, 2025; Victoriano et al., 2022). The buildup also slows the current and causes flooding when drainage channels are blocked.

Garbage in creeks usually comes from poor disposal habits and weak waste management. Studies link river and creek pollution to poorly performing solid waste programs and low public awareness (Development Asia, 2024; UNEP, 2023; World Bank, 2022). Households dump waste from their homes and daily activities straight into drains and waterways that empty into creeks and rivers. These conditions call for better monitoring and waste management.

Several methods have been used to monitor garbage pollution and locate where it accumulates in waterways. Traditional approaches rely on manual inspection, which is slow and labor-intensive, particularly over large areas (Robles & Monjardin, 2025; Magyar et al., 2023; Victoriano et al., 2022). Advances in artificial intelligence and computer vision in recent years have opened many uses for object detection, environmental monitoring among them. You Only Look Once (YOLO) is one example of a real-time detection algorithm (Murat & Kiran, 2025; Yaseen, 2024; Kang et al., 2022). Such technologies already appear in monitoring systems, but many current garbage detection systems depend on satellite imagery or very large datasets and are not optimized for small waterways such as creeks.

Computer vision, deep learning, and real-time monitoring together offer a way past these limitations. YOLO can process camera video streams and detect garbage automatically with good accuracy. It can also be linked to platforms that let environmental managers track the amount of garbage and respond quickly to hazards. This study proposes Garbage-Guard, which applies object detection to a camera feed to identify floating garbage in creeks, count how much has built up, and alert the people responsible once it reaches a set level.

Through computer vision for urban environmental sustainability, the research supports three United Nations Sustainable Development Goals: Goal 6 (Clean Water and Sanitation), Goal 9 (Industry, Innovation and Infrastructure), and Goal 11 (Sustainable Cities and Communities) (United Nations, 2025).

## 1.2 Objectives of the Study

### *General Objective*

Development of Computer Vision for Creeks Garbage Detection using Convolutional Neural Network and YOLO26.

### *Specific Objectives*

1. To design a computer vision model for creeks garbage detection using a convolutional neural network and YOLO26n, capable of detecting the following, and to evaluate it under Functional Suitability using precision, recall and mAP@0.5 on a held-out test set:
    a. Bottles
    b. Bags
    c. Food Containers
2. To build a prototype system using a Raspberry Pi 5 computer to detect garbage within an operator-drawn region of a creek, to keep track of each piece of garbage to maintain its identity between frames, to count the number of garbage items in the operator-drawn region and to automatically display an alarm on a touchscreen dashboard and an Android app when a threshold number of items (default is 20 garbage items) is reached.
3. To test the system's performance using ISO/IEC 25010:2023, specifically the following characteristics:
    a. **Functional Suitability:** The accuracy of object classification and accumulation.
    b. **Performance Efficiency**: The real-time responsiveness of Raspberry Pi 5.
    c. **Usability:** The system's operability, user interface and the clarity of its accumulation alerts.

## 1.3 Scope and Limitations of the Study

### *Scope*

This study focuses on the development of Garbage-Guard, an automated, computer vision-based floating garbage detection system deployed on an edge computing device, specifically the Raspberry Pi 5. The primary scope of the system encompasses the following:

- Develop an automated floating garbage detection system, Garbage-Guard, using the YOLO26n algorithm on a Raspberry Pi 5.
- Identify and categorize three specific types of floating solid garbage: Bottles, Bags, and Food Containers.
- Use a custom dataset split 70/10/20 into training, validation and test partitions, with a frozen held-out test set, to achieve a precision of at least 80% with recall and mAP@0.5 reported alongside it.
- Execute item-tracking and region-of-interest logic that counts the floating items present in the monitored area, smoothed over recent frames, and triggers an “Accumulation Alert” once a threshold, by default 20 garbage items, is reached; the threshold is customizable.
- Install a touchscreen dashboard on the unit and offer an Android app for remote alerts, history and settings.
- Conduct a field deployment of the prototype at a local creek in a barangay for real-world environmental testing.
- Evaluate the system’s performance against the ISO/IEC 25010:2023 criteria: Functional Suitability, Performance Efficiency and Usability.

***Limitations***

The Garbage-Guard system has several technical and environmental constraints:

- The system can only detect waste that is floating on the water's surface, and does not detect sunken waste or waste that may be obscured.
- Only three classes are detected (bottles, bags and food containers); other pollutants such as metal, glass or organic material are not.
- The quality of detection depends on camera placement and lighting; too much or too little light affects recognition.
- The quality of detection also drops with distance. The farther the camera is from the water, the smaller each item appears in the frame, and small items such as bottles and bags are missed more often.
- The system is limited to monitoring and early warning; it does not collect garbage or analyze water quality.

### 1.4 Significance of the Study

Garbage-Guard is expected to be useful to different groups who deal with the impact of floating garbage in urban creeks or waterways.

- **Local Government Units (LGUs) and Barangay Officials.** The system helps LGUs and barangay officials plan maintenance work for creeks and waterways by sending alert notifications, capturing a snapshot when garbage builds up and keeping a history of each build-up.
- **Environmental and Flood Control Agencies.** The data collected can help environmental and flood control agencies such as the Department of Environment and Natural Resources (DENR) and the Metropolitan Manila Development Authority (MMDA) in prioritizing and planning for the management of flooding and waste disposal along waterways that lead to the major water bodies.
- **Waste Collection Personnel.** Employees who clear creeks or waterways can save time and make better use of their vehicles and time by knowing the location of garbage buildup and approximately how much garbage is present before heading out.
- **Filipino Residents.** Families living near urban creeks and waterways experience flooding and foul odor when garbage blocks the flow of water and leaves it stagnant. Faster cleanup in response to alerts can help eliminate or reduce these conditions and promote a cleaner and safer environment.
- **TUP Academic Community.** The study shows TUP and the students of the Computer Engineering Technology program how a trained object detection model, deployed on an edge device and linked to a mobile app, can be a low-cost alternative to other computer technologies. It can also be a reference for similar projects.
- **Future Researchers.** The documented design, training process and test results can serve as a base for studies that cover larger water bodies, use other edge computing hardware, and include more classes of garbage.

# II. LITERATURE REVIEW

## 2.1 Conceptual Framework of the Study

**Figure 1**

*Conceptual Framework of Garbage-Guard*

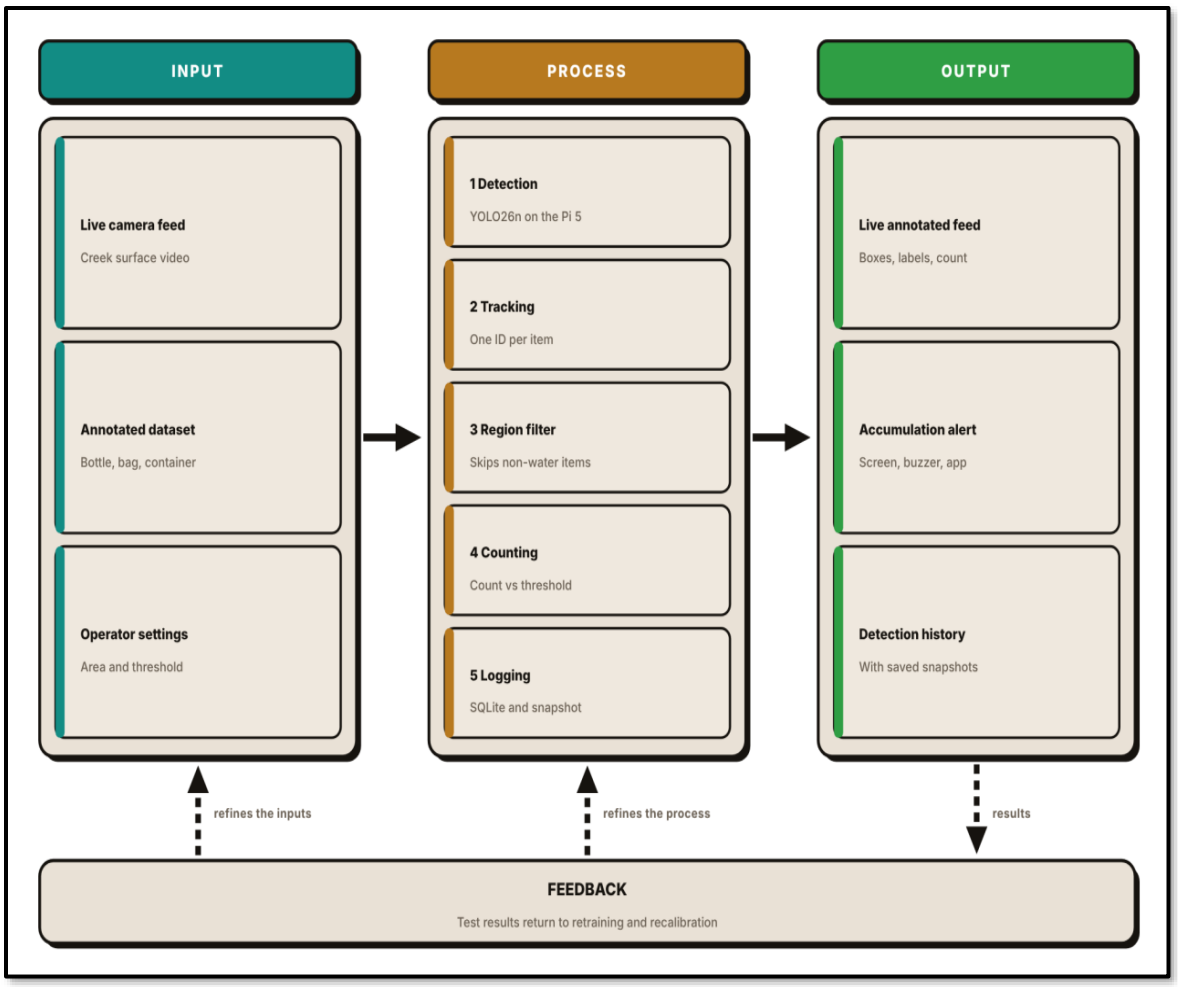



Figure 1 shows the conceptual framework of Garbage-Guard as an Input-Process-Output (IPO) model with a feedback path. The process column is expanded to show every system block, since these blocks are what make the system a monitoring system rather than a frame labeler.

The system has three inputs. The first is the live feed from the network camera mounted above the creek, which supplies frames continuously rather than on a schedule.

The second is the annotated dataset, divided into three classes: bottles, bags, and food containers. The third is the set of operator settings: the region of interest drawn on the water, the number of items that triggers an alert (twenty by default), and the alert preferences.

Processing runs in five stages on a Raspberry Pi 5. Detection uses a YOLO26n convolutional network exported to NCNN, the format the board runs fastest. Tracking keeps one identity per object from frame to frame, so each item holds a steady labeled box instead of flickering in and out of view. Region filtering then discards any detection outside the traced water area, which keeps fixed objects along the bank out of the count. Counting takes the number of objects inside the area in each frame, smooths it as the median of the last 15 frames so that one missed or false detection does not move it, and compares the result with the threshold. Logging writes each alert and its snapshot to a local database and syncs it to the cloud.

The region filter and the smoothed count are what turn a detector into a monitoring system. Twenty bounding boxes in one frame do not prove that twenty objects are present, since a single frame can hold a false detection or miss an object. A system that cannot exclude the bank will also report a concrete wall as garbage for as long as it stays in frame.

The system has three outputs. The touchscreen dashboard shows the live annotated feed with bounding boxes, class labels, and the running count. When the threshold is reached, an accumulation alert appears on the screen, sounds on the buzzer, and is sent to the Android app. Every recorded event is kept in a searchable detection history with its own snapshot, so any reading can be checked afterward.

Evaluation follows ISO/IEC 25010:2023 (functional suitability, performance efficiency and usability), and its results feed back into the input and process stages. Weak per-class results lead to further annotation and retraining, latency measured during operation sets the frame sampling rate, and operator comments on the interface guide the

design of the dashboard. The feedback path is drawn because it is what happened in this study: the detection model was trained over several versions, and the accumulation threshold was tuned on site rather than fixed in advance.

## 2.2 Review of Related Literature and Studies

### *A. Environmental Garbage Issues in Philippine Waters*

According to the United Nations Environment Programme (UNEP, 2023), the World Bank (2022), and Acot et al. (2022), urban waterways such as rivers and creeks are central to the lives of people in urban centers.

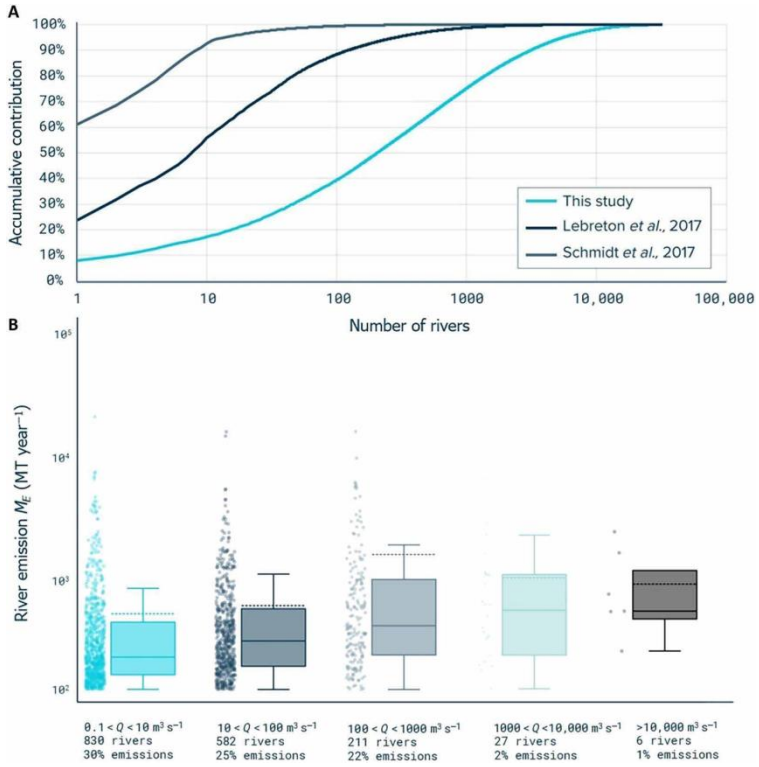
However, due to the effects of rapid urbanization and an ever-increasing population, the amount of solid garbage generated has increased over the years, particularly in the Philippines.

The researchers emphasized that a huge volume of garbage generated in the urban area ends up in the waterway due to inappropriate garbage disposal mechanisms. The researchers' study results revealed that waterways are considered accumulation points for solid garbage originating from the surrounding communities.

However, the researchers' study focused on the garbage management policies and environmental effects rather than the technological aspect. These researchers' studies are related to the present research because they emphasized the need to monitor garbage pollution in the waterway, such as a creek.

**Figure 2**

*Cumulative Contribution of Rivers to Global Plastic Emission*




*Note.* From "More than 1000 rivers account for 80% of global riverine plastic emissions into the ocean," by L. J. J. Meijer, T. van Emmerik, R. van der Ent, C. Schmidt and L. Lebreton, 2021, Science Advances, 7(18). CC BY-NC.

## Rivers as the Pathway for Ocean Plastic (Meijer et al., 2021)

According to Meijer et al. (2021), rivers are the main pathways that carry garbage from land to the ocean (Figure 2). The research used global modeling of environmental data to trace the sources and pathways of plastic in the oceans, and it found that a large share of the plastic entering the ocean comes from rivers in heavily populated areas. However, the research relied on data analysis and modeling, which cannot track the build-up of garbage in an individual creek. It relates to the current study because it shows the role of rivers in carrying garbage.

**Mismanaged Plastic Waste and Urban Flooding (MacAfee & Löhr, 2023)**

MacAfee and Löhr (2023) examined the interaction between mismanaged plastic waste (MPW) and urban flooding under rapid urbanization and climate change. The review indicates that more plastic waste in drainage channels raises the risk of flooding, especially in coastal cities. Its limitation is that it is based on existing literature and offers no technological solution. It is relevant because it emphasizes the need for waste monitoring to prevent environmental and social hazards.

**Identifying Plastic Waste in Marine Ecosystems (Das et al., 2025)**

According to Das et al. (2025), the paper "Plastic Waste in Marine Ecosystems: Identification Techniques and Policy Interventions" reviewed many research papers on marine plastic waste, focusing on identification techniques and policy interventions. The findings showed that polyethylene and polypropylene are the main types of plastic waste, and that the best ways to identify plastic waste are infrared spectroscopy and AI recognition. The limitation of this research is that it relied only on literature and not on experiments. It is still relevant to the current study because it shows that waste in water can be managed.

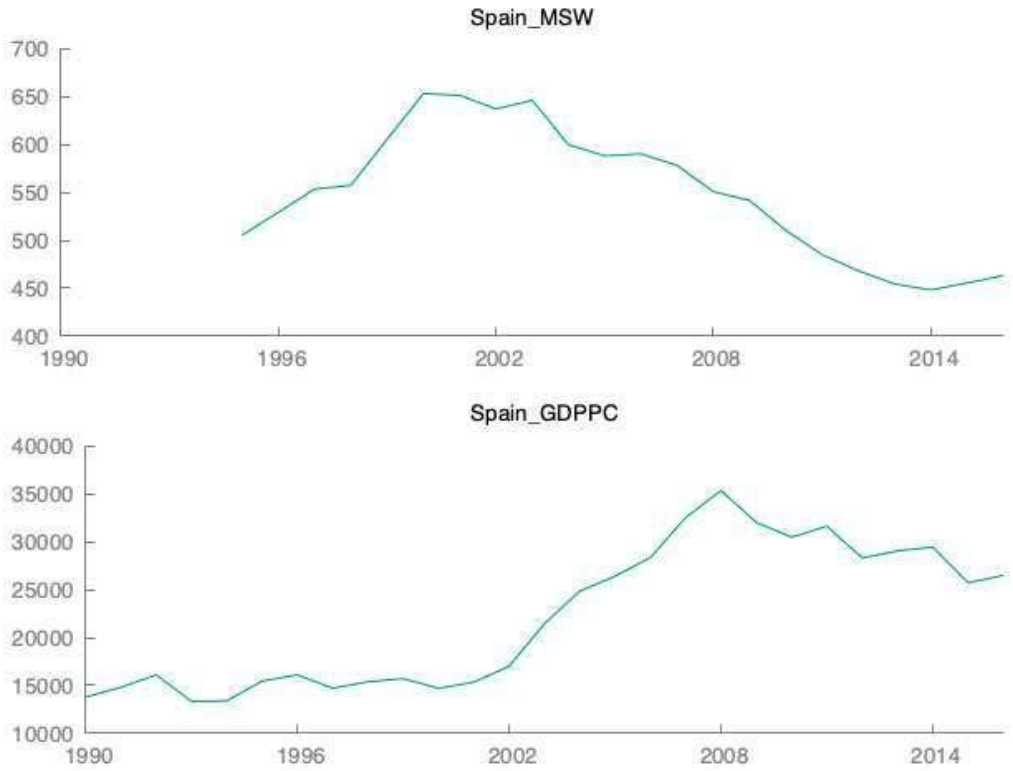
**Wastewater Management in the Philippines (Corpuz, 2025)**

Corpuz (2025) examined the problems of wastewater management in the Philippines, such as a growing population and poor treatment facilities, which leave wastewater untreated and discharged into water bodies. The research used policy and environmental review as its method. Its limitation is that it does not cover technology for monitoring wastewater. It relates to this study because it shows the importance of monitoring water bodies, an objective of Garbage-Guard.

**Solid Waste Management and Economic Growth (Ella et al., 2022)**

**Figure 3**

*Municipal Solid Waste against Gross Domestic Product per Capita*



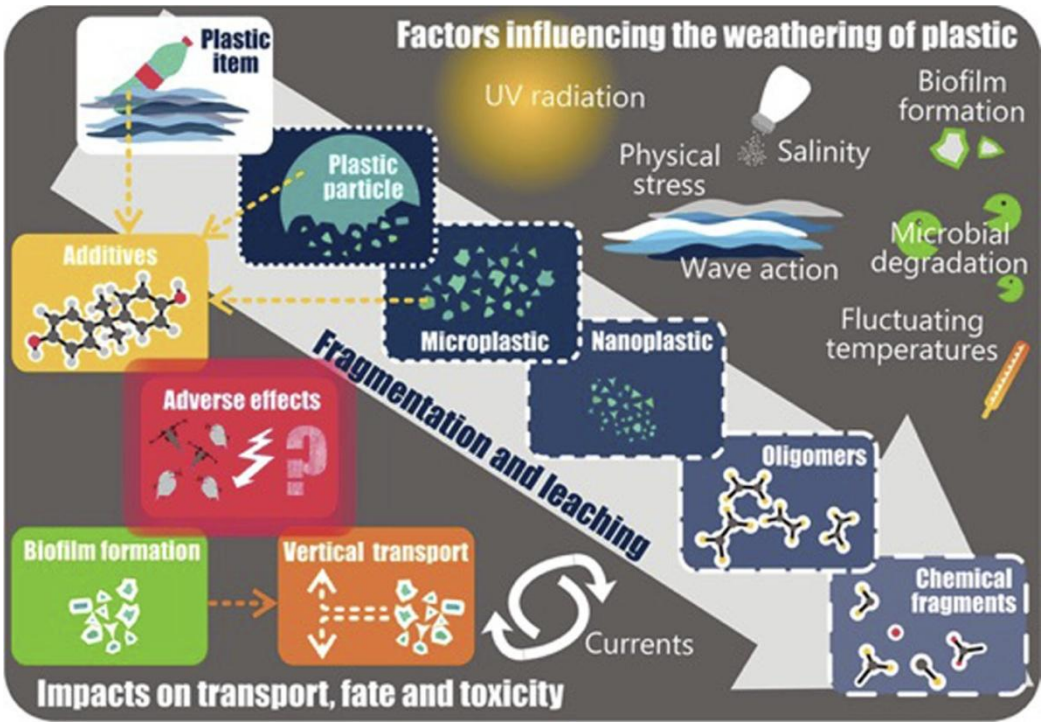

*Note.* From "The impact of solid waste management to the economic growth in selected OECD countries and Philippines," by D. M. Ella, D. M. Lim, B. Lu and R. Cabauatan, 2022, Journal of Economics, Finance and Accounting Studies, 4(1). CC BY.

The study by Ella et al. (2022) on the relationship between municipal solid waste (MSW) management and economic growth in selected OECD countries and the Philippines revealed that MSW has a positive effect on GDP per capita (Figure 3). However, the effect of recycling varies by country. The limitation of the study is its focus on statistical data rather than real-time waste management. This study is similar because it shows the importance of managing waste.

**Plastic and Microplastic Waste in Southeast Asia (Ng et al., 2023)**

**Figure 4**

*Weathering and Fragmentation of Plastic in the Marine Environment*




*Note.* From "Plastic waste and microplastic issues in Southeast Asia," by C. H. Ng et al., 2023, Frontiers in Environmental Science, 11. CC BY.

According to Ng et al. (2023), weak structures and poor waste collection practices in Southeast Asia are increasing plastic pollution in water sources (Figure 4). The research takes a review approach, its data coverage is limited and it depends on secondary sources. It is relevant because it highlights the importance of monitoring and detecting plastic pollution, which Garbage-Guard supports through computer vision.

### *B. Computer Vision Systems*

**Floating Object Detection on Edge Hardware (Li et al., 2024)**

**Figure 5**

*Floating Objects Detected on a Water Surface*



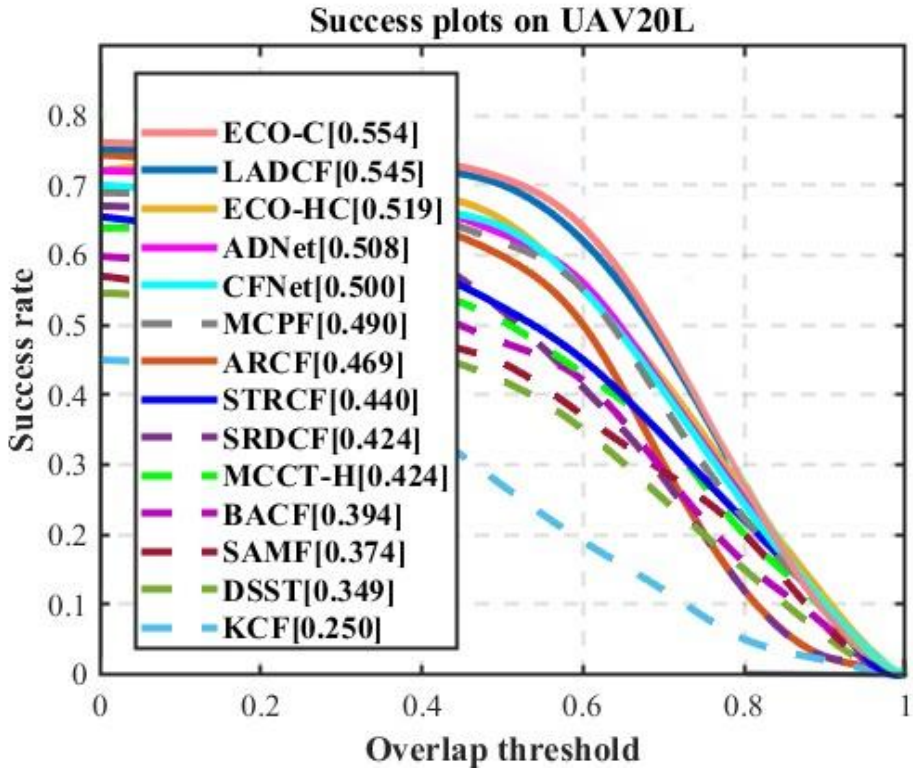
*Note.* From "Detection of floating objects on water surface using YOLOv5s in an edge computing environment," by H. Li et al., 2024, Water, 16(1), 86. CC BY.

Li et al. (2024) addressed the problem this study also faces, which is running a detector on hardware small enough to sit beside the water rather than in a server room (Figure 5). Their improved YOLOv5s model raised detection accuracy by 15.7% over the baseline, cut false detections on small targets by 83%, and sustained 33 frames per second on an edge device. Two findings carry directly into Garbage-Guard. The first is that a small YOLO variant is sufficient for floating-object detection, which supports the choice of YOLO26n over a heavier model. The second is that accuracy is lost on small and distant objects, which matches the weakness this study observes at the creek, where floating bags are the least reliably detected class.

**Computer Vision Detection and Visual SLAM (Chen et al., 2024)**

**Figure 6**

*Success Rates of Visual Tracking Algorithms on a Benchmark Set*



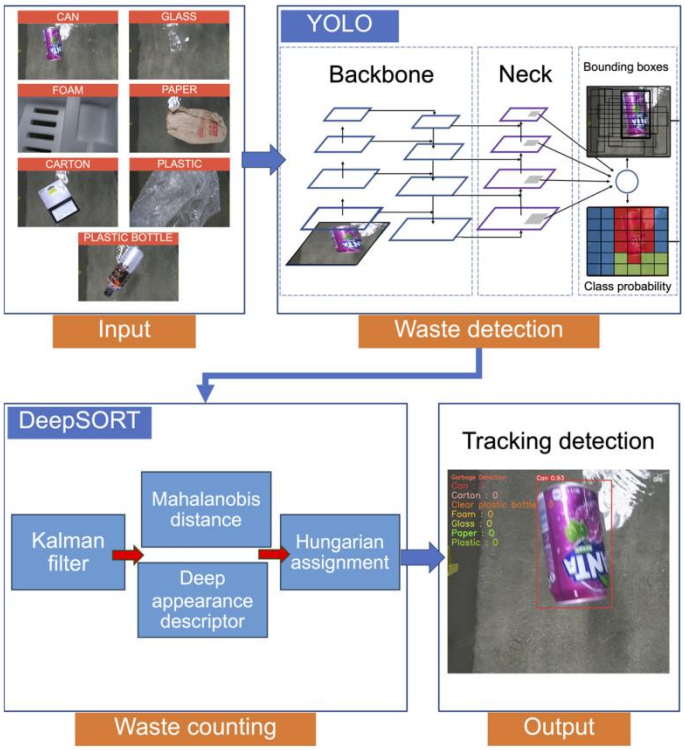

*Note.* From "A survey of computer vision detection, visual SLAM algorithms, and their applications in energy-efficient autonomous systems," by L. Chen et al., 2024, Energies, 17(20), 5177. CC BY.

Chen et al. (2024) describe computer vision as a foundation of intelligent systems and AI technologies (Figure 6). It can replace traditional sensors as the main source of input data, acting as a pair of intelligent "eyes" that processes data to drive unmanned operations such as autonomous driving, obstacle avoidance, and detection tracking. The study is relevant because the researchers use computer vision to detect waste in creeks in real time.

**Detecting and Tracking River-Floating Waste (Nunkhaw & Miyamoto, 2024)**

**Figure 7**

*Detection and Tracking Pipeline for River-Floating Waste*



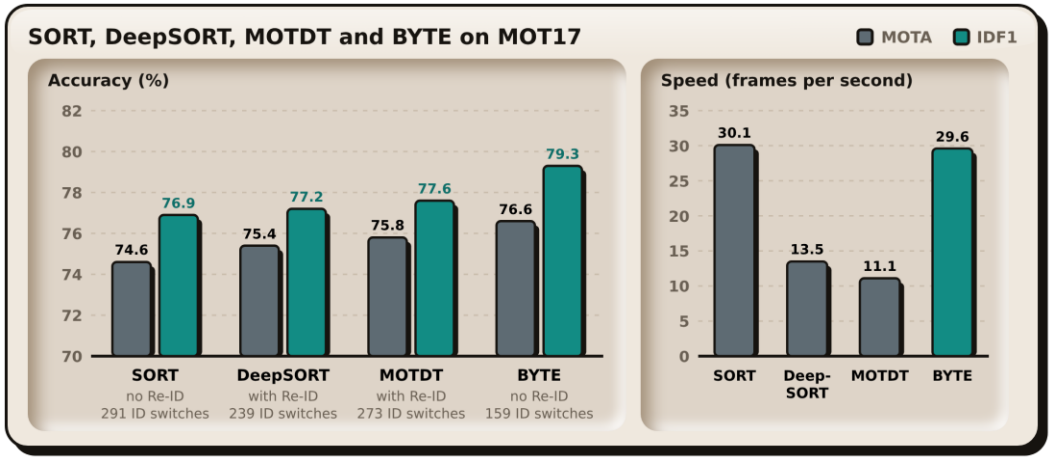

*Note.* From "An image analysis of river-floating waste materials by using deep learning techniques," by M. Nunkhaw and H. Miyamoto, 2024, Water, 16(10), 1373. CC BY.

Nunkhaw and Miyamoto (2024) combined a YOLOv5 detector with DeepSORT tracking to analyze river-floating waste, reporting a mean average precision at or above 88% and roughly 80% accuracy across three scenarios (Figure 7). Theirs is the published method closest to the one used here, and it makes clear that detection alone is not enough on moving water. An item moving through the frame must keep a single identity in every frame, or it is counted again in each one. Garbage-Guard reaches the same conclusion and solves it the same way, with a tracker, using ByteTrack in place of DeepSORT.

**Tracking Every Detection Box with ByteTrack (Zhang et al., 2022)**

**Figure 8**

*Tracking Accuracy and Speed of Four Association Methods on the MOT17 Validation Set*



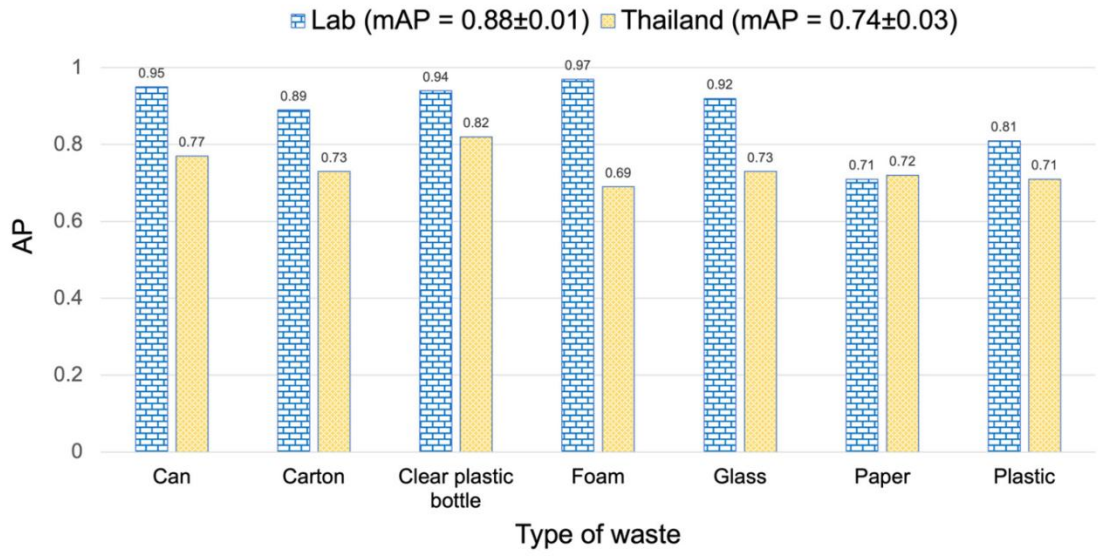

*Note.* Data from "ByteTrack: Multi-object tracking by associating every detection box," by Y. Zhang et al., 2022, Computer Vision – ECCV 2022, Table 2. BYTE is shown without Re-ID.

Zhang et al. (2022) proposed ByteTrack, a tracker that matches almost every detection box from one frame to the next, including the low-confidence boxes that other trackers discard, and links them by motion and box overlap instead of appearance. On the MOT17 validation set its association method, BYTE, reached a MOTA of 76.6% and an IDF1 of 79.3% with 159 identity switches, against 75.4%, 77.2% and 239 for DeepSORT, and it ran at 29.6 frames per second against 13.5 for DeepSORT, which needs a separate appearance (Re-ID) network (Figure 8). The study is the reason Garbage-Guard uses ByteTrack rather than the DeepSORT tracker of Nunkhaw and Miyamoto (2024). Floating items that are partly covered or half under the water tend to receive low detection scores, which is the case ByteTrack was designed to keep, and without an appearance network it leaves more of the processing time of the Raspberry Pi 5 for detection.

**River Waste Detection in Urban Canals (Nunkhaw et al., 2025)**

**Figure 9**

*Detection Accuracy in the Laboratory Compared with the Field*



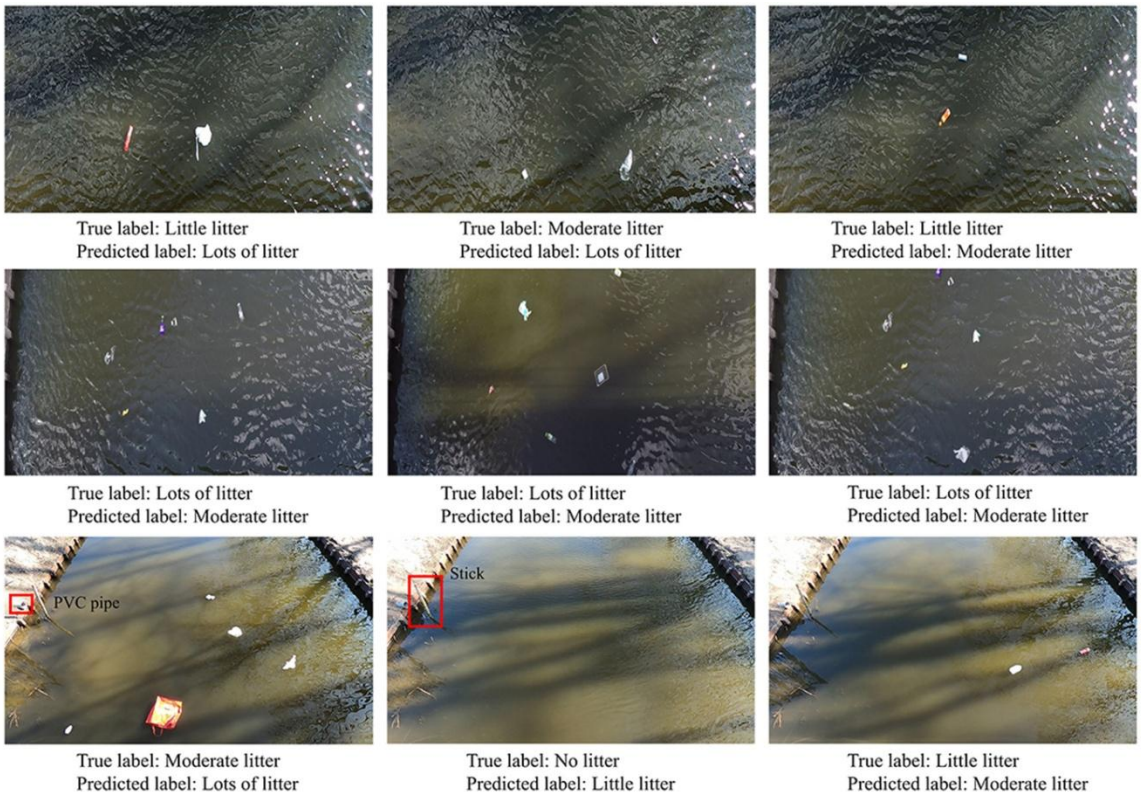

*Note.* From "Enhancing river waste detection with deep learning and preprocessing: A case study in the urban canals of the Chao Phraya River," by M. Nunkhaw, D. Chitwatkulsiri and H. Miyamoto, 2025, Water, 17(22), 3193. CC BY.

Nunkhaw et al. (2025) extended that work to the urban canals of the Chao Phraya River, training a YOLO detector on laboratory flume images and then adapting it to real canal conditions through a preprocessing pipeline (Figure 9). The study bears on Garbage-Guard in two ways. It is set in Southeast Asian urban waterways, where waste composition and water conditions resemble a Metro Manila creek more closely than European or North American river studies do. It also documents the gap between performance on controlled images and performance in the field, which is the same gap this study reports between its full test partition and its creek-only subset.

**An Open Dataset for Floating Litter Detection (Jia et al., 2023)**

**Figure 10**

*Floating Litter Images with True and Predicted Labels*



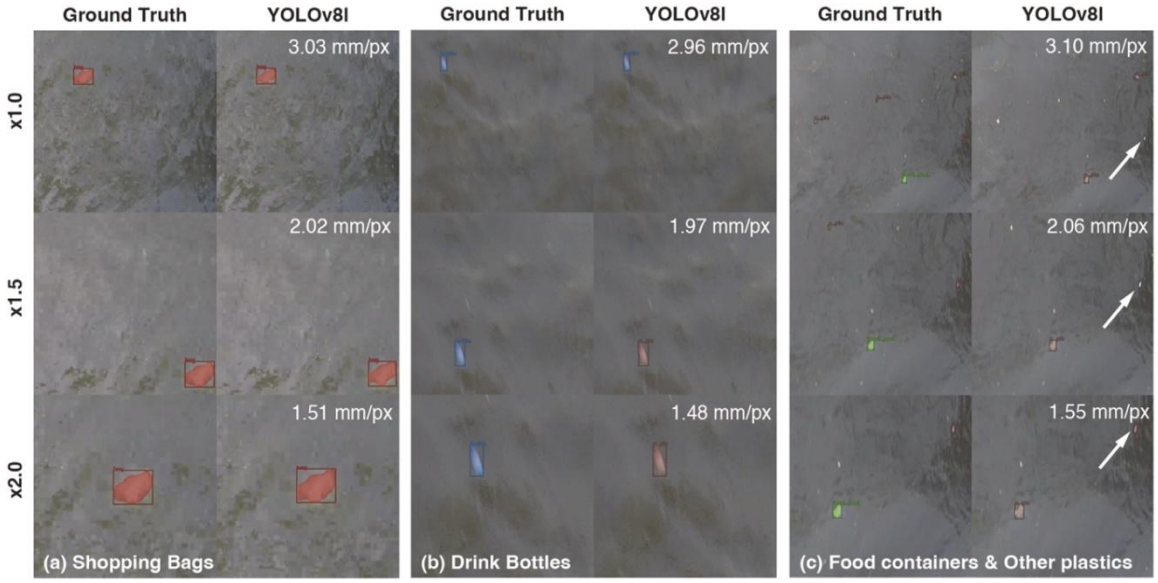

*Note.* From "Advancing deep learning-based detection of floating litter using a novel open dataset," by T. Jia, A. J. Vallendar, R. de Vries, Z. Kapelan and R. Taormina, 2023, Frontiers in Water, 5. CC BY.

To overcome the lack of public training data for floating litter detection, Jia et al. (2023) created an open training dataset of floating litter and tested the detection performance it supports (Figure 10). The issue they raise is the one that drove the data collection for this study: no available dataset matched the kinds of waste found in Philippine creeks. Their work gives a published reason for building a new dataset instead of adapting an existing one.

**Segmentation Compared with Detection for Floating Macroplastic (Kataoka et al., 2024)**

**Figure 11**

*Ground Truth Compared with Model Output for Three Waste Classes*




*Note.* From "Instance segmentation models for detecting floating macroplastic debris from river surface images," by T. Kataoka, T. Yoshida and N. Yamamoto, 2024, Frontiers in Earth Science, 12. CC BY.

Kataoka et al. (2024) compared instance segmentation models for floating macroplastic in river surface images (Figure 11). Segmentation returns a precise outline of each object rather than a rectangle around it, which is more informative but considerably more expensive to compute. Their results support the decision taken here to use bounding-box detection instead. For counting accumulation against a threshold, what matters is the position and identity of an item rather than its exact outline, and the computation saved is what allows the model to run on a Raspberry Pi at the waterside.

**Classifying Waste Fragments with Convolutional Features (Nežerka et al., 2023)**

Nežerka et al. (2023) addressed the improper sorting of construction and demolition waste, which carries significant environmental and economic costs, by developing a machine-learning-assisted procedure that recognizes waste fragments with an RGB camera. Their convolutional network reached an overall accuracy of 85.9%, and classifiers built on selected texture features reached up to 92.3%, offering a path to efficient and accurate sorting. The study relates to the present research through its use of computer vision to detect waste.

**Lane Detection under Varying Light (Al Noman et al., 2022)**

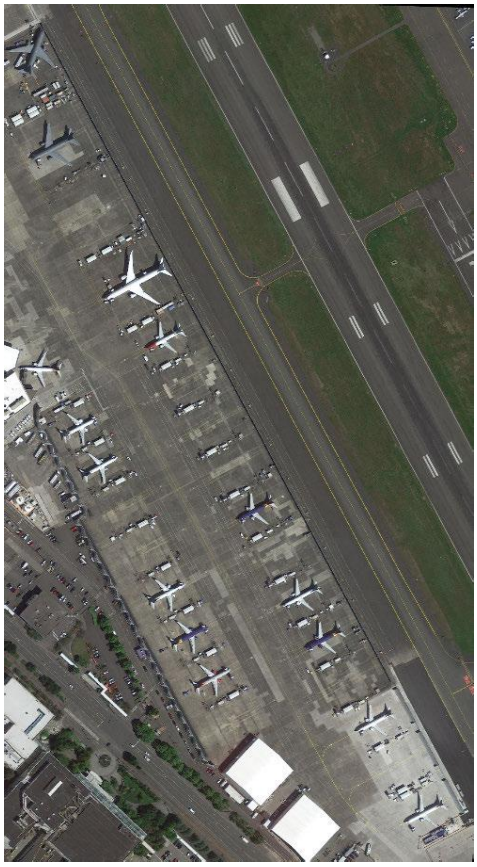
In the study by Al Noman et al. (2022), "A computer vision-based lane detection technique using gradient threshold and hue-lightness-saturation value for an autonomous vehicle," the system uses computer vision to identify road lines. The left and right lanes are found with a sliding-window search. The system works with 96% accuracy and a positive rate of 0.640%. Its accuracy and efficiency could be improved by adding computing features and applying deep learning algorithms.

**Object Detection in Overhead Imagery (Kang et al., 2022)**

According to Kang et al. (2022), computer vision has improved the processing of objects in high-resolution overhead images from drones, airplanes, and satellites. It has been applied to crop analysis, economic analysis, and disaster tracking. Object detection in overhead images remains difficult, however, because of image volume, resolution, small object size, and complex backgrounds.

**Figure 12**

*Overhead Imagery of the Kind Used for Object Detection*



*Note.* From "A survey of deep learning-based object detection methods and datasets for overhead imagery," by J. Kang, S. Tariq, H. Oh and S. S. Woo, 2022, IEEE Access, 10. CC BY.

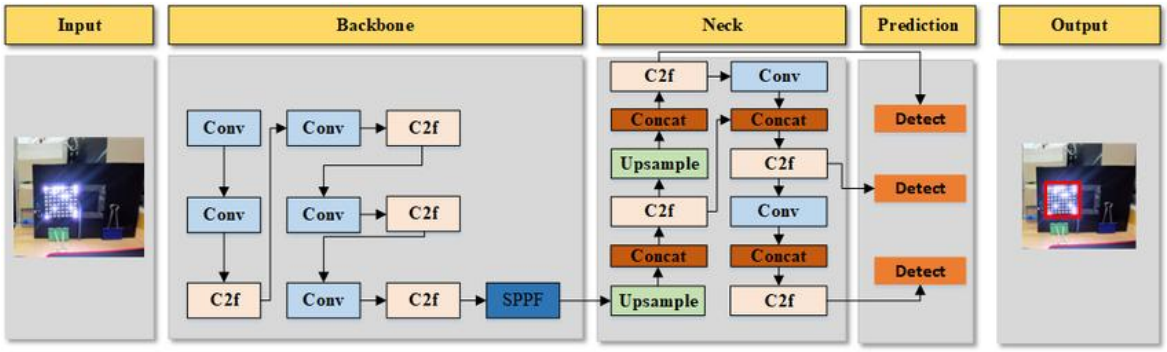
## A Review of YOLO Versions for Object Detection (Murat & Kiran, 2025)

Murat and Kiran (2025) described how deep learning transformed object detection algorithms (Figure 12). YOLO is among the most popular, known for fast and accurate real-time detection. The authors reviewed YOLO versions from YOLOv1 to YOLOv11 and described the changes in architecture and performance across them.

**The Internal Architecture of YOLOv8 (Yaseen, 2024)**

**Figure 13**

*Architecture of the YOLOv8 Detector*



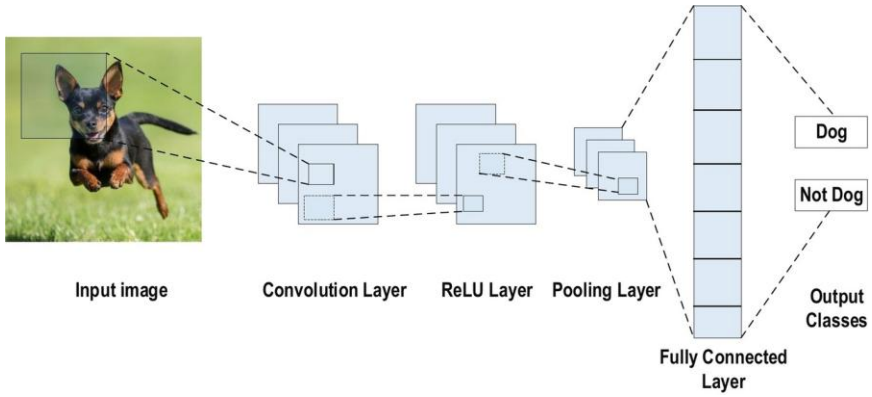

*Note.* From "What is YOLOv8: An in-depth exploration of the internal features of the next-generation object detector," by M. Yaseen, 2024, arXiv. CC BY.

Yaseen (2024) examined YOLOv8 and its advances over earlier versions (Figure 13). It uses architectures such as CSPNet and FPN+PAN together with an anchor-free design to improve feature extraction and multi-scale detection. Evaluations on several datasets, including Microsoft COCO, show its accuracy and real-time detection capability.

**Convolutional Neural Networks for Image Classification (Alzubaidi et al., 2021)**

**Figure 14**

*Architecture of a Convolutional Neural Network for Image Classification*




*Note.* From "Review of deep learning: Concepts, CNN architectures, challenges, applications, future directions," by L. Alzubaidi et al., 2021, Journal of Big Data, 8, 53. CC BY.

Alzubaidi et al. (2021) reviewed deep learning with a focus on the convolutional neural network (CNN). A CNN for image classification stacks convolution layers, each followed by a pooling layer, and ends with fully connected layers (Figure 14). Each convolution layer slides a set of filters over the image to produce feature maps, and each pooling layer sub-samples those maps, so the network learns its own features from labeled images instead of relying on features chosen by hand. The review names weight sharing, which reduces the number of trainable parameters, as a main advantage of the CNN over earlier networks. The study relates to the present research because YOLO26n, the detector in Garbage-Guard, is built on the same convolutional layers, with a detection head that predicts boxes and classes in place of the final classification layers.

### *C. Smart Environmental Monitoring Systems*

**Real-Time Environmental Monitoring with IoT Sensors (Narayana et al., 2024)**

In the research conducted by Narayana et al. (2024), environmental conditions are made up of both living and non-living components that affect human life. The non-living components include land, water, stones, atmosphere etc., and living components include organisms like plants, woodlands, fisheries and wildlife etc. Environmental parameters, like temperature, humidity, pressure, air quality, greenhouse gases, wind speed and direction, rainfall, soil pH, soil moisture, water availability, light, turbidity, etc., influence people's lives, especially in the case of workers who practice activities such as agriculture, navigation, traveling, and mining.

In addition to that, to address the issue of monitoring environmental variables and to mitigate the challenges that come with monitoring environmental parameters, IoT technology was used to develop IoT-based smart environmental monitoring systems. This technology uses microcontrollers and other technologies to monitor environmental parameters from different locations and send the data to a website for monitoring and observation remotely using IoT technology. IoT technology is a dynamic system that observes and responds to environmental variables and conditions (Narayana et al., 2024).

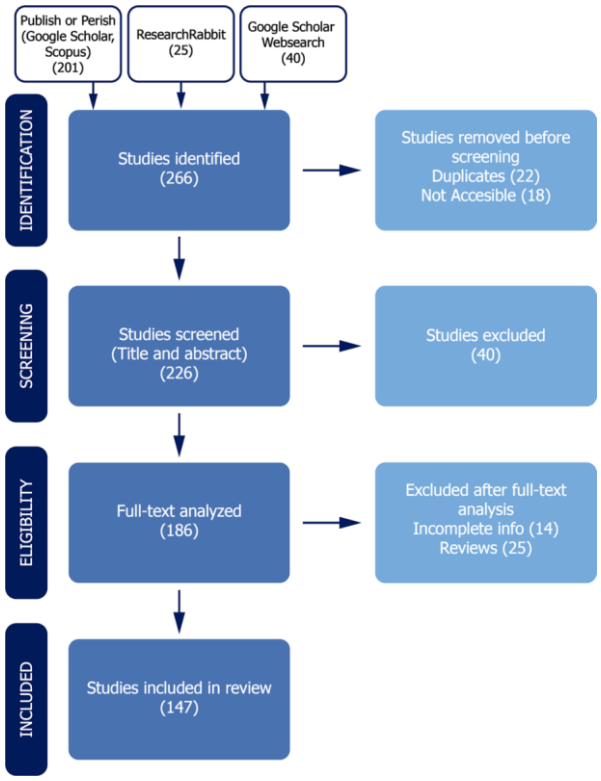
### Sensors in Smart City Systems (Zeng et al., 2024)

In the study of Zeng et al. (2024), the integration of IoT technology into smart city systems has several advantages in the management of the environment, as it allows real-time monitoring of important parameters such as air quality, water quality, noise levels, soil quality, and others. This has enabled city administrators to proactively manage the resources of the city in a cost-effective manner. These parameters have collectively proven the importance of IoT sensors in achieving sustainability in the environment.

### IoT and Artificial Intelligence in Air Quality Monitoring (Garcia et al., 2025)

**Figure 15**

*Selection of Studies for a Systematic Review of Air Quality Monitoring*




*Note.* From "Advancements in air quality monitoring: A systematic review of IoT-based air quality monitoring and AI technologies," by A. Garcia, Y. Saez, I. Harris, X. Huang and E. Collado, 2025, Artificial Intelligence Review, 58. CC BY.

Garcia et al. (2025) show how Internet of Things (IoT) technologies have improved environmental monitoring, with networks of low-cost sensors producing large volumes of real-time air quality data (Figure 15). Pairing these IoT systems with machine learning and deep learning makes environmental assessment as accurate as possible. IoT-based systems can monitor air quality parameters such as PM, $CO_2$, $NO_2$, ozone, and VOCs, which makes them an efficient means of thorough environmental surveillance. Integrated IoT and artificial intelligence systems show how such tools can improve environmental management decisions.

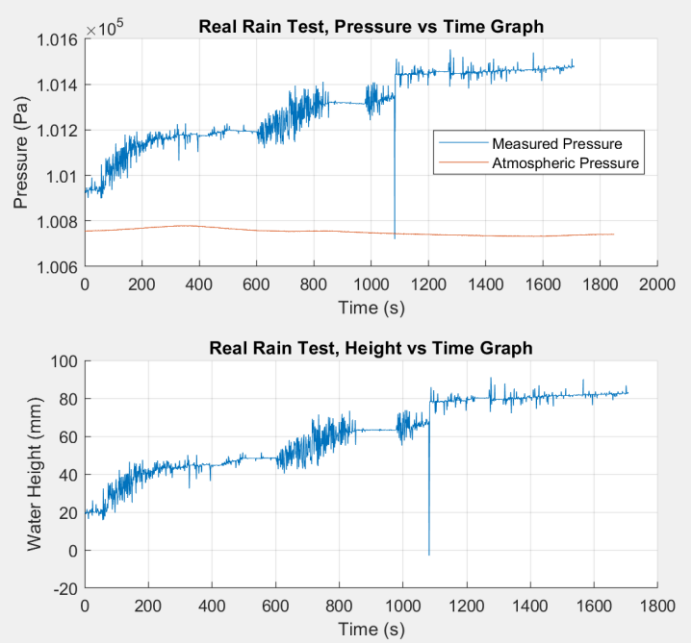
**Machine Learning and IoT for Pollution Prediction (Gryech et al., 2024)**

In the recent literature of Gryech et al. (2024), it has been observed that there is a rising trend towards the inclusion of IoT technologies with machine learning methods for better outdoor air pollution prediction and forecasting. Though the traditional air pollution stations are accurate, the high cost of these stations has encouraged the development of IoT-based low-cost sensors, along with machine learning methods that are capable of predicting the levels of outdoor air pollution through time-series, feature-based, and spatio-temporal methods, although the limitations of the lack of diversity in the data and the context-specific features have to be addressed, as identified by the recent literature.

**Urban Flood Monitoring with LoRaWAN Sensors (Te et al., 2024)**

**Figure 16**

*Water Level and Pressure Recorded during a Rainfall Event*



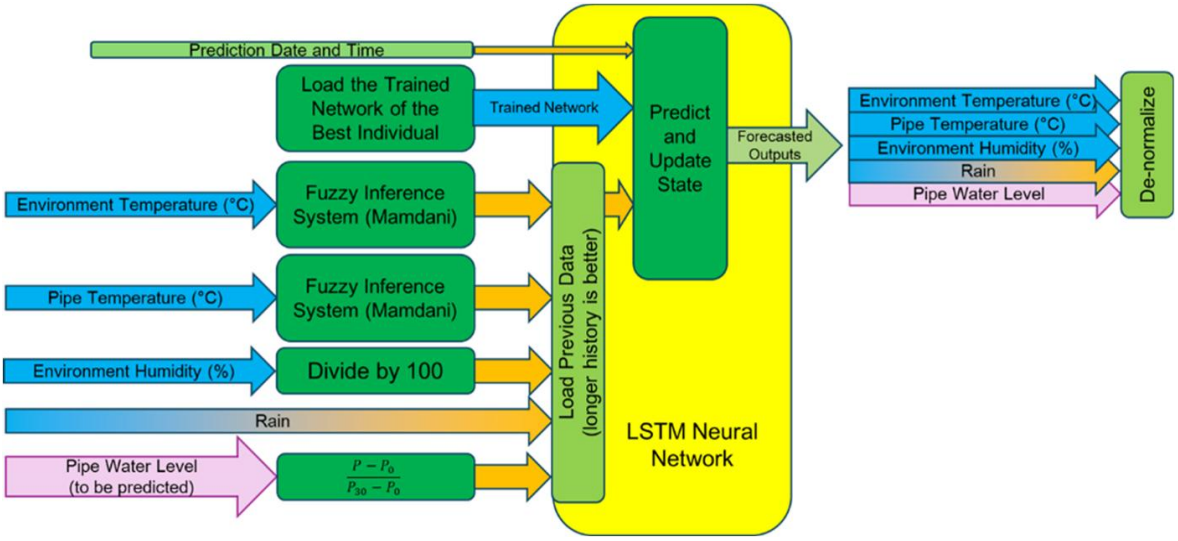

*Note.* From "A smart IoT urban flood monitoring system using a high-performance pressure sensor with LoRaWAN," by M. C. L. Te et al., 2024, HighTech and Innovation Journal, 5(4). CC BY.

Flooding is a significant environmental problem in the Philippines, driven by monsoon rains, local topography, and inadequate drainage. It causes heavy loss of life and property, especially in urban areas. To reduce delays in flood monitoring, Te et al. (2024) proposed a smart IoT-based flood monitoring system for an urban area that combines efficient pressure sensors, LoRaWAN, and solar power (Figure 16). Their review of the background shows a need to apply IoT to disasters, since efficient monitoring would improve flood response (Te et al., 2024).

**Secured Flood Monitoring and Forecasting (Arante et al., 2025)**

**Figure 17**

*Neuro-Fuzzy Network for Flood Forecasting*




*Note.* From "Development of a secured IoT-based flood monitoring and forecasting system using genetic-algorithm-based neuro-fuzzy network," by H. R. C. Arante et al., 2025, Sensors, 25(13), 3885. CC BY.

The monitoring and forecasting of floods in the Philippines are important due to the occurrence of heavy rains and typhoon scenarios, which cause the rapid rise of water levels. In this regard, Arante et al. (2025) contextualized their research by highlighting the limitations of the existing flood monitoring systems, which are not effective in the Philippines, and the need to develop a secure IoT infrastructure to transmit data on water levels to aid in the prediction of floods. The authors proposed a neuro-fuzzy LSTM machine learning model to improve the accuracy of flood event prediction (Figure 17), which is a form of smart environmental monitoring system in a water-related disaster scenario.

## 2.3 Review of Related Patents

Table 1 lists the ten related patents reviewed in this section, with their inventors and year of filing or grant.

## Table 1

*Table of Related Patents*

| Patent Title | Author | Adopted Feature | Enhanced Feature |
|---|---|---|---|
| Garbage Detection Method, Related Equipment and Device | Li & Wang (2020) | Image recognition to detect garbage | Enables automatic garbage detection in areas. |
| Vision garbage classification system based on TensorFlow | Wu et al. (2019) | The AI system classifies garbage using TensorFlow. | Uses a similar AI system and improves garbage recognition with a deep learning model, YOLO26n |
| Smart City Waste Collection and Management using IoT | Chordiya et al. (2020) | IoT sensors monitor garbage collection systems. | Using a camera for real-time monitoring of garbage |
| Cleaning Unit Using Deep Learning for Floating Trash Detection | *Kim et al.* (2019) | Uses object detection to identify trash | Enhances garbage identification using AI algorithms. |
| Drainage monitoring apparatus and system | Campbell et al. (2022) | Water level sensors monitor drainage conditions and detect faults. | Detects garbage as a cause of drain blockage in waterways. |
| System for monitoring water drainage | Den Hartog & Voncken (2017) | Sensors monitor water levels in drain systems. | Detects water blockage and water conditions. |
| Detection and monitoring of waste disposal using artificial intelligence | Liu et al. (2022) | AI monitors garbage in the environment | Tracks garbage data using automated analysis. |
| Flood warning system | Torres et al. (2024) | Sensors detect rising water levels. | Alerts users when garbage build-up may cause flooding |
| Method and System for Flood Early Warning in Smart Cities Based on the Internet of Things | Shao et al. (2023) | IoT sensors monitor rainfall and water levels | Garbage detection as an early indicator of flooding risk |
| Apparatus and Method for Waste Monitoring and Analysis | Balachandran & Talavera (2023) | Sensors and cameras monitor garbage levels in containers. | Automates garbage detection using computer vision |

**Figure 18**

*Garbage Detection Method, Related Equipment and Device*

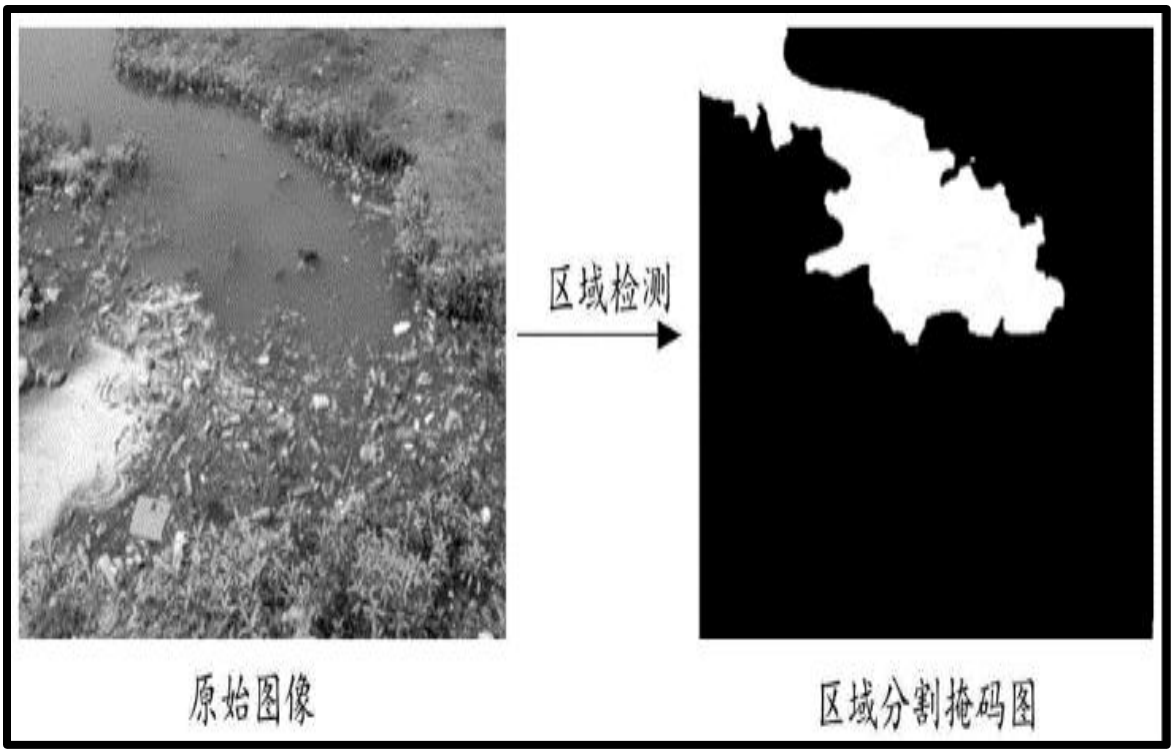



Figure 18 shows a garbage detection method using image recognition, patented by Li and Wang (2020) in China. The system takes pictures of monitored rivers and processes them to find garbage.

This technology can be used in different urban areas such as lakes and parks. It analyzes captured images to determine whether garbage is visible in the scene. The patent relates to Garbage-Guard because it uses computer vision to detect garbage, as the researchers' system does in creeks.

**Figure 19**

*Vision Garbage Classification System and Classification Method Based on TensorFlow*

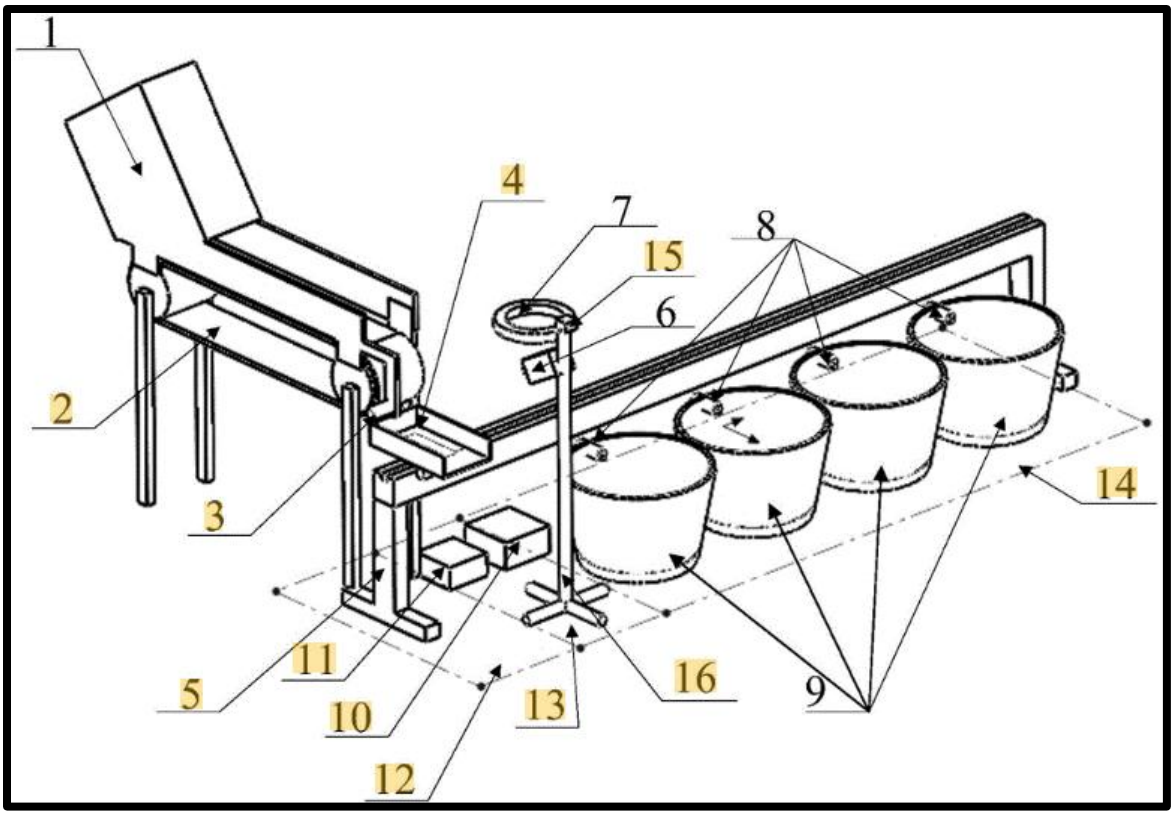



Figure 19 shows the Vision Garbage Classification System and Classification Method Based on TensorFlow by Wu et al. (2019), a patent that uses computer vision. The system feeds images of garbage items into a model trained on garbage images. Its methods include data collection, image processing and automatic classification of garbage categories.

This patent is similar to the present study because it also applies computer vision and machine learning to automated garbage detection.

**Figure 20**

*WCM-Smart City: Waste Collection and Management under Smart City Using IoT*

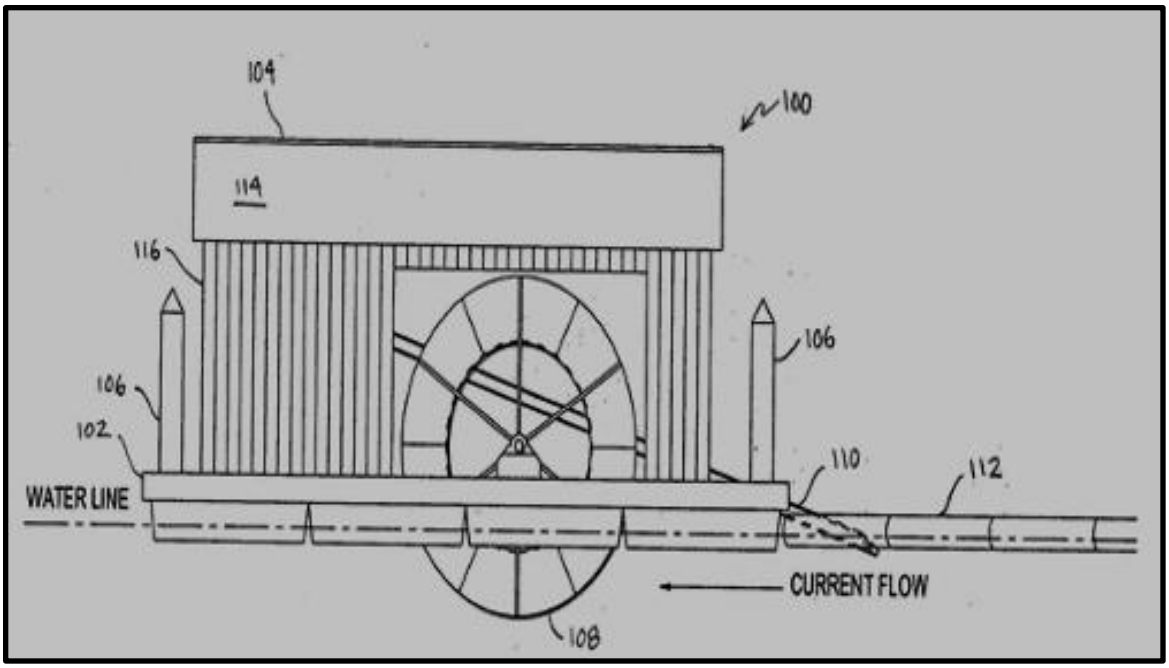



Figure 20 is a patent by Chordiya et al. (2020) for a waste collection system made for gathering garbage in rivers and streams. This invention uses a conveyor mechanism that collects waste and transports it into a container. It incorporates monitoring systems with sensors that detect waste levels and can transmit that information to a platform.

This patent demonstrates automated methods for managing garbage, and it is similar to the researcher's Garbage-Guard system that aims to find garbage in creeks using computer vision technology.

**Figure 21**

*Cleaning Unit Using Deep Learning for Floating Trash Detection*

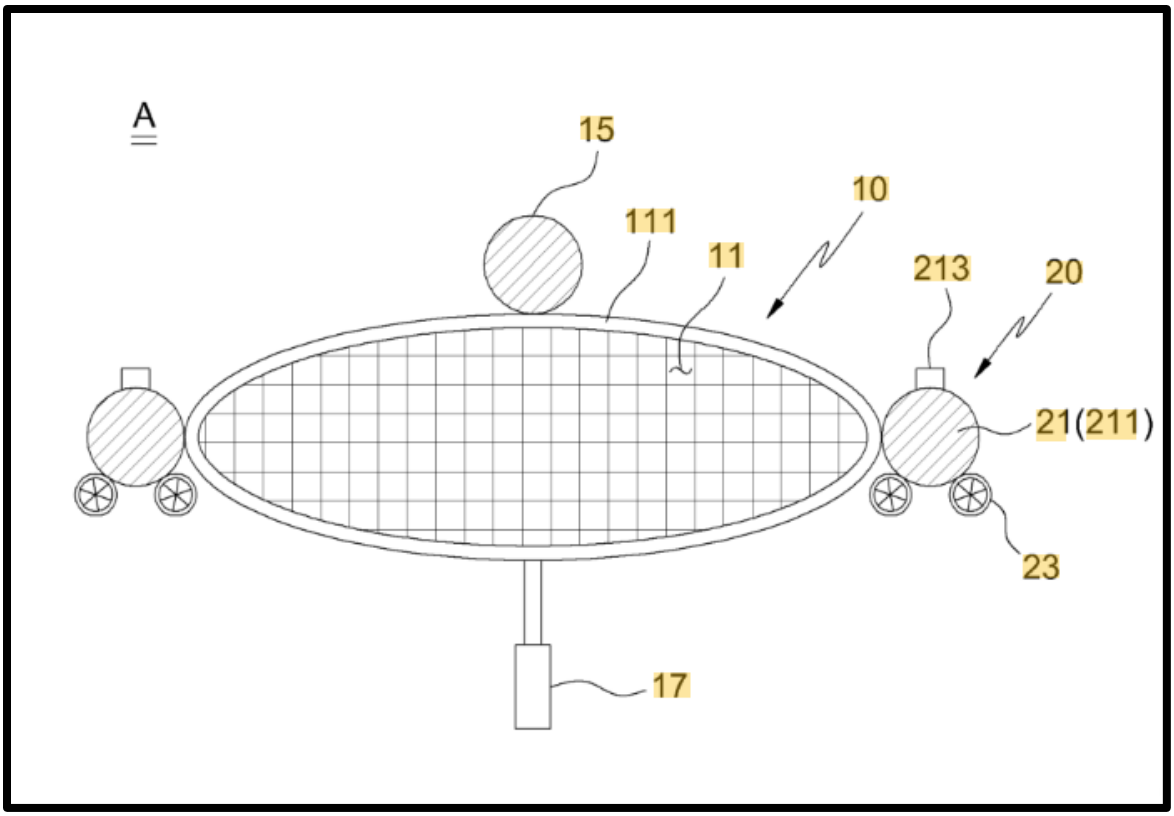



Figure 21 is a related patent from South Korea by Kim et al. (2019): a water-surface cleaning unit that uses deep learning to detect floating garbage. The system integrates a camera module with a deep-learning model which helps in locating garbage on water surfaces. Then, it activates a mechanism that collects the debris from the water.

This invention is relevant to the present study because it also uses artificial intelligence to detect garbage on water.

**Figure 22**

*System for Monitoring Water Drainage*

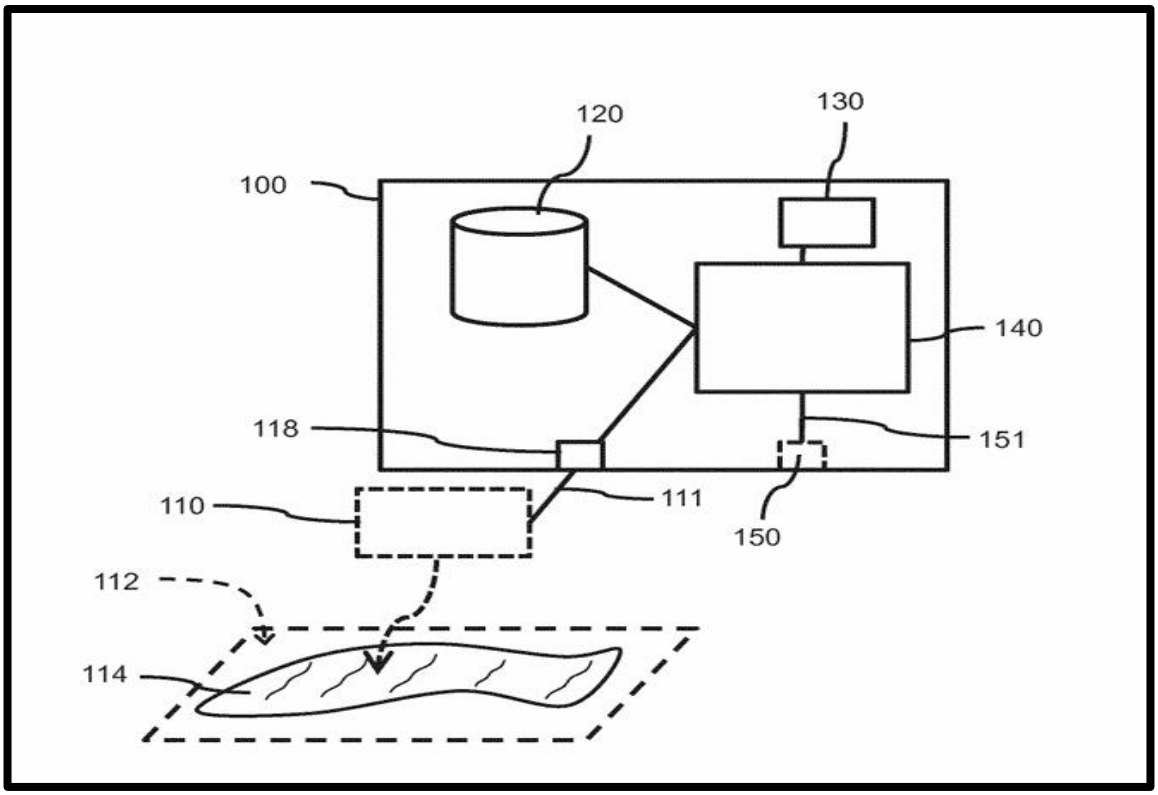



Figure 22 shows a smart drainage patent by Den Hartog and Voncken (2017), a system that monitors water drainage with environmental sensors. The system collects data on water present on surfaces and compares it with predicted water drainage conditions based on what happens during rainfall. If the detected water remains longer than usual, the system makes warning signals about potential drainage problems.

This type of technology integrates sensors, weather information, and data processing to check drainage conditions. This type of patent is related to Garbage-Guard because both systems monitor water environments; Garbage-Guard does so by detecting garbage that collects in creeks and may cause drainage problems.

**Figure 23**

*Drainage Monitoring Apparatus and System*

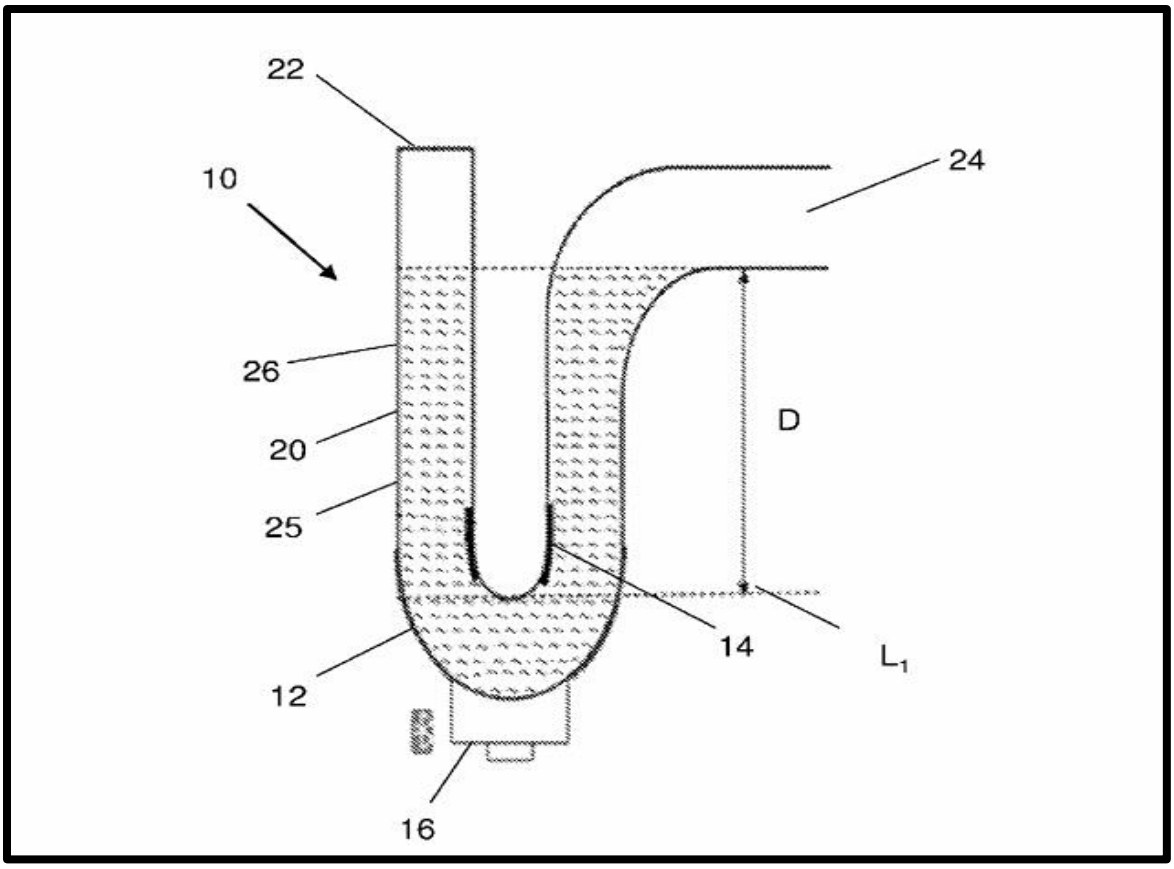



Figure 23 is another smart drainage monitoring patent, by Campbell et al. (2022), which describes a drain monitor apparatus designed to look at water levels within building drainage systems. The invention uses a water trap equipped with a water level sensor that checks the level of water inside the drainage trap. When the water level drops to a specific point, the sensor sends a signal to a system or mobile application that can make audible alerts to inform that there are drainage faults.

This technology relates to Garbage-Guard because both monitor drainage systems. Instead of water levels, however, Garbage-Guard uses computer vision to detect the amount of garbage in creeks.

**Figure 24**

*Detection and Monitoring of Waste Disposal Using Artificial Intelligence*

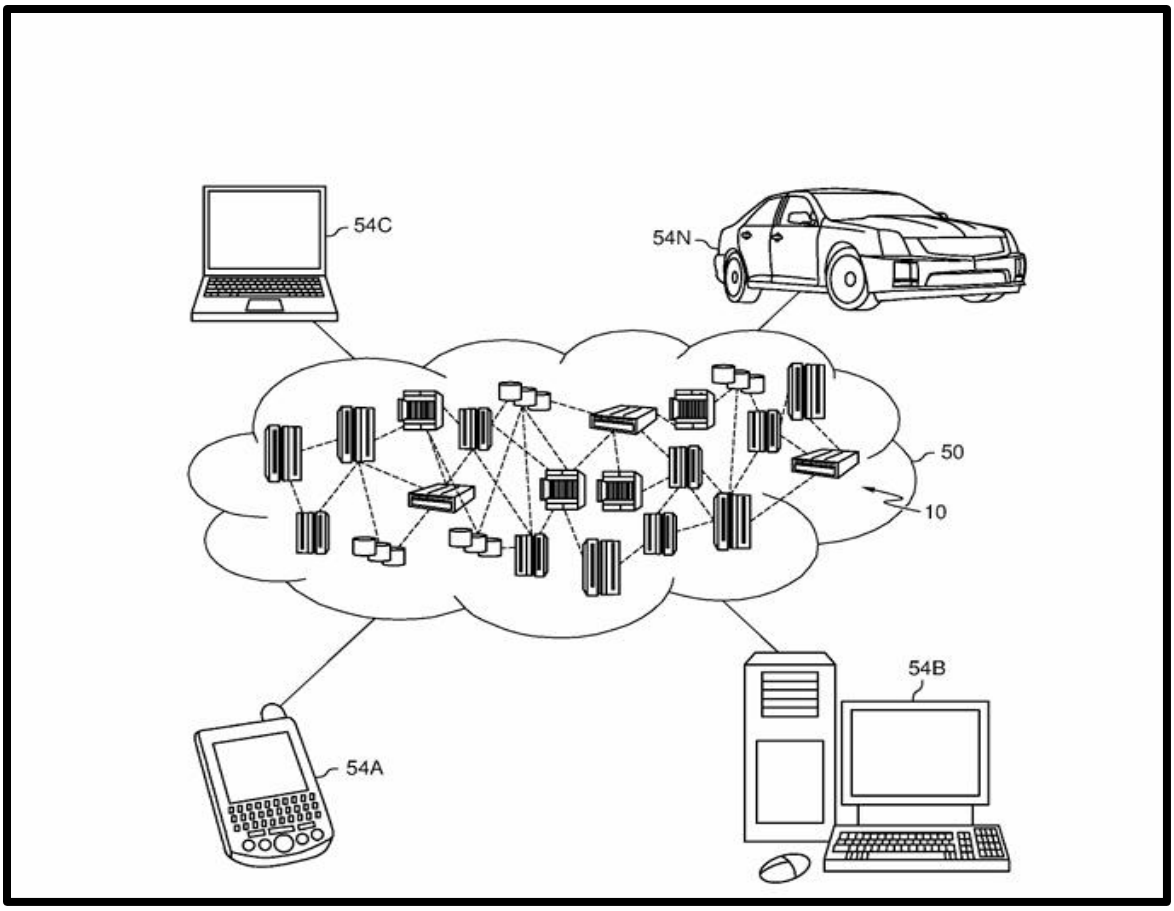



Figure 24 is a patent by Liu et al. (2022) for an artificial intelligence system that detects waste in places such as natural environments. The system collects images with sensors or cameras and passes them to machine learning algorithms that identify waste objects. Then, the system tracks waste accumulation patterns and finds the location and size of garbage in monitored areas.

This patent is similar to Garbage-Guard's usage of Artificial Intelligence and image analysis to find garbage in different locations.

**Figure 25**

*Flood Warning System*

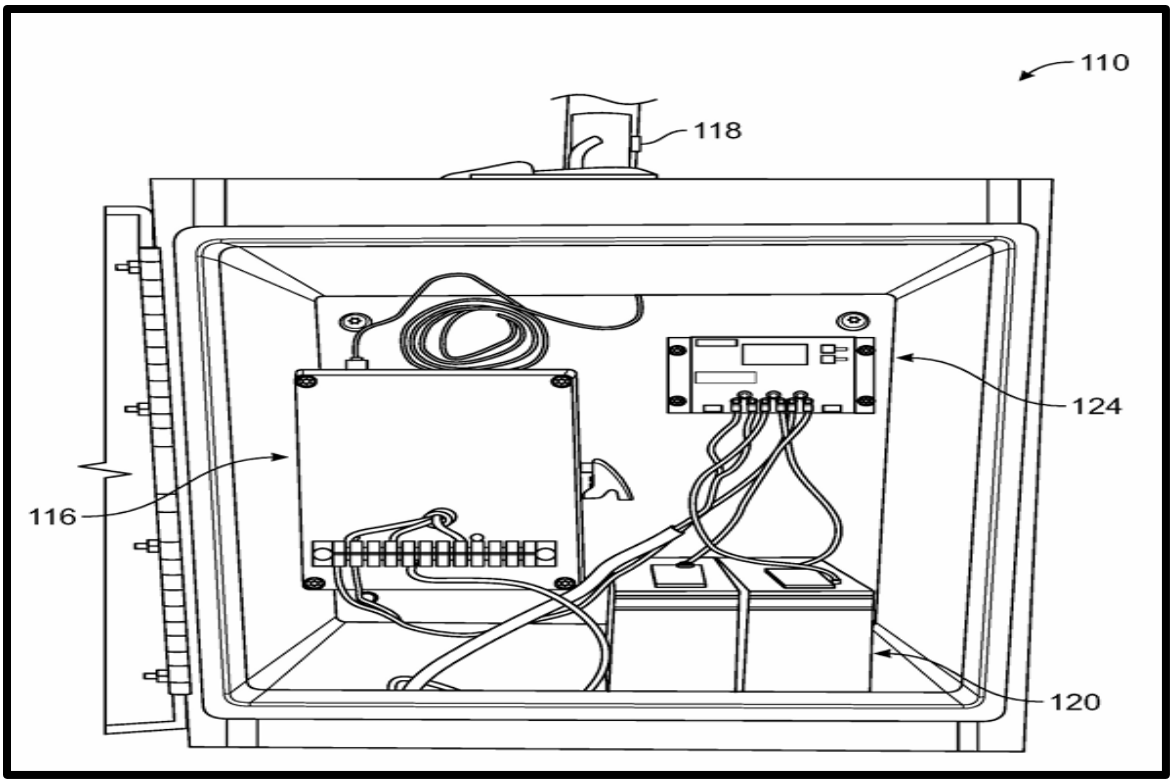
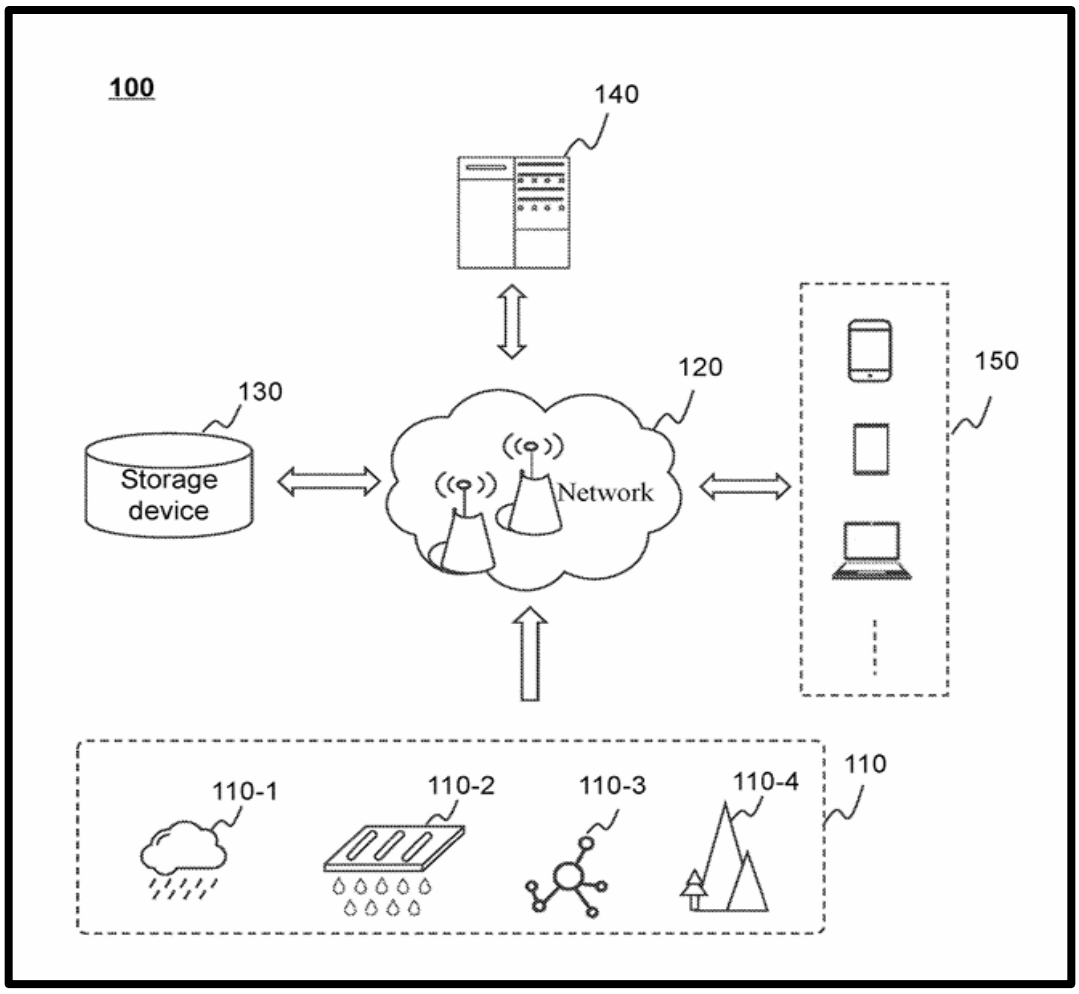



Figure 25 is a patent by Torres et al. (2024) for a flood warning system that gives early warnings of floods. It places monitoring stations near rivers or drainage channels, each equipped with sensors for water level and rainfall. The collected data is sent wirelessly to a flood warning station, where it is checked against water-level thresholds. When a flood risk is found, the system sends visual alerts to users' mobile devices.

This patent is useful and similar to Garbage-Guard's system, since warning systems are important in monitoring the environment and can notify when the creeks are filled with garbage.

**Figure 26**

*Method and System for Flood Early Warning in Smart Cities Based on the Internet of Things*




The patent in Figure 26, by Shao et al. (2023), describes a smart flood early warning system that uses Internet of Things (IoT) data to detect potential flooding in urban areas. The system collects precipitation information from weather stations and combines it with sensor data to estimate the water level in specific areas.

This invention is similar to Garbage-Guard in that both involve environmental monitoring and alerts.

**Figure 27**

*Apparatus and Method for Waste Monitoring and Analysis*

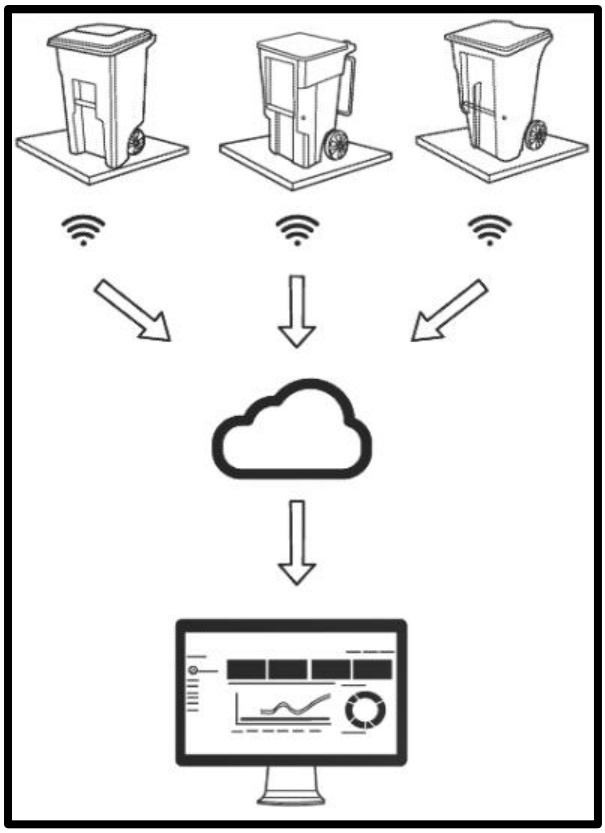


Figure 27 is a patent by Balachandran and Talavera (2023) for a waste monitoring and analysis system that analyzes waste collected in containers. It uses sensors such as load cells and cameras to measure the weight and type of waste placed in bins. The data is sent to a cloud platform for analysis, which identifies patterns in waste disposal and produces reports on waste generation.

This patent is relevant to Garbage-Guard because it shows how sensor-based systems can monitor and analyze data in real time. Garbage-Guard, by contrast, detects garbage collecting in creeks using computer vision.

## 2.4 Synthesis of the Review

The literature reviewed in this chapter converges on four points. First, a large volume of plastic enters the ocean through rivers. Meijer et al. (2021) state that more than 1000 rivers are responsible for 80% of riverine plastic emissions, and Southeast Asia (Ng et al., 2023) and the Philippines (Corpuz, 2025) are among the places where collection infrastructure falls short. The problem this study addresses is therefore a documented and measured one rather than a local inconvenience.

Second, manual inspection does not scale. Studies that have tried to quantify waste in waterways depend on a survey campaign, a satellite pass or a modelled estimate, none of which gives the operator a reading to act on the same afternoon.

Third, convolutional object detection is mature enough for the task. Alzubaidi et al. (2021) describe the convolution and pooling layers that these detectors share, and Murat and Kiran (2025) and Yaseen (2024) report the accuracy and speed improvements of all the YOLO versions over the past 10 years, while Nežerka et al. (2023) show a convolutional classifier recognizing waste fragments with 85.9% accuracy, although selected texture features scored higher on their task. Detection of floating waste is not an open research question.

Fourth, the supporting pieces exist separately. Li et al. (2024) present a ‘Very small YOLO’ (VsYOLO) that identifies floating objects at 33fps on an edge device. Nunkhaw and Miyamoto (2024) combine detection with tracking so that an item passing through the frame is counted only once. Zhang et al. (2022) show that ByteTrack keeps identities better than DeepSORT at about twice its speed, without an appearance network. Nunkhaw et al. (2025) extend this work to urban canals in Southeast Asia and report the gap between lab and field results. Jia et al. (2023) address the shortage of public floating-litter data, and Kataoka et al. (2024) compare segmentation with bounding-box detection. Narayana et al. (2024), Zeng et al. (2024), and Te et al. (2024) show that environmental sensing on limited hardware with real-time alerting is common.

None of the studies examined here draw the components together in the way needed for this project. The waste-detection work classifies images without tracking build-up over time. The accumulation work measures waste in containers rather than on moving water. The environmental monitoring work alerts on scalar sensor readings rather than on a count of items. None of the systems reviewed runs an operator-trained detector at the edge at creek level, limits the count to an operator-defined area of water, keeps one identity per item across frames and raises a threshold alert on that count. Table 2 sets out that gap against what this study proposes.

**Table 2**

*Proposed Solution Matrix*

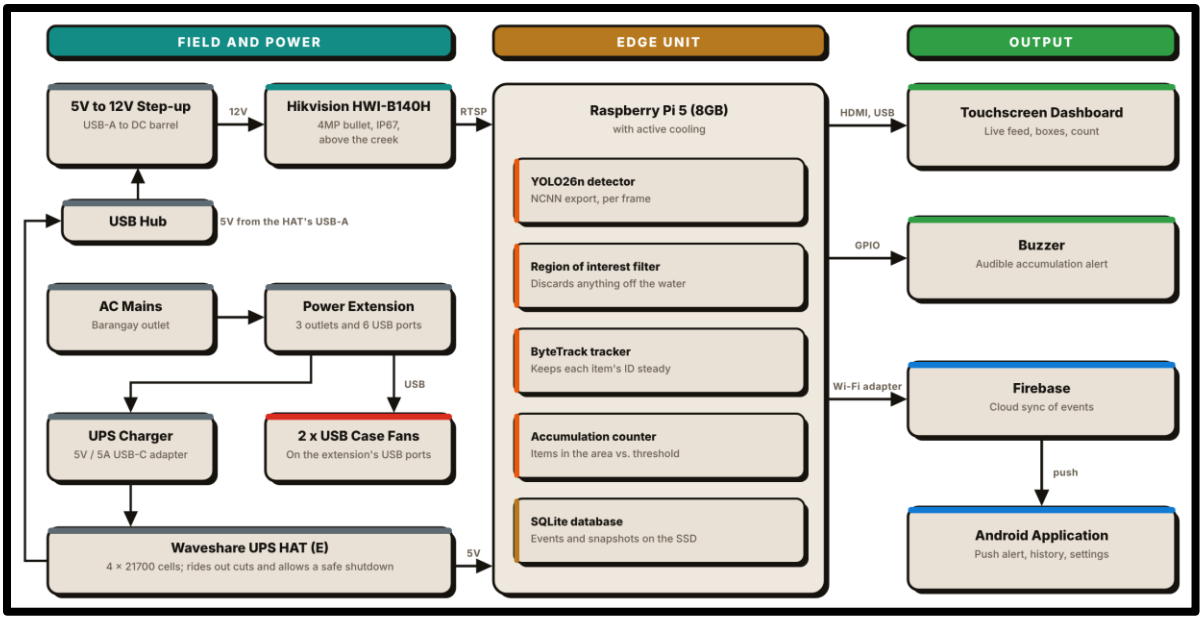
| Limitation in the examined literature | Representative source | How Garbage-Guard responds |
|---|---|---|
| Waste is classified frame by frame, with no measure of build-up over time. | Nežerka et al. (2023); Magyar et al. (2023) | The objects in the monitored area are counted, smoothed across 15 frames and compared to an operator-set threshold; thus, the output is a sustained accumulation reading and not a label per frame. |
| Waterway monitoring requires manual survey or satellite passes, which results in the readings being periodic. | Meijer et al. (2021); Victoriano et al. (2022) | Continuous processing and monitoring of a fixed camera feed, and logging of every event with the time recorded on site. |
| Counting systems assume a contained, static scene | Balachandran and Talavera (2023) | Region filtering helps to limit the counting within the traced region of water, and tracking helps to maintain a one-to-one correspondence between one floating object and one identity. |
| Alerting in the environment is based on scalar thresholds on sensors and not visual observation and evidence | Te et al. (2024); Arante et al. (2025) | The alert is based on a visual count and includes a saved snapshot, so the reading that triggered the alert can be viewed. |
| Detection models are published on curated datasets and not deployment conditions | Li et al. (2024); Nunkhaw et al. (2025) | The model is reported on a frozen held-out partition, and a creek-only subset, so field performance is reported separately. |

# III. METHODOLOGY

## 3.1 System Design

**Figure 28**

*Garbage-Guard's System Architecture*




The system design covers the key features, components and processes needed to support the goals of the study. It contains hardware and software integration, block diagrams, flow diagrams and sequential operations for the functionality of the system.

The proposed Garbage-Guard prototype detects floating garbage automatically while monitoring a creek. A high-definition CCTV camera supplies real-time video, and the YOLO26n object detection model classifies the floating objects as bottles, bags, or food containers. All processing runs on the Raspberry Pi 5 edge computer, as seen in Figure 28.

The prototype includes the following components:

1. **Input:** Live video footage of the creek surface recorded by a CCTV camera.

2. **Processing:** A YOLO26n model, exported to NCNN and run on a Raspberry Pi 5, with a region-of-interest filter and ByteTrack tracking, classifies and counts floating solid garbage.
3. **Storage:** Detection and accumulation records are stored in a SQLite database on the unit, with a snapshot for every event, and synchronized to Firebase.
4. **User Interface:** Real-time detections and accumulation alerts are shown on a touchscreen dashboard, sounded on a buzzer and pushed to an Android application.

As a real-time system, Garbage-Guard can give local authorities information for enforcing environmental regulations and help reduce severe pollution in local waterways. It reports garbage accumulation immediately, which helps communities stay aware of conditions in the creek and manage them. This early warning supports better water quality, lower environmental health risks, and closer adherence to garbage disposal regulations.

**Figure 29**

*Front and Side Views of the Garbage-Guard Enclosure with Dimensions*

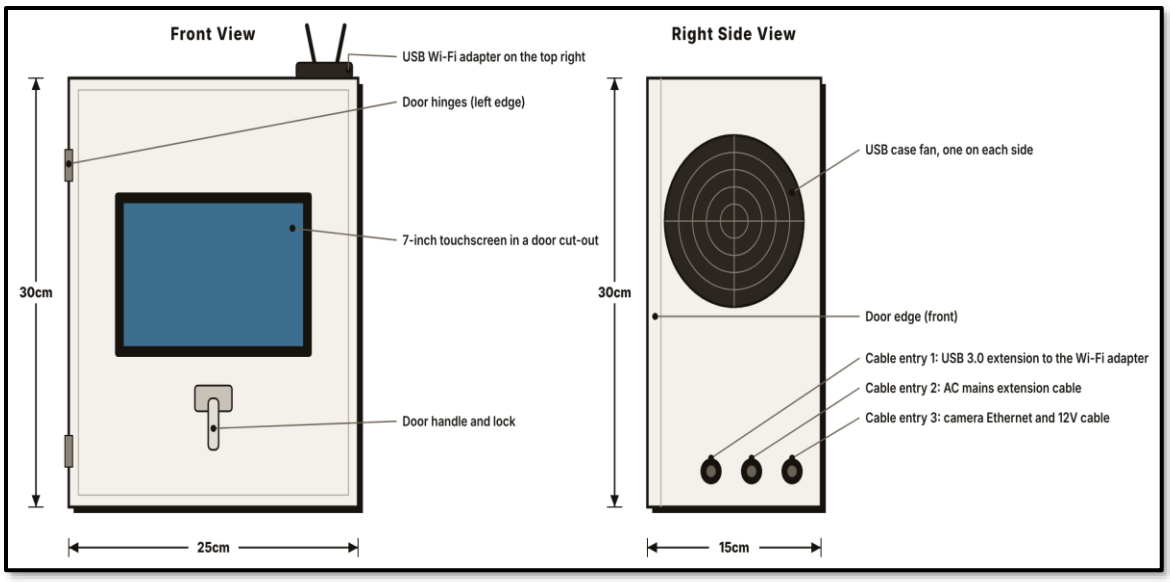


Figure 29 shows the physical design of the prototype: the outside dimensions of the enclosure after it was modified, in front and side view. The 7-inch touchscreen sits in a cut-out in the door, so the dashboard can be read and touched without opening the box, and the door handle with its lock is below the screen. The door is hinged on its left edge. A USB case fan is fitted on each side wall. Three cable entries are cut low on the right-side wall and sealed with rubber sealant. From front to back they carry the USB 3.0 extension cable to the Wi-Fi adapter, the AC mains extension cable, and the 2-in-1 Ethernet and power cable to the camera. The USB Wi-Fi adapter sits on top of the enclosure at the right, outside the metal walls. The camera is not drawn. It is mounted outside, apart from the box, and joined to it by its cable only.

***Block Diagram***

**Figure 30**

*System Flow of the Waste Detection System*

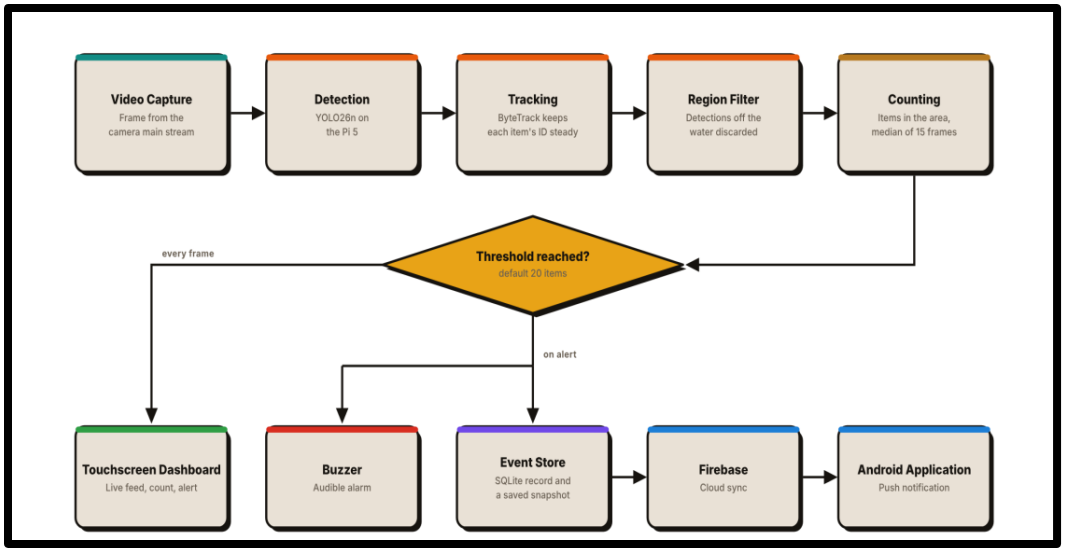



Figure 30 shows the block diagram of the system, which combines the YOLO26n model, an edge computing device (Raspberry Pi 5) and a CCTV camera for efficient real-time garbage detection and data storage.

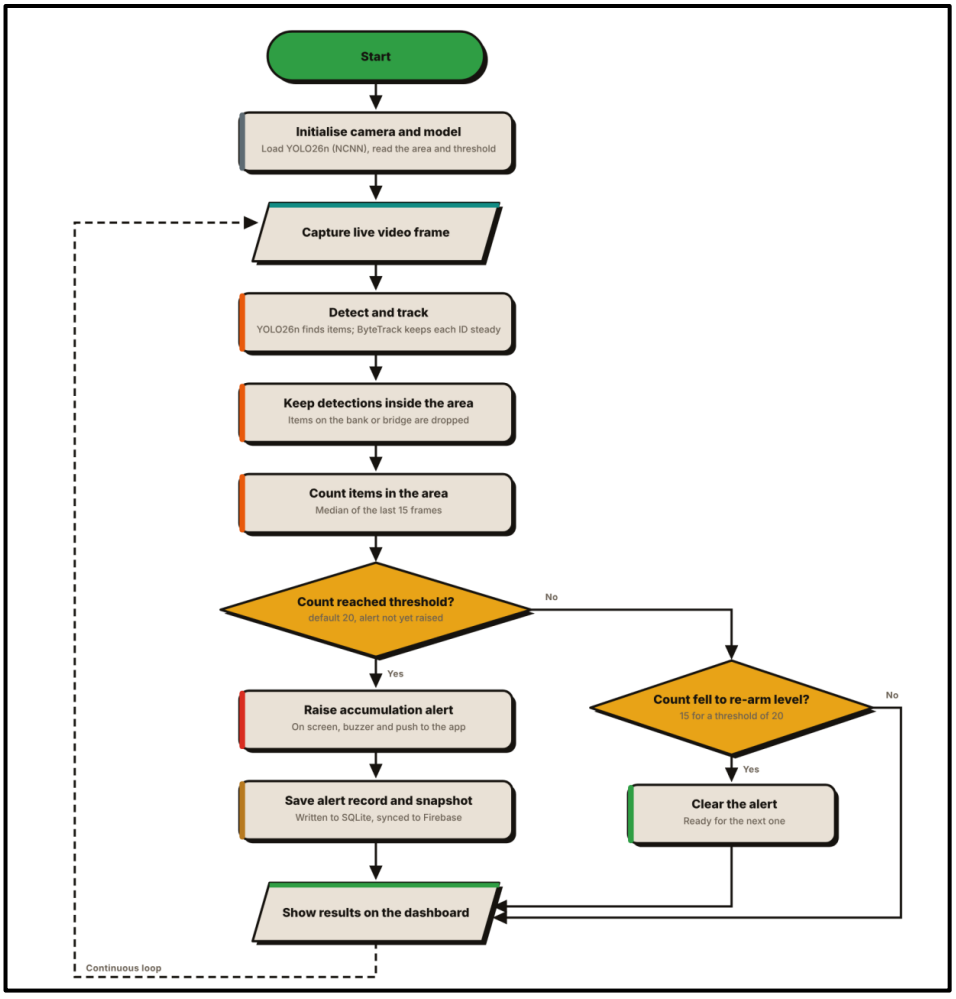
Once initialized, the camera records real-time video of the target water surface. The video passes to the main processing unit, a Raspberry Pi 5, which uses the YOLO26n

model to classify floating solid waste, keep a consistent identity for each object, exclude objects outside the monitored water area, and count the rest. Users see the annotated results in real time on the live dashboard, along with accumulation alerts and stored data. The processing unit raises an alert when the count reaches the garbage threshold (20 by default). It also stores all garbage data in the local database, including category, time, and accumulation, for later analysis. Combining the hardware (Raspberry Pi 5 and camera) with the software (YOLO26n) lets local communities monitor and manage environmental hazards safely and efficiently.

***Flowchart***

**Figure 31**

*Garbage-Guard's System Flowchart*



The garbage detection and data management process is shown in Figure 31. It begins by switching on and initializing the camera and loading the model, the saved monitored area and the threshold. The system then captures a live video frame, detects floating garbage with YOLO26n and passes the detections to ByteTrack, which keeps a single identity for each piece of garbage so that its box and label stay constant from frame to frame.

Detections outside the region of interest are discarded, so objects on the bank or a bridge are not counted. The items left in the area are tallied, and the tally is smoothed as the median of the previous 15 frames. The count therefore reflects the current amount of garbage in the area: it rises as items accumulate and falls as they drift away.

In the first decision point the smoothed count is compared with the accumulation threshold, which has a default value of 20 items but can be configured in the dashboard. When the count reaches the threshold and no Accumulation Alert is active, the system alerts the user on the touchscreen, activates the buzzer and sends a push notification to the Android app. The alert is written to the SQLite database with its counts, the totals for each class and an annotated snapshot, and synchronized to the cloud.

The second decision point checks whether an active alert can be cleared. Once the count drops to the re-arm level (15 for a threshold of 20), the alert is removed and the system is ready to trigger the next alert. This difference between the two levels prevents a hovering pile from generating a new alert every few seconds.

In each case the annotated frame and the count are shown on the dashboard, and the procedure repeats for the next frame. Frames that fire no alert are displayed but not stored. Besides alerts, the system stores manual captures, test alerts, and mutes, along with a latency summary every 100 frames. It keeps analyzing new video while the record stays small enough for a long deployment.

***Component Materials***

This section describes the parts that make up one Garbage-Guard unit, divided into the physical components and the software that runs on them. Each entry states what the component is and what it does in this system, since several were chosen for reasons that are not obvious from the part name alone.

**Hardware Components**

***Raspberry Pi 5 (8GB)***

**Figure 32**

*Raspberry Pi 5 Single-Board Computer*




*Note. Image from The Pi Hut (n.d.-b).*

The Raspberry Pi 5 is a single-board computer based around a quad-core Arm Cortex-A76 processor clocked at 2.4GHz with 8GB of memory (Figure 32). It has a 40-pin general-purpose input/output header, micro-HDMI outputs, USB 3.0 ports, Gigabit Ethernet and a PCIe interface.

It is the whole of the processing side of Garbage-Guard. It reads the camera's RTSP stream, runs the detection model, applies the ROI, maintains the tracker and the accumulation counter, writes to the local database, drives the buzzer (GPIO12), serves the dashboard over HTTP and pushes events to the cloud. The 8GB version was selected because the model, the tracker and the video decoder run at the same time with their working buffers in memory.

***Hikvision HiWatch HWI-B140H Network Camera***

**Figure 33**

*Hikvision HiWatch HWI-B140H Bullet Network Camera*



*Note. Image from Multi-Line (n.d.).*

The HWI-B140H is a 4MP fixed bullet network camera with a 1/3-inch progressive scan CMOS sensor, which supports up to 2560x1440 at 20fps (Figure 33). It has a fixed-focus M12 lens at F2.0, an 850nm infrared illuminator with a maximum off-axis range of approximately 30m, and an 8-stop (120dB) wide dynamic range, and is rated as IP67 with an operating temperature range between -30°C to 50°C. It accepts either power over Ethernet or 12V direct current on a 5.5mm coaxial plug, drawing at most 5W on the direct current input (Multi-Line, n.d.).

It is mounted above the creek, pointed along the water surface, and it is the only sensor in the system. Everything Garbage-Guard provides comes from this one feed, so its IP67 rating and wide temperature range matter for an outdoor deployment. Because the region of interest is traced over the camera's actual view, the camera is mounted first and the region is drawn second, never the other way around.

The camera operates from a 12V input power source and not through power over Ethernet (PoE); its Ethernet cable carries data only. Garbage-Guard reads the camera's main stream, set to 1280x720 at 15 frames per second in H.264, and crops each frame to the monitored area before it reaches the model, which runs at a detection size of 960 pixels by default, the size at which v8, v9 and v10 were trained. The Model card in the Settings tab installs and switches between trained models without editing code. v10 is the model reported in Chapter IV, and v8 is kept on the unit as the fallback. Each model

works best at the size it was trained at, so the detection size is the first setting to check after a switch; v7, for example, was trained at 640 pixels and scored higher on the frozen test set at 640 than at 960. Cropping keeps the detail of the main stream inside the area that matters, so a floating item covers more pixels than it did on the 640x360 sub-stream that earlier builds used. At 1920x1080 the Pi could not decode the stream fast enough and the picture fell six to seven seconds behind; at 1280x720 it runs about one second behind on the touchscreen. The sub-stream remains available from the Settings tab as a fallback.

***Waveshare UPS HAT (E)***

**Figure 34**

*Waveshare UPS HAT (E) with Four Battery Bays*




*Note. Waveshare (n.d.-b). Image from Waveshare.*

The Waveshare UPS HAT (E) is a four-cell 21700 lithium-ion uninterruptible power supply (UPS) board (Figure 34). It is mounted under the Raspberry Pi rather than on top of it, connecting through high-current pogo pins pressed against the underside of the board, and it provides 5V at up to 6A. It passes mains power through while charging, takes over when mains power fails and reports its state over I2C, including the individual cell voltages, the four currents, the power and the charger input. It also has a USB-A output.

One point about the pogo-pin connection is worth noting: it leaves the 40-pin header of the Pi completely free (Waveshare, n.d.-b), so the buzzer connects directly to the header pins it needs, without a pass-through header or a ribbon extension.

In Garbage-Guard, the HAT is what makes an unattended deployment safe. The program reads it over I2C and, when the battery is low, starts a 5-minute countdown to a clean shutdown, with a minimum of 5% charge and an optional grace period after boot, so the Pi is never cut off mid-write.

***5V to 12V USB-A to DC Barrel Step-up Cable***

**Figure 35**

*5V to 12V USB-A to DC Barrel Step-up Cable*




*Note. Image from Shopee (n.d.).*

A step-up (boost) converter raises a voltage from a lower to a higher level (Figure 35). This part is the cable version of such a converter, with a USB-A plug at one end and a 5.5mm barrel plug at the other. It is rated 12V at 0.8A. The step-up cable is not plugged into the UPS HAT directly. Its USB-A plug cannot fit beside the HAT's USB-C charging cable, so a small USB hub inside the enclosure is plugged into the HAT's USB-A output, and the step-up cable runs from the hub.

The camera needs no separate power supply. It draws from the battery-backed USB-A output of the UPS HAT through the hub, so an AC mains failure does not blind the system: the camera and the Pi stay up or go down together. The cable rating is also adequate, since the camera draws no more than 0.42A at 12V, just over half of the cable's 0.8A rating at 12V.

That margin matters because a converter run near its limit is a common cause of intermittent faults in field equipment, and an intermittent camera would present as a detection problem rather than a power one.

***USB Wi-Fi Adapter and USB 3.0 Extension Cable***

**Figure 36**

*Erlvsoiis WF-ER1800M Dual-Band USB Wi-Fi Adapter*




*Note. Image from Shopee (n.d.).*

**Figure 37**

*Vention USB 3.0 Type A Male-to-Female Extension Cable*




*Note. Image from Shopee (n.d.).*

The Raspberry Pi 5's own Ethernet port is given over to the camera on a direct link, so the unit reaches the internet over Wi-Fi. The Erlvsoiis WF-ER1800M (Figure 36) is a dual-band USB 3.0 adapter for the 2.4GHz and 5.8GHz bands with two external antennas (Shopee, n.d.), and Raspberry Pi OS recognized it with no driver installed. It is plugged into a Vention USB 3.0 Type A male-to-female extension cable (Figure 37) that leaves the box through a cable entry, so the adapter sits on top of the enclosure at the right, as Figure 52 in Chapter IV shows.

During testing, the built-in wireless adapter of the Pi could not complete a connection to any network, an open one included, and the metal walls of the enclosure weaken any radio inside it. The USB adapter replaced an earlier setup of a Wi-Fi repeater cabled through a USB-to-Ethernet adapter. The camera link and the internet link still run on separate interfaces, so a fault on one does not affect the other.

### *7-inch IPS Touchscreen*

**Figure 38**

*7-inch IPS HDMI Touchscreen*



*Note. Image from Shopee (n.d.).*

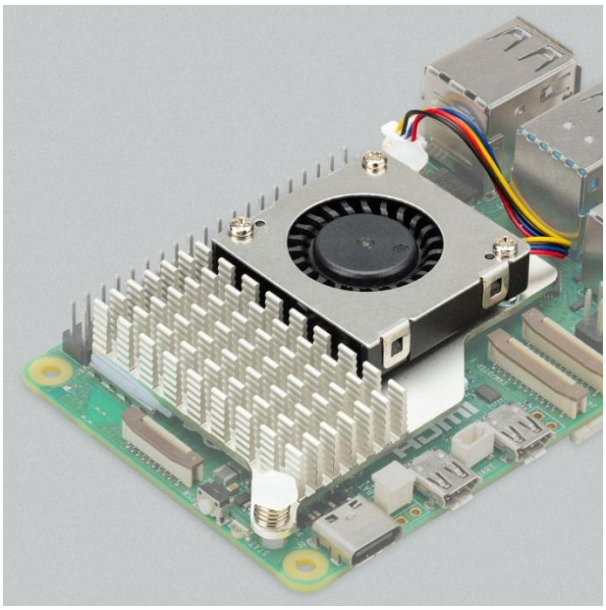
The display is a 7-inch IPS panel at 1024x600 (Figure 38). It takes video on HDMI and reports touch over USB-A. The panel type matters because the mounted screen is often viewed at an angle as well as straight on, and an IPS panel keeps its color and contrast off-axis where a cheaper panel would not.

It is the operator's only local interface. The dashboard runs full screen on it, and the region of interest is tapped directly on the video. A setting turns the HDMI output off after a set time, and sooner on battery, and the touch input is read directly so that a tap wakes the screen when the output is off.

***Raspberry Pi Active Cooler***

**Figure 39**

*Raspberry Pi Active Cooler Fitted to a Raspberry Pi 5*



*Note. Image from The Pi Hut (n.d.-a).*

The Raspberry Pi Active Cooler is a heatsink and fan combination that attaches to the Pi 5's mounting points and runs from the board's dedicated fan header, which lets the firmware adjust the fan speed to the CPU temperature (Figure 39).

It is fitted because of sustained inference. Running detection on every frame keeps the processor near its thermal limit for long periods, and a Pi 5 that reaches the limit slows down, which shows up as rising latency or a falling frame rate rather than as an obvious fault. Active cooling keeps the measured latency a reflection of the hardware rather than of its temperature.

***SanDisk 64GB High Endurance microSD Card***

**Figure 40**

*SanDisk High Endurance 64GB microSD Card*




*Note. Image from JB Hi-Fi (n.d.).*

This SanDisk card belongs to the High Endurance line, which is designed not for the occasional large transfer a standard card is rated for but for devices that write continuously, such as a dashboard camera or a surveillance recorder (Figure 40).

The card holds only Raspberry Pi OS; the detection data is stored on the solid-state drive described below. Its workload is the operating system's own: journal writes, logs and the housekeeping of a machine that is never switched off. That workload is lighter but still continuous and lasts a whole field deployment, so an endurance-rated card makes sense in a system left running unattended.

***KingSpec 256GB M.2 Solid-State Drive with Waveshare PCIe Adapter***

**Figure 41**

*KingSpec M.2 2242 NVMe Solid-State Drive*




*Note. Image from KingSpec (n.d.).*

**Figure 42**

*Waveshare PCIe to 2-Channel M.2 Adapter*




*Note. Image from Waveshare (n.d.-a).*

The solid-state drive is a KingSpec 256GB module in the 2242 form factor, mounted on a Waveshare PCIe to two-channel M.2 adapter that plugs into the PCIe interface of the Raspberry Pi 5 (Figures 41 and 42). Because the adapter has two channels, a second module can be added beside the first to extend storage for a longer deployment without changing the board or the mounting.

The drive holds the project folder, including the exported model, the database, and the snapshot store, while the operating system stays on the microSD card. The point of the arrangement is separation. Garbage-Guard writes a snapshot for every recorded event and keeps appending to the database, a steady stream of small writes that is the worst case for flash wear. Placing these on the drive puts the heavy writing on the storage best suited to absorb it, and if the drive fails, the operating system survives.

***Passive Buzzer Module***

**Figure 43**

*MH-FMD Passive Buzzer Module*




*Note. Image from Shopee (n.d.).*

A passive buzzer produces no tone unless the controller drives it with a varying signal (Figure 43). That is a drawback on paper but an advantage in practice, because the pitch is then under software control and one device can produce several distinct sounds.

It is wired with the supply to physical pin 17 (3.3V), with the signal line to physical pin 32 (GPIO12), and with the ground to physical pin 34. Each event has its own sound with the buzzer module, and when the count reaches the accumulation threshold, a repeated alarm is raised.

It drives the pin by software pulse-width modulation (PWM) by default and switches to hardware modulation, which is louder and steadier, when the pin has been configured for it. Quiet hours and a mute panel on the dashboard silence it when the operator sets them.

***Metal Waterproof Junction Box***

**Figure 44**

*Metal Waterproof Junction Box Used for the Enclosure, Before Modification*




*Note.* Image from Shopee (n.d.).

Figure 44 shows the junction box as it was bought, before it was cut for the touchscreen, the fans and the cable entries. It is a metal box with a hinged door, a rubber seal around the door, a grounding post and a handle lock. The seller lists it as IP65. That rating applies to the unmodified box only: once the cut-outs for the screen, the fans and the cables were made, the enclosure no longer meets it, which is why the unit is kept under cover.

The electronics are housed in a metal waterproof junction box of the type sold for closed-circuit television installations, measuring 25cm x 30cm x 15cm, with three openings cut low on the right side for the USB 3.0 extension cable to the Wi-Fi adapter, the AC mains extension cable and the camera cable, sealed with a rubberized water sealant. The USB Wi-Fi adapter is mounted on top of the box at the right. A USB fan is fitted on each side. The boards are stacked in a specific order: UPS HAT, Raspberry Pi (above the UPS HAT and running its active cooler), and PCIe with the SSD on top.

The enclosure is not exposed to the weather. It is installed under cover, and only the camera is mounted outside, with its cable running back to the box. The camera, the only part exposed to rain and sun, has an IP67 rating; the other components have no such rating and, by design, are kept out of the weather.

***Power Extension with USB Ports***

**Figure 45**

*Mitsushi 2500W Power Extension with Three Universal Outlets and Six USB Ports*




*Note.* Image from Shopee (n.d.).

Figure 45 shows the Mitsushi power extension, the point where AC mains power enters the enclosure. It is rated 2500W maximum at 10A, 220V and 50/60Hz, carries BPS certification, and has three universal outlets, six USB ports and a power switch (Shopee, n.d.). The UPS charger is plugged into one of its outlets and the two case fans into two of its USB ports, so the fans need no power adapter of their own.

This split matters during a power cut. The UPS HAT, the Active Cooler and the camera keep running from the battery when mains power fails, but the case fans stop,

because they draw from the extension and not from the HAT. On battery the processor keeps detecting inside a closed box with only its own cooler. This was deliberate: running the fans from the battery would shorten the time the unit can keep detecting.

***5V USB Case Fans***

**Figure 46**

*5V USB Case Fan*




*Note.* Image from Shopee (n.d.).

Figure 46 shows the FUGONGDA FGD9025H9BL case fan, two of which cool the inside of the enclosure, one on each side wall (Figure 29). Each runs on 5V DC from a USB-A plug, so it needs no separate supply, and the seller lists it in the 12x12 size (Shopee, n.d.). Both are plugged into the USB ports of the power extension and move air through the box past the board stack, while the Active Cooler handles the processor itself. A wire grille covers each fan on the outside of the wall.

### Software Components

#### *Raspberry Pi OS and the Python Environment*



The unit runs Raspberry Pi OS with the detection program in a virtual environment kept separate from the system packages. An installer registers the launcher so that switching the Pi on is enough to bring the system up: it checks the camera, raises the network profile if needed, activates the environment, opens the dashboard full screen and starts detection. If no network is in range, it shows a wireless setup screen first.

#### *Roboflow*

Roboflow is a web-based platform for building computer vision datasets. It provides an editor for labeling images, keeps each version of a dataset, and exports labeled images in the formats that training libraries read (Roboflow, n.d.).

In this study it served one purpose. The bounding boxes for the three garbage classes were drawn and checked in its editor, then exported in the YOLO format for training on the researchers' laptop. Nothing from Roboflow runs on the Raspberry Pi: it is part of how the model was made, not part of the deployed unit, and its place in development is described in Section 3.2.

#### *YOLO26n Detection Model*

The detector is a YOLO26n convolutional neural network, trained on the custom dataset and exported to NCNN, an inference framework built for mobile and embedded

processors. NCNN is used rather than the original weights because it is markedly faster on the Pi's Arm processor.

The model classifies three categories, in order: bottle, bag and food container. The reported model, v10, scores a precision of 0.846, a recall of 0.672 and a mAP@0.5 of 0.744 on the frozen test partition, and a precision of 0.689, a recall of 0.423 and a mAP@0.5 of 0.536 on the creek-only subset. Chapter IV reports these figures in detail.

***ByteTrack Multi-Object Tracker***

ByteTrack tracks objects from frame to frame, even when they are missed at some time steps, so each object keeps a consistent label for its whole appearance in the scene.

This keeps the live view readable and the count stable. Without it, each item's box would be drawn afresh in every frame and could switch between classes or disappear for a frame. The boxes on the dashboard carry a small number, starting at 1, and the number is released after the object has been missing for about thirty frames.

***Region of Interest and Accumulation Counter***




The region of interest (ROI) is a polygon that the operator draws on the video; it is saved on disk and reloaded at start-up. A detection whose position falls outside it is discarded before counting, which is why the bank and the bridge wall are not counted.

At the end of each frame the counter is updated with the number of items inside the region, smoothed as the median of the last 15 frames so that a single missed or false detection does not move it. When the smoothed count reaches the threshold (20 by default, adjustable in the Settings tab), one alert is raised, and it re-arms once the count

drops to 15. A pile hovering around the threshold therefore does not trigger repeated alerts. Because the model misses some items, a displayed count of twenty can stand for more items than that in the monitored area, so the threshold is calibrated on site against a hand count rather than assumed (Table 17).

***SQLite Database***




Accumulation alerts, brief alerts that reached the threshold but were not sustained, mutes, manual captures, and test alerts are stored in a local SQLite database. Each alert and capture has a snapshot with the boxes, monitored area, and count stamped onto the image. A processing latency summary is written every 100 frames. Frames that trigger no alert are shown on the dashboard but not stored, which keeps the database small over a long deployment.

As a result, nothing is lost when the network goes down, and the saved snapshots can be used later to verify any reading the system made.

***HTML5, CSS, JS***




The dashboard is a single HTML file that holds its HTML5 (Hypertext Markup Language 5) markup, Cascading Style Sheets (CSS) and JavaScript (JS). Python's built-in web server serves it over HTTP from the Pi, with no build step and no external dependencies. It has three tabs: Monitor, History and Settings.

Monitor displays the live feed with the bounding boxes, running count, frame rate and mute panel and is where the region of interest is drawn. History displays snapshots of

events that have been recorded and a filter bar. Settings contains the detection threshold, the alert preferences, the screen saver and the power options.

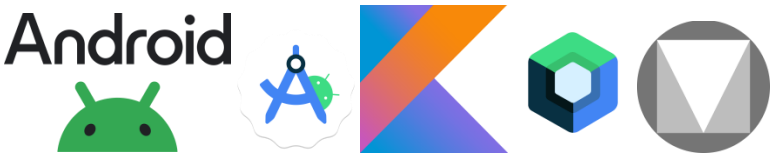
***Firebase***




Firebase carries events and the system state from the unit to the phone, and a set of access rules controls who can read and write them.

It is in place so that an operator does not have to be at the creek to know that the threshold has been reached. The Pi is the system of record and the cloud has a copy.

***Android Studio***



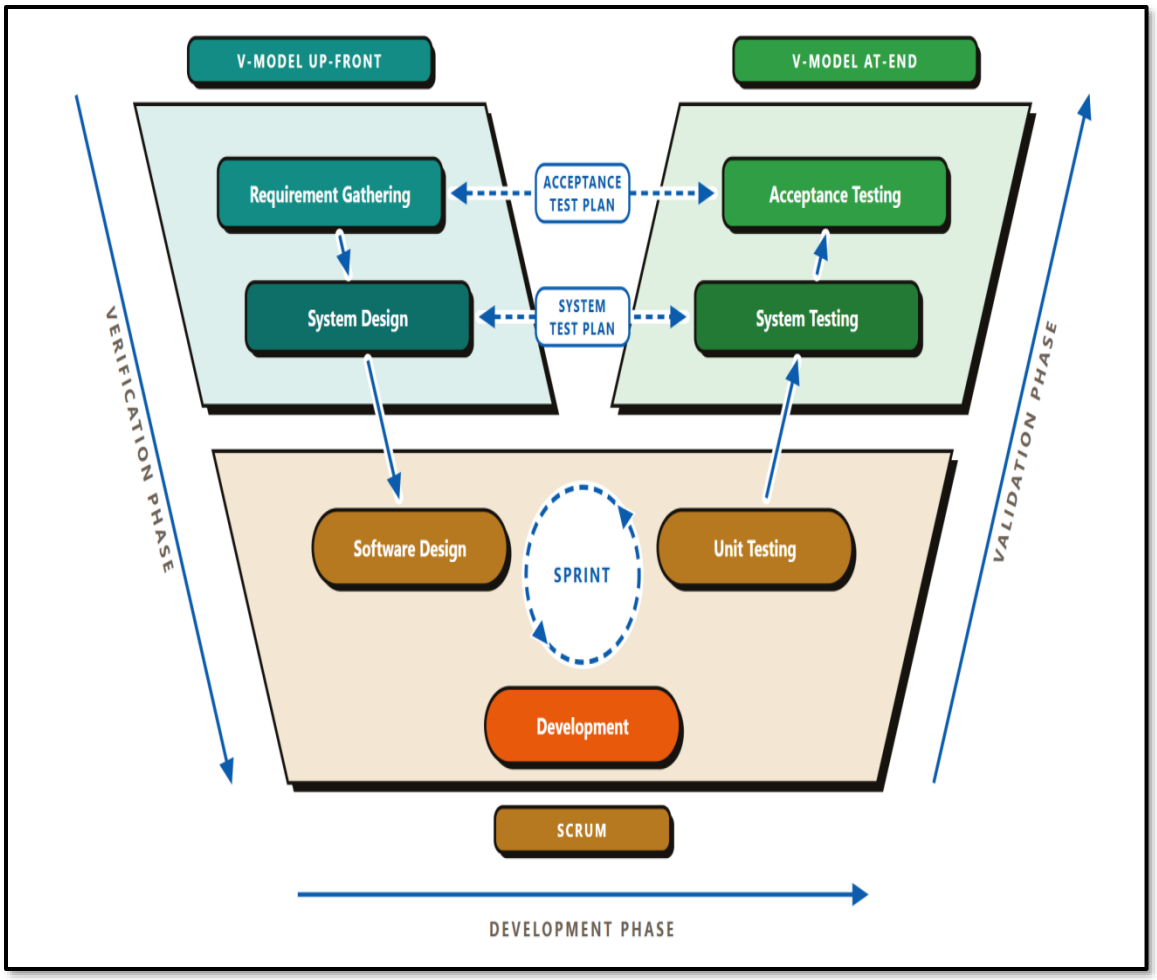

The Android app, built in Android Studio, is written in Kotlin, and its screens are built with Jetpack Compose rather than XML layouts. It receives push alerts, shows the detection history and changes detection settings remotely.

It is the second half of the alerting path. The buzzer and the on-screen warning serve whoever is present; the application serves whoever is not.

## 3.2 System Development

**Figure 47**

*V-Model and Scrum Hybrid Development Model*




The researchers followed a hybrid of the V-Model and Scrum, shown in Figure 47. In this hybrid the V-Model is kept only at the two ends of the project, for requirements and planning at the start and for integration and system testing at the end, while the work in between is done in Scrum sprints with testing inside every iteration (Samolovac, 2026). A pure V-Model did not describe what happened in this study. It assumes each stage is finished before the next one starts, while the detection model here was trained, tested and retrained many times before the system as a whole was tested at the creek.

The upper left of the figure is the V-Model up-front. Requirement gathering fixed the scope, the three garbage classes and the ISO/IEC 25010:2023 targets with the adviser, and system design fixed the hardware: the camera, the Raspberry Pi 5, the power path and the enclosure. Both were settled early and once, because the parts had to be bought and the tests depend on them. Each is paired with the stage directly across from it by a dashed test-plan link. The acceptance test plan, the indicators in Tables 6 to 8, was written during requirement gathering, and the system test plan, the cases in Table 5, was drafted as soon as the hardware layout was settled, long before there was a system to test.

The lower part of the figure is the development phase, run in Scrum, where most of the weeks on the Gantt chart in Appendix A were spent. The dashed circle is one sprint, and every sprint went around the same three steps. Software design chose what the sprint would change: a new dataset version, the counter logic, a dashboard tab or an application screen. Development built it, which for the model meant annotation in Roboflow, training on the researchers' laptop and export to NCNN. Unit testing closed the sprint with the cases in Table 4. Each new model was scored on the frozen test partition (TC_UNIT_01), and the counter and region logic were checked, before the build went onto the Raspberry Pi. A weak result did not move up to system testing. It became the reason for the next sprint.

The model went around this loop many times. It was trained ten times (v1 and v3 to v10, plus a v6b run; the v2 dataset was assembled but never trained), and each retraining was a sprint of its own. The test partition changed as the dataset was rebuilt between the early runs, so the scores of v5 and v6b cannot be compared with those of v7. With v7 its 807 images were frozen, and any later model is measured against the same set. The v8 model, trained with a relabeled set of creek photographs and 157 frames from the unit's own camera, went through the same cycle and was the model the unit ran during the October 31 session. v9 added more frames from the unit's camera, and v10, the final model, added pictures from a third data session at the creek. Both were measured against

the same frozen set, and v10 was also tested on pictures from that session that it had not been trained on. Changes outside the model were sprints too. When the vibration motor was removed from the unit, the alert delivery case (TC_SYS_09) was repeated with the buzzer alone, and when detection moved from the camera's sub-stream to a crop of the main stream, the detection and region cases were run again because every item now covered more pixels.

The upper right of the figure is the V-Model at-end. Once a build passed unit testing, the whole system was tested together, with the camera, the dashboard, the buzzer and the Android application (Table 5), and the cases that depend on detection, TC_SYS_01, TC_SYS_06 and TC_SYS_07, were repeated whenever a new model was loaded. Acceptance testing at the creek is the one stage meant to run a single time, on the final build. The three long arrows around the figure give the direction of the work: the verification phase runs down from the requirements into the sprints, the development phase runs across the sprints from software design to unit testing, and the validation phase runs up from unit testing to acceptance.

Roboflow, the platform used to label the dataset, belongs to the development step of the sprint and not to either end of the V. Annotation does not define a requirement and does not test one; it produces the material the model is built from. Its link to testing runs through the dataset. The frozen test partition used in unit testing is part of the same annotated set, so a weak score sends the next sprint back to annotation.

**Figure 48**

*Model Training and Deployment Pipeline*

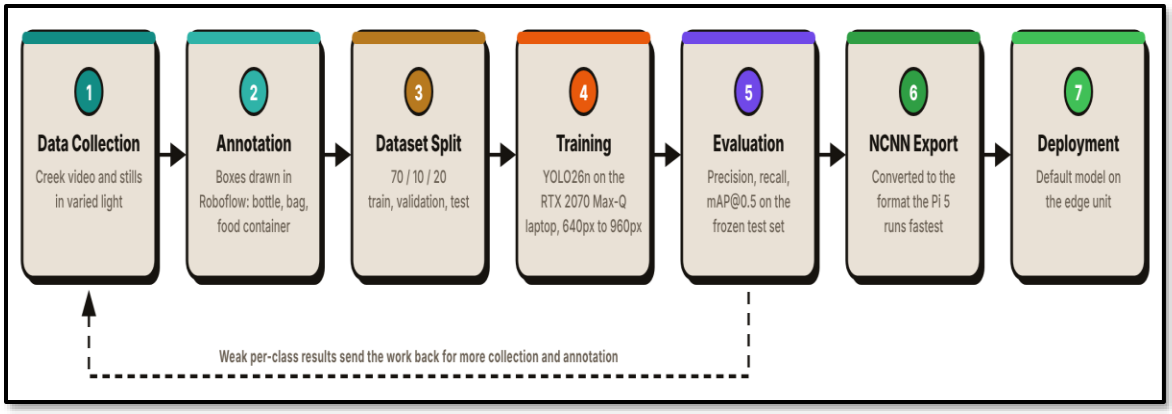



Figure 48 shows how the detection model itself was produced inside the development step of each sprint. The sequence runs from collection, through annotation in Roboflow, the dataset split, training on the researchers' laptop and evaluation on the held-out partition, to export to NCNN and deployment on the Raspberry Pi. The dashed return path is not decorative. Weak per-class results sent the work back for further collection and annotation, and each pass around it was one sprint.

***Phase 1: Verification (V-Model Up-Front)***

This phase defines the requirements the system must meet and designs the hardware and software to meet them.

**Requirement Gathering**

The researchers consulted their thesis adviser, Dr. Jonel R. Macalisang, to set the scope and title of the Garbage-Guard project, and the requirements were defined during this stage. With the guidance of Prof. Ma Ian Delos Trinos, the professor of the Project Development subject, the acceptance criteria were based on the ISO/IEC 25010:2023 standard. The assessment covered three software quality characteristics:

- **Functional Suitability:** The accuracy of the garbage detection and of the accumulation count.
- **Performance Efficiency:** The speed of detection on the Raspberry Pi 5.
- **Usability:** How readable and usable the dashboard and alerts are for the people who will use them.

**System Design**

In this stage, the researchers planned the physical system architecture and the components needed for portability and continuous operation. The hardware layout uses these components:

- **Camera (Hikvision HiWatch HWI-B140H) -** The sensor for real-time computer vision capture.
- **Raspberry Pi 5 (8GB) with the official Active Cooler -** The core processor that executes the computer vision algorithm.
- **7-inch IPS touchscreen -** The display interface that shows the real-time video feed and the log history of detected garbage, and is used to draw the monitored area.
- **Storage -** A 64GB High Endurance microSD card holds the operating system, and a 256GB M.2 solid-state drive on a Waveshare PCIe adapter holds the model, the database and the snapshots.
- **Waveshare UPS HAT (E) with four 21700 cells -** Provides an uninterruptible power supply with a safe shutdown on power loss; its USB-A output also feeds the camera through a USB hub and a 5V to 12V step-up cable.
- **Power extension, UPS charger and two USB case fans** - AC mains power reaches the UPS charger through an outlet of the Mitsushi power extension and the case fans through its USB ports; the fans stop when AC mains are lost.

- **USB Wi-Fi adapter -** It joins the site's wireless network and carries the "cloud link" to Firebase.
- **Passive buzzer module -** Gives an audible accumulation alert from a GPIO pin.
- **Metal waterproof junction box -** Protects the electronics; its openings are sealed with water sealant, and only the camera is exposed to the elements.

***Phase 2: Development Phase (Scrum)***

The sprints are where the design turned into a working unit. Development began with a custom dataset of video frames and still images of floating garbage, labeled in Roboflow, and continued with training YOLO26n on a laptop fitted with an NVIDIA GeForce RTX 2070 Max-Q graphics card. Each model that passed unit testing was exported to NCNN and tried on the Raspberry Pi 5 for compatibility and speed. The tracker, the region filter, the counter, the dashboard and the Android application were built around the model while it was still being retrained, which is why the same software was tested against several versions of the detector.

**Software Design**

Software design was revisited at the start of every sprint, and the design was organized in three layers:

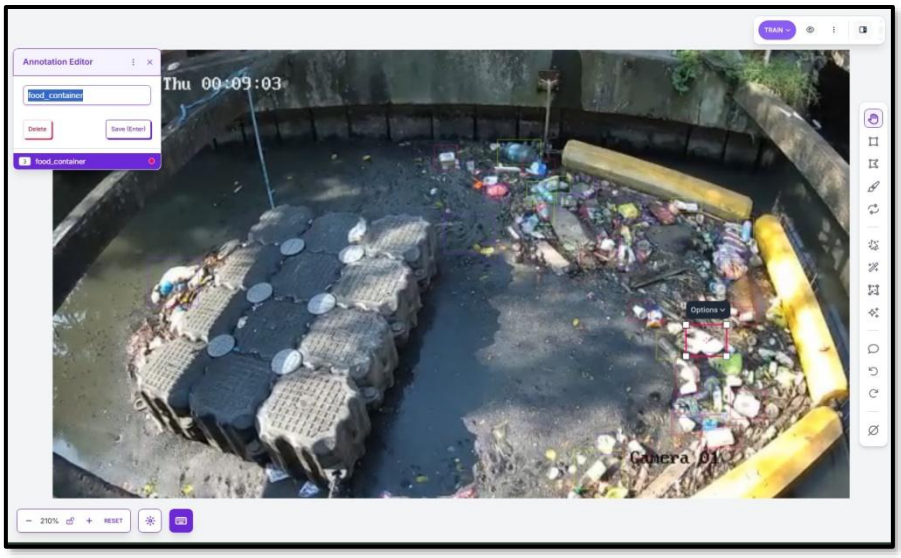
- **AI / Computer Vision:** Real-time detection and counting using a YOLO26n convolutional neural network exported to NCNN, a region of interest filter and ByteTrack tracking.
- **Front-End (UI/UX):** The touchscreen dashboard on the unit, an Android application for remote alerts, history and settings.
- **Back-End (Database):** A local SQLite database that keeps the detections, counts and snapshots and syncs them with Firebase.

The architecture keeps detection, processing and visualization in separate modules.

## Dataset Annotation and Model Training

**Figure 49**

*Annotating a Food Container in the Roboflow Annotation Editor*

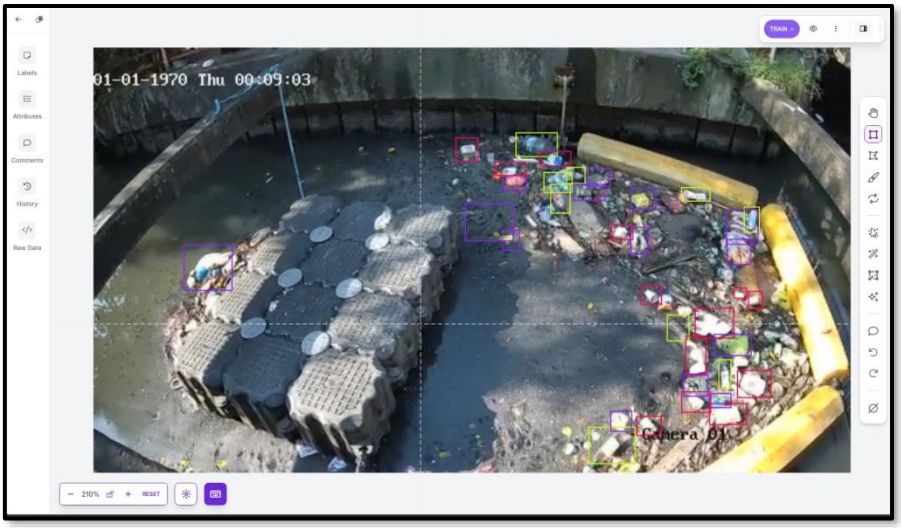



*Note.* Screenshot of the researchers' own Roboflow project, image 16 of 100 in a batch of creek frames captured on October 21, 2026.

Figure 49 shows the annotation editor of Roboflow (Roboflow, n.d.) open on one of the researchers' own creek frames. A box has just been drawn around a food container, and the editor asks for the class name before the box is saved. Every object in a frame is labelled this way: a tight bounding box, then a class. For this study the class list was limited to bottle, bag and food_container, in that order, so that the label numbers in the exported files match the class order the detector reports.

**Figure 50**

*A Fully Annotated Creek Frame in the Garbage-Guard Dataset*




*Note.* Screenshot of the researchers' own Roboflow project. The box colors distinguish the three classes.

Figure 50 shows the same frame with all of its boxes. One frame from the creek can hold dozens of labelled items, many of them small, partly submerged or touching one another, and each one was boxed by hand. This density is why annotation took most of the time in each sprint, and why the creek photographs changed the model more than the public images did.

The labeled images were exported from Roboflow in the YOLO text format, one file per image with a line for each box, and divided into the 70/10/20 training, validation and test partitions. Training did not run on Roboflow's servers. It ran locally on the laptop's RTX 2070 Max-Q, where the v7 run stopped early at epoch 93 of a planned 100, at an image size of 640 pixels and a batch size of 16, after about 1.4 hours. A run of that length made it practical to retrain whenever the evaluation exposed a weak class, which is the sprint cycle in Figure 47 and the return path in Figure 48.

The prototype was made portable. The Raspberry Pi, the UPS, the storage and the step-up cable are all inside a metal waterproof junction box, with holes cut in the sides for the cables and sealed with water sealant, and a USB case fan on each side. The unit is kept under shelter, and only the camera is placed outside, mounted above the water and wired back to the unit.

Models v8, v9 and v10 were trained on the same laptop with the same settings as v7, except that the image size was raised from 640 to 960 pixels, the detection size used on the Raspberry Pi. The batch size was lowered from 16 to 8 for v8 to fit the graphics memory and set back to 16 for v9 and v10. The v8 dataset had 3,667 training and 391 validation images. It added 157 frames from the prototype's camera taken at the creek on October 21, 325 bag images from public datasets, and a relabeled set of creek photographs that replaced 750 older ones. Training stopped at epoch 136 and the best epoch was 106. The v9 dataset had 3,742 training and 393 validation images. It added 57 frames from the prototype's camera (1280x720, October 28) and a few more creek photographs to the v8 data, and it trained for 120 epochs in 3.2 hours. The v10 dataset added 27 labelled pictures from the third data session, held at the creek on November 4, 2026, to the v9 data. v10 was trained with the same settings as v9, stopped early at epoch 109 of 120, and its best epoch was 79, after about 2.9 hours. The Roboflow exports list the classes as bag, bottle, food container, so the label numbers were remapped to the order bottle, bag, food container and the result was checked.

The frozen test set of 807 images was kept the same for every version and was never trained on, so the scores of v7, v8, v9 and v10 can be compared. Two steps kept test images out of training. Images were matched to the frozen test list by their original file names, because Roboflow changes the file contents when it exports again and a check on file contents alone would miss them. Photographs taken within 60 seconds, or within two shots, of a test photograph were also removed from training (163 images for v8 and

v9). Each merge script stopped if the test count was not 807 or if any test image appeared in the training or validation set.

A second test set was made from 59 frames of the prototype's own camera at 1280x720, with 2,621 labelled items (640 bottles, 998 bags and 983 food containers). These frames were kept out of the training of v7 and v8. v9 and v10 were trained on them, so they are not used to score those two models.

A third test set came from the third data session at the creek on November 4, 2026, which produced 58 pictures: 31 frames saved by the prototype and 27 photographs taken with a phone. Pictures taken within 30 seconds of each other show almost the same scene, so they were grouped as one burst, and every burst was placed whole on either the training side or the test side. This gave 28 pictures for training and 30 for testing, with prototype frames and phone photographs on both sides. Images from earlier sessions taken within 60 seconds of a test picture were also removed from training. Of the 30 test pictures, 19 were labelled, with 361 boxes, and these 19 form the session 3 test set used in Chapter IV. Only v10 was trained on pictures from this session, and none of the 19 test pictures was used in training.

**Assembly and Installation Procedure**

The unit was assembled and installed in the order below. The order matters at two points: the camera has to be in its final position before the monitored area is drawn, and the threshold is set only after the dashboard count has been compared with a hand count at the site.

***Assembly of the Unit***

1. The Waveshare UPS HAT (E) is fixed under the Raspberry Pi 5 so that its pogo pins meet the power pads on the underside of the board, and the four 21700 cells are loaded. The Active Cooler is attached to the Pi.

2. The Waveshare PCIe adapter is connected to the Pi's PCIe connector by its ribbon cable and the KingSpec SSD is fitted to it. The microSD card holding Raspberry Pi OS goes into the Pi's card slot.
3. The buzzer module is wired to the 40-pin header: VCC to pin 17 (3.3V), IN to pin 32 (GPIO12) and GND to pin 34.
4. The board stack is fixed inside the junction box, and the touchscreen is mounted behind the cut-out in the door and connected to the Pi by micro-HDMI for video and USB for touch.
5. The two USB case fans are mounted on the side walls and plugged into the USB ports of the power extension, and the UPS charger into one of its outlets. The three cable entries on the right-side wall are sealed with rubber sealant once the cables are through.

***Installation at the Creek***

6. The enclosure is placed under cover at the site. Only the camera is installed outside.
7. The camera is mounted above the water and aimed along the creek surface. Its Ethernet cable runs to the Pi's Ethernet port, and its power plug to the 5V to 12V step-up cable, which is connected through the USB hub inside the box to the UPS HAT's USB-A output.
8. The USB Wi-Fi adapter is plugged into the Vention USB 3.0 extension cable, set on top of the enclosure at the right, and joined to the site’s wireless network or a phone hotspot from the Wi-Fi setup screen of the Pi.
9. The extension lead is plugged into the mains and the Pi powers up. The launcher checks the camera, opens the dashboard full screen and starts detection; if no network is in range, the Wi-Fi setup screen appears first.

10. With the camera in its final position, the operator presses Draw on the Monitor tab, taps points along the waterline and presses Apply. The monitored area is saved and reloaded after every restart.
11. The threshold is calibrated on site. Over several sessions the dashboard's item count is compared with a hand count of the items visible on the water at the same moment, with the time and lighting recorded, and the Accumulation Limit in the Settings tab is set so that the alert fires at the real count wanted. The default is 20 items, re-arming at 15.
12. A test alert is raised from the Settings tab to confirm that the dashboard, the buzzer and the Android application all respond before the unit is left running.

## 3.3 Testing Procedure

This section explains how the testing objective was carried out: the parameters and scale of measurement used to grade the system, followed by the unit, system and acceptance tests of the third phase of the development model in Figure 47.

### *Testing Parameters and Scale of Measurement*

**Table 3**

*ISO/IEC 25010:2023-Based Metrics Descriptions and Scale of Measurements for Garbage-Guard*

| Objectives | | Definitions | Metrics | Description | Scale |
|---|---|---|---|---|---|
| 1. | Functional Suitability (Classification) | The ability of the YOLO26n model to correctly identify and classify solid garbage (bottles, bags, food containers). | Precision (%): the percentage of the detections made on the samples tested that are correct. | Excellent | 95% - 100% |
| | | | | Very Good | 90% - 94.99% |
| | | | | Good | 80% - 89.99% |
| | | | | Fair | 70% - 79.99% |
| | | | | Poor | Below 70% |
| 2. | Functional Suitability (Accumulation Detection) | The system's ability to accurately track the item count and successfully trigger an "Accumulation Alert" once the threshold (default 20 items) is reached. | Percentage average (%) of successful alert triggers out of total simulated accumulation test cases. | Excellent | 95% - 100% |
| | | | | Very Good | 90% - 94.99% |
| | | | | Good | 80% - 89.99% |
| | | | | Fair | 70% - 79.99% |
| | | | | Poor | Below 70% |

| Objectives | Definitions | Metrics | Description | Scale |
|---|---|---|---|---|
| 3. Performance Efficiency | The inference time and processing latency of Raspberry Pi 5 to draw bounding boxes on the live camera feed on each floating garbage. | Average inference time (in seconds) during real-time monitoring. | Excellent | Below 0.5s |
| | | | Very Good | 0.51s - 1.5s |
| | | | Good | 1.51s - 3.0s |
| | | | Fair | 3.1s - 5.0s |
| | | | Poor | Above 5.0s |

Table 3 shows and defines the metrics that will be used to grade each objective of the Garbage-Guard system based on the ISO/IEC 25010:2023 software quality standards.

(1) The first metric, Functional Suitability (Classification), measures how accurately the YOLO26n computer vision model identifies and classifies floating solid garbage such as bottles, bags, and food containers. Precision and recall are computed on the frozen test set from the confusion matrix counts of True Positives (TP), False Positives (FP) and False Negatives (FN). True negatives are not used, because in object detection there is no countable set of empty water that the model correctly leaves unboxed. In the field, where no reference boxes are drawn, classification is checked by counting the labels on the dashboard that name the item correctly, and accuracy is the share of correct labels among all labels counted. The formulas are:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{1}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{2}$$

$$\text{Accuracy} = \frac{\text{Correct Labels}}{\text{Total Labels Counted}} \times 100 \tag{3}$$

(2) The second metric adds a layer of Functional Suitability regarding the system's Accumulation Detection capability. The system counts the items found, and when the threshold is reached (20 items by default), it raises an "Accumulation Alert" to warn the people responsible about the level of pollution in the creek. The metric is the success rate, the number of successful alert triggers divided by the total number of trials, as in the equation below:

$$\text{Success Rate} = \frac{\text{Number of Successful Alerts}}{\text{Total Number of Trials}} \times 100 \quad (4)$$

(3) Lastly, the third metric, Performance Efficiency, measures the real-time responsiveness of Raspberry Pi 5. This metric evaluates the inference time and processing latency of the Raspberry Pi 5 to successfully process a live camera feed request and draw bounding boxes on each floating garbage without severe hardware degradation. The system automatically records the latency time for each frame, defined as the difference between the time the Raspberry Pi receives the frame (Start Time) and the frame detection completion time (End Time) to provide an objective evaluation. The last parameter is measured by applying the averaging method to determine the mean inference time (in seconds) over a fixed test time, as follows:

$$\text{Average Latency} = \frac{\sum(T_{\text{End}} - T_{\text{Start}})}{\text{Total Frames}} \quad (5)$$

***Phase 3: Validation Phase (V-Model At-End)***

This phase sets out the tests. Unit testing is described first because its cases are defined here, but in practice it closed every sprint of Phase 2. System testing and acceptance testing form the V-Model at-end and were run on the integrated unit.

**Unit Testing**

Unit testing is the last step of every sprint. It tests the software and hardware aspects of the system in a controlled environment. Depending on the scope and goal of the project, this includes isolating the YOLO26n model, the accumulation counter logic and the latency logging on the Raspberry Pi 5 to make sure they work correctly before the full integration. Its cases, listed in Table 4, were written during software design and were run again after every retraining of the model.

***1. Classification Evaluation***

To verify its software components, particularly the object detection algorithm, the frozen held-out test partition is used. The model is validated with the confusion matrix counts of True Positives (TP), False Positives (FP) and False Negatives (FN), using the measures defined in Equations 1 to 3:

- **Precision:** Measures the model's ability to avoid False Positives, that is, how many of the items the YOLO26n model identified as garbage were actually garbage (Equation 1).
- **Recall:** Measures the model's ability to detect all the actual garbage that is present, that is, how many of the garbage items in the water the YOLO26n model captured. In short, the model's ability to avoid False Negatives (Equation 2).
- **Accuracy:** The share of labels the system names correctly in the field, used for TC_ACC_01 (Equation 3). On the test set, mean average precision at an overlap threshold of 0.5 (mAP@0.5) is reported alongside precision and recall as the standard detection measure. The model must reach a precision of at least 80%.

***2. Accumulation Detection Evaluation (Counter Logic)***

To mathematically determine if the system correctly tracks item counts and triggers the "Accumulation Alert" at the 20-item threshold, simulated accumulation test cases are conducted.

**Success Rate:** Measures the percentage of successful alert triggers out of the total number of trials (Equation 4).

***3. Performance Efficiency Evaluation***

This measures the real-time responsiveness of the Raspberry Pi 5; the system automatically records the latency of each frame.

**Average Latency:** The mean inference time (in seconds) over a test period, used to confirm that there is no severe hardware degradation. It is the time from the moment the Raspberry Pi receives a frame (Start Time) to the moment detection and tracking finish (End Time), so camera encoding and network delay are not included (Equation 5).

**Table 4**

*Unit Testing Cases for Controlled Environment Evaluation*

| Test Case ID | Objective | Procedure | Expected Result | Actual Result | Successful Trials / 10 |
|---|---|---|---|---|---|
| TC_UNIT_01 | Evaluate Classification Precision, Recall and mAP@0.5 | Process the trained model on the frozen 20% of the annotated data (held-out test partition) and store the per class and overall results | The reported precision is at least 0.80 with recall and mAP@0.5 indicated for each of the three classes. | | Not counted out of 10 (one evaluation run) |
| TC_UNIT_02 | Validate Accumulation Counter Logic | Programmatically simulate the continuous detection of floating garbage items, incrementing up to the default threshold | The system triggers the automated "Accumulation Alert" once when the smoothed count reaches the 20-item threshold, and does not raise it again until the count has fallen to the re-arm level of 15. | | |
| TC_UNIT_03 | Verify the Performance Efficiency Logging | Run sample video frames through Raspberry Pi 5 to execute the latency tracking script. | The system is able to calculate the latency without hardware crashing, which is the difference between the start time and end time. | | |

## System Testing

After unit testing, system testing is performed. This stage tests the entire system: the Raspberry Pi 5, the camera, the YOLO26n model, the region filter, the tracker, the database, the touchscreen dashboard, the Android application and the alerting logic. The prototype is installed at a local creek for environmental monitoring. Beyond the model itself, the tests check that the system receives real-time video, alerts only on floating items visible in the video (excluding items that are sunken or hidden by other items), counts the items in the monitored area, stores every alert, notifies the user of an alert and

keeps processing efficiently under continuous load. System testing is the partner of system design. Table 5 is its system test plan, drafted when the hardware layout was fixed, as the dashed system test plan link in Figure 47 shows, and run in full on each build that passed unit testing.

**Table 5**

*System Testing Cases for Real-World Environmental Conditions*

| Test Case ID | Objective | Procedure | Expected Result | Actual Result | Successful Trials / 10 |
|---|---|---|---|---|---|
| TC_SYS_01 | Real-World Object Detection | Deploy the setup beside the waterways/creek and stream the live footage onto the Raspberry Pi 5 for real-time analysis | Bounding boxes are accurately drawn over visible floating bottles, bags, and food containers, ignoring sunken garbage or organic debris | | |
| TC_SYS_02 | Real-time Accumulation Alert | Allow the system to continuously monitor the creek until the valid target garbage items on the water surface reach the set threshold (20 items by default). | Successfully recognizes the threshold breach and triggers the automated accumulation alert in a live setting. | | |
| TC_SYS_03 | Real-time Processing Latency | Monitor the edge computing hardware's responsiveness while processing the live video feed for a standardized test duration. | The average inference latency per frame is logged automatically and remains stable without causing the system to crash or overheat. | | |
| TC_SYS_04 | System Flow (End to End) | Walk the flowchart in one run: switch the unit on, confirm the camera feed appears, place test items in the monitored water area, watch the count rise, cross the threshold, and observe the alert and the saved record. | Each step of Figure 31 happens in order: camera on, detection and tracking, counting inside the region, threshold alert, alert record saved, and the result shown on the dashboard and the application. | | |
| TC_SYS_05 | Live Dashboard | Open the dashboard on the touchscreen while detection runs. | The live feed with bounding boxes, class labels, the running count and the frame rate is shown and updates continuously. | | |
| TC_SYS_06 | Region of Interest | Draw the monitored area on the water, then place the same test item inside it and outside it. | The item inside the area is counted and the item outside it is ignored. | | |

| Test Case ID | Objective | Procedure | Expected Result | Actual Result | Successful Trials / 10 |
|---|---|---|---|---|---|
| TC_SYS_07 | Stable Tracking and Count | Let one item drift into the monitored area and out of it again. | The item keeps a single identity number while in view, adds one to the count while it is inside the area, and drops off the count after it leaves. | | |
| TC_SYS_08 | Event Storage | Raise an accumulation alert, then take a manual capture, then open the History tab. | Both records appear with their time, counts and a saved snapshot, read back from the database. | | |
| TC_SYS_09 | Alert Delivery | Raise an accumulation alert while the Android application is connected. | The alert is displayed on the dashboard, the buzzer is activated, and a push notification goes to the application. | | |
| TC_SYS_10 | Threshold Setting | Change the accumulation threshold in the Settings tab, then repeat the count. | The new threshold takes effect and the alert is raised at the new value. | | |

## Acceptance Testing

Acceptance testing is the partner of requirement gathering. Its acceptance test plan, made up of the test cases in Table 6, the developer-side indicators in Table 7 and the end-user survey in Table 8, was set with the adviser during that stage, as the dashed acceptance test plan link in Figure 47 shows. This phase demonstrates the working prototype live in the field to confirm that it meets the original aims of the study.

The prototype is operated at the creek on the deployment days covered by the permit in Appendix D (November 6, 7 and 9, 2026), after test sessions at the same site. The researchers maintain an uptime log of the number of days and hours the system ran without a crash or overheating, and the threshold calibration that took place on site. This log is the evidence for the Performance Efficiency indicators, and the end users answer the usability survey after using the system during the deployment.

To make the assessment accurate and set clear testing limits, this phase will be split into two perspectives: the developer side (Internal Acceptance) and the end-user side (External Acceptance).

**Developer Side Acceptance Testing**

The developer side evaluates only the technical performance, the stability of the hardware and the accuracy of the algorithms of Garbage-Guard. The evaluation is done by the researchers alone, based on the ISO/IEC 25010:2023 quality characteristics in the scope of the study. The researchers monitor the system during the live demonstration to see whether it reaches the target percentages and latency.

**Table 6**

*Acceptance Testing Evaluation Test Cases*

| Test Case ID | Objective | Procedure | Expected Result | Actual Result | Successful Trials / 10 |
|---|---|---|---|---|---|
| TC_ACC_01 | Functional Suitability (Classification) | Conduct a live field demonstration to evaluate the percentage average of correct garbage recognition against total samples. | The system achieves a target of "Good" (80%) or better in successfully classifying the 3 target garbage types. | | |
| TC_ACC_02 | Functional Suitability (Accumulation Detection) | Conduct a live field demonstration to check the percentage of successful alert triggers. | System achieves a target of "Good" (80%) or better in successfully triggering alerts when the set threshold (20 items by default) is reached. | | |
| TC_ACC_03 | Performance Efficiency (Latency and Stability) | Review internal logs generated during the live demonstration to assess edge computing latency | Raspberry Pi 5 maintains a target average inference time of "Good" (1.51s - 3.0s) or faster (below 1.5s) without crashing. | | Not counted out of 10 (computed from all logged frames) |

The demonstration will involve watching the system in the field and determining whether the prototype achieves the desired percentage averages and latency. This phase has structured test cases as shown in Table 6.

To quantify the results of the live demonstration formally, the researchers rate the system against the internal evaluation checklist in Table 7.

**Table 7**

*Developer-Side Internal Acceptance Indicators*

| ISO Characteristics | Indicator Code | Evaluation Statement |
|---|---|---|
| Functional Suitability (Target: Algorithm Accuracy) | DEV-FS1 | The YOLO26n algorithm accurately detects the three target types of floating garbage (bottles, bags and food containers) while ignoring sunken debris and organic matter. |
| | DEV-FS2 | In a live situation, the automated "Accumulation Alert" is activated when the set threshold (20 items by default) is reached with the system's counter logic ensuring it happens at the right time. |
| Performance Efficiency (Target: Hardware & Speed) | DEV-PE1 | The video stream is processed by the Raspberry Pi 5 with an average inference latency below 3.0 seconds per frame. |
| | DEV-PE2 | Edge computing hardware maintains uninterrupted operation without crashing or overheating. |

## End User Side Acceptance Testing

This boundary moves the focus from the software and hardware to the system's external interface and its usability. Target stakeholders from the local barangays will interact with the system's touchscreen monitor during the live field demonstration, and they make this evaluation.

To quantify the user acceptance, a survey based on the ISO/IEC 25010:2023 characteristics of Usability will be administered to the stakeholders immediately after the demonstration.

**Table 8**

*End User Side Acceptance Survey Indicators*

| ISO Characteristics | Indicator Code | Evaluation Statement |
|---|---|---|
| Usability (Target: Interface and Operability) | USR-U1 | Information displayed on the monitor is easy to read and understand. |
| | USR-U2 | Visual bounding boxes on the camera feed clearly showed where the garbage is located on the creek. |
| | USR-U3 | Accumulation Alert is clear, noticeable and effectively informs when the creek needs cleaning. |
| | USR-U4 | The system provides helpful insights that would assist local authorities in monitoring creek pollution. |
| | USR-U5 | The Android application shows the alerts and the history clearly, and what it shows matches the touchscreen. |

Each statement in Table 8 is rated on the five-point Likert scale in Table 9. The responses to a statement are summarized as a weighted mean, the sum of each scale value multiplied by the number of respondents who chose it, divided by the total number of respondents, and the mean is read against the ranges below. The usability target is a weighted mean of at least 2.61 (Good) for every indicator, matching the Good target used for the other characteristics in Table 3.

**Table 9**

*Five-Point Likert Scale and Interpretation of the Weighted Mean*

| Scale | Verbal Description | Weighted Mean Range | Interpretation |
|---|---|---|---|
| 5 | Strongly Agree | 4.21 – 5.00 | Excellent |
| 4 | Agree | 3.41 – 4.20 | Very Good |
| 3 | Neutral | 2.61 – 3.40 | Good |
| 2 | Disagree | 1.81 – 2.60 | Fair |
| 1 | Strongly Disagree | 1.00 – 1.80 | Poor |

$$\text{Weighted Mean} = \frac{\sum(f \times w)}{N} \tag{6}$$

where $f$ is the number of respondents who chose a scale value, $w$ is that scale value, and $N$ is the total number of respondents.

# IV. RESULTS AND DISCUSSION

This chapter presents the finished Garbage-Guard prototype and the results of the tests set out in Chapter III. Section 4.1 describes the prototype, where it was operated and how it was built, following the path of one camera frame through the system. Section 4.2 reports the test case results for each specific objective, the field deployment, the developer-side and end-user evaluation, and closes with the capabilities and limitations that the results support.

## 4.1 System Description and Structure

Garbage-Guard is a fixed monitoring unit that watches the surface of a creek, detects floating bottles, bags and food containers, and counts how many of them are inside a monitored area drawn over the water. When the smoothed count reaches the set threshold, the unit raises an Accumulation Alert on its touchscreen dashboard, sounds its buzzer, saves a snapshot and a record, and notifies the Android application. The purpose is to tell barangay personnel when a stretch of creek needs clearing, before the waste builds up enough to block the flow.

The prototype was operated at a creek in Barangay 870, Pandacan, Manila, under the barangay testing permit in Appendix C. The testing permit covers October 21, 2026, 1:00 to 3:00 PM, and the deployment permit in Appendix D covers November 6, 7 and 9, 2026, 10:00 AM to 4:00 PM. The October 31 session was an independent test held with the verbal permission of a barangay kagawad. It was also used to collect pictures for the dataset at several spots along the creek, not only at the monitored one.

**Figure 51**

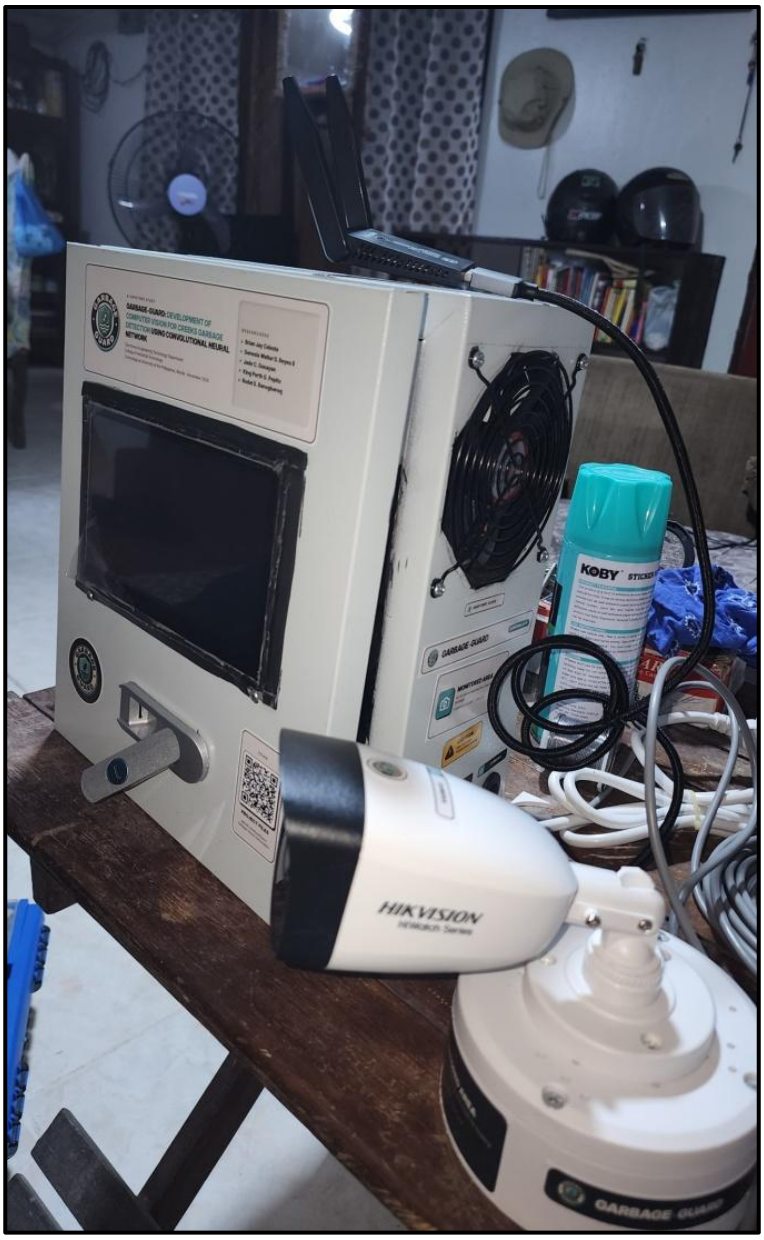
*The Assembled Garbage-Guard Unit with the Hikvision HiWatch HWI-B140H Camera*




Figure 51 shows the assembled unit from the front. The electronics are housed in a metal junction box with the 7-inch touchscreen set into the door and a locking handle below the screen. Stickers on the door carry the study title, the names of the researchers, the Garbage-Guard logo and a QR code that links to the project files. The Hikvision HiWatch HWI-B140H bullet camera is mounted separately on its own base and is joined to the box by its cable only.

**Figure 52**

*Side View of the Enclosure Showing the Case Fan and the USB Wi-Fi Adapter*

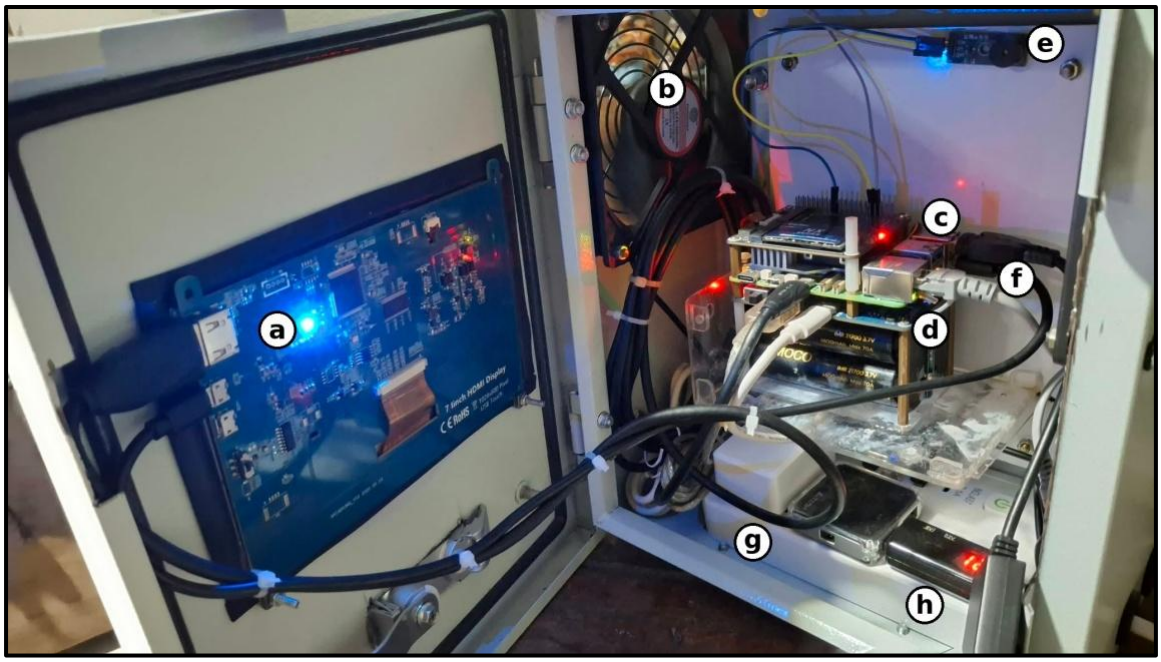



*Note.* The USB Wi-Fi adapter sits on top of the enclosure at the right, plugged into the Vention USB 3.0 extension cable that runs to the Raspberry Pi inside. The camera stands in the foreground on its base.

Figure 52 shows the right side of the enclosure with its case fan and the USB Wi-Fi adapter on top.

**Figure 53**

*Inside the Enclosure with the Main Parts Labelled*



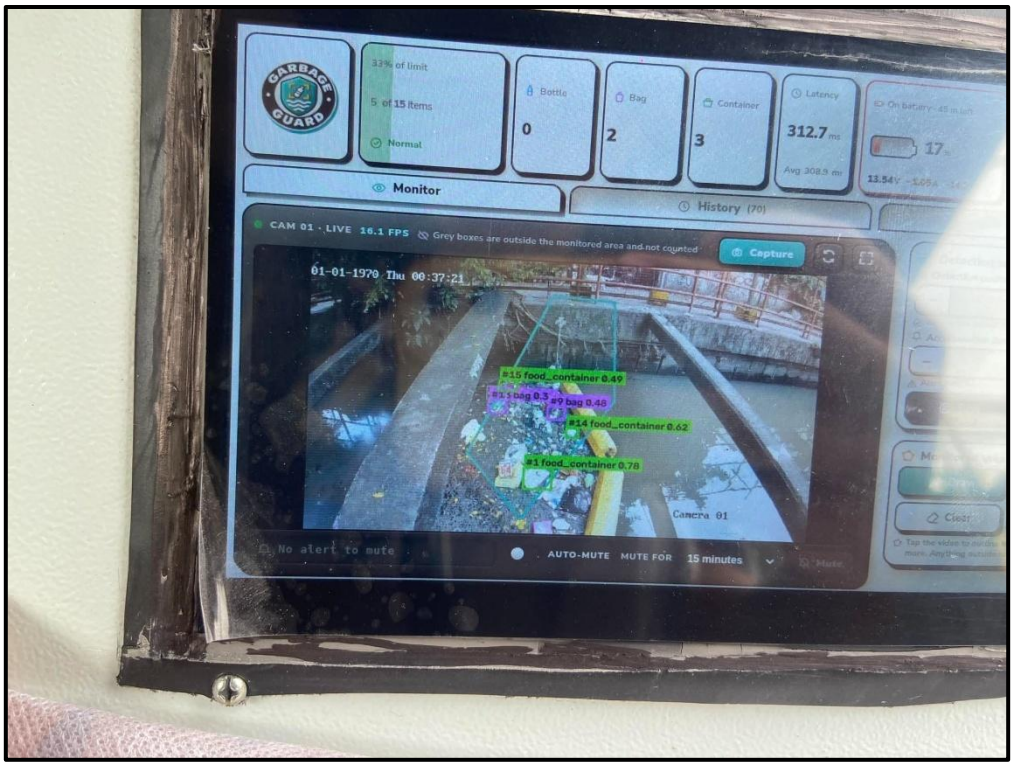

*Note.* (a) Back of the 7-inch 1024x600 HDMI touchscreen, mounted on the door; (b) USB fan; (c) board stack, from top to bottom: NVMe HAT, Raspberry Pi 5 with the Active Cooler, and Waveshare UPS HAT (E); (d) the four 21700 lithium-ion cells of the UPS HAT (E); (e) passive buzzer module; (f) 2-in-1 Ethernet and power cable to the camera; (g) official Raspberry Pi 27W (5.1V, 5A) power adapter, plugged into the AC mains extension; (h) 5V to 12V USB to DC barrel jack step-up cable for the camera, connected to a USB hub.

Figure 53 shows the inside of the enclosure with each part labelled.

During the session on October 31, 2026, the unit ran from 1:00 to 3:00 PM in sunny weather with normal daylight. Figure 54 shows the touchscreen during that session, with the monitored area drawn over the water and detections labelled by class, identity number and confidence.

**Figure 54**

*The Touchscreen Dashboard During the Field Session at the Creek*




*Note.* The date stamped in the camera image reads 01-01-1970 because the camera clock was not set on site. The Raspberry Pi keeps its own clock, so the records in the database carry the correct time.

### *System Structure*

The structure is explained by following one camera frame from the moment it is captured to the moment its result appears on the phone. The parts are the camera, the Raspberry Pi 5, the detection stage, the accumulation and alert stage, the database, the touchscreen dashboard, the Firebase link and the Android application. The overall layout is the system architecture shown in Figure 28 of Chapter III.

### Camera

The Hikvision HiWatch HWI-B140H sends its video over RTSP through an Ethernet cable connected directly to the Raspberry Pi 5. The camera is powered from the

UPS HAT through a USB hub, a 5V to 12V step-up cable and a 2-in-1 Ethernet and power cable, so the camera and the Raspberry Pi share one battery. The camera has two streams. The main stream, used for detection, was set to 1280x720 at 15 frames per second with H.264 encoding. The sub-stream, at 640x360, is kept as a fallback. The main stream at 1920x1080 was also tried, but the Raspberry Pi could not decode it fast enough and the picture fell six to seven seconds behind, so 1280x720 was chosen. At that setting the picture on the touchscreen was about one second behind the scene.

**Raspberry Pi 5**

The Raspberry Pi 5 runs Raspberry Pi OS (64-bit) and the Garbage-Guard program, written in Python with OpenCV. A separate reading thread keeps only the newest frame, so the detector never works on a backlog of old frames. If no frame arrives for 12 seconds the camera is reconnected, and if the main stream gives no picture twice in a row the program switches to the sub-stream on its own and shows a note on the dashboard. Internet access, needed for Firebase, comes through a USB Wi-Fi adapter mounted outside the enclosure, because the built-in Wi-Fi of the board could not join any network during testing.

**Detection**

Each frame is cropped to the box around the monitored area, with a small margin, before it reaches the model. Only that part of the picture is passed to YOLO26n, exported to NCNN for the Raspberry Pi, at a detection size of 960 pixels. Cropping keeps small items in the water large enough to detect and spends no processing time on the banks. ByteTrack gives each detected item an identity that stays the same while the item is in view, and the dashboard shows it as a short number next to the class name and the confidence.

**Accumulation and alert**

An item is counted when the center of its box lies inside the monitored area. To keep a single missed or extra detection from causing a false alarm, the count used for the

alert is the median of the last 15 frames. The Accumulation Alert is raised once when this smoothed count reaches the threshold. It is not raised again until the count has fallen back to the re-arm level, which is the threshold minus a quarter of it, rounded down (15 for the default threshold of 20, 12 for a threshold of 15), and at least 60 seconds have passed. A short spike that never lasts long enough to raise the alert is recorded as a suppressed event. The threshold is not fixed: it can be changed on the dashboard and in the application, so it can be calibrated to the size of each site. During the October 31 session it was set to 15 items. During the deployment it was tried at 15 and at 25 and ended at 20 (Table 17). When the alert is raised, the passive buzzer sounds and repeats until it is muted, except during the quiet hours set on the dashboard.

**Database**

Every alert, suppressed event, manual capture, test alert, mute and power event is written to a local SQLite database with its time, the count per class, the peak confidence, the threshold in use and the path of its snapshot. Each snapshot is saved with the boxes, the monitored area and an information card, and a clean copy without markings is saved beside it for labelling future training data. Every 100 frames the program also writes the mean, median and maximum latency to the database, which is the source of the latency results in Section 4.2.

**Touchscreen dashboard**

The dashboard is a local web page served by the Raspberry Pi on port 8080 and shown full screen on the 7-inch touchscreen. It has three tabs. Monitor shows the live picture with the detections, the count per class against the threshold, the latency, the battery level and the alert state, with buttons to capture a snapshot and to mute the alert. The monitored area is drawn here by tapping points around the water. History lists the stored events, each numbered and with a snapshot thumbnail, with filters by event type and date, and opens each snapshot. Settings holds the screen saver, power and automatic shutdown, buzzer and quiet hours, camera stream, box labels and the model upload.

**Firebase**

When a phone is not on the same network as the Raspberry Pi, the two communicate through a Firebase Realtime Database. The Raspberry Pi uploads its state, the history and compressed snapshots, and reads the commands the application leaves there, such as muting the alert. Uploads are kept small and slowed down when no phone is watching, so the free monthly allowance of Firebase is not exceeded.

**Android application**

The application connects to the Raspberry Pi directly on the local network, or through Firebase when it is away from it. It receives the Accumulation Alert as a notification with its own sound and vibration, shows the live picture, the history and the snapshots, and can mute the alert, change the detection settings and shut the Raspberry Pi down safely.

Figures 55 to 60 show the screens that satisfy each requirement. Figures 55 to 59 use the camera frame and counts of an accumulation alert recorded by the unit on November 4, 2026, in which 20 items were counted inside the monitored area. The complete set of screens is in Appendix H.

**Figure 55**

*Monitor Tab of the Touchscreen Dashboard with Live Detections (TC_SYS_05)*




**Figure 56**

*Dashboard During an Accumulation Alert (TC_SYS_02, TC_SYS_09)*



**Figure 57**

*Drawing the Monitored Area on the Dashboard (TC_SYS_06)*




**Figure 58**

*History Tab with the Stored Alerts and Snapshots (TC_SYS_08)*

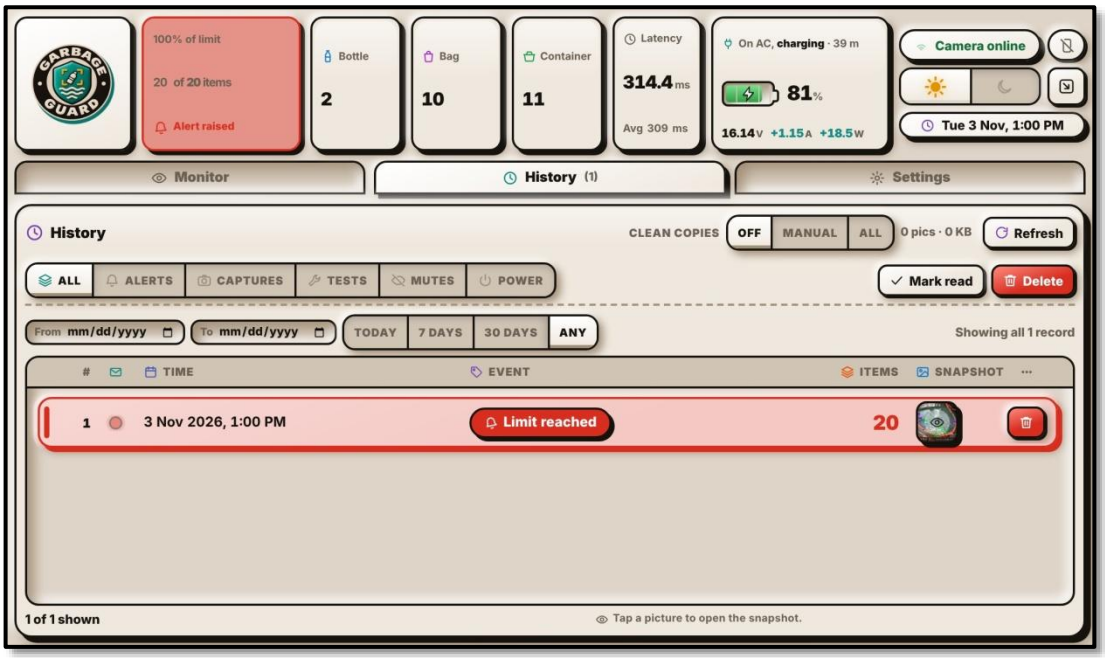


**Figure 59**

*Settings Tab with the Threshold, Camera Stream and Model Options (TC_SYS_10)*

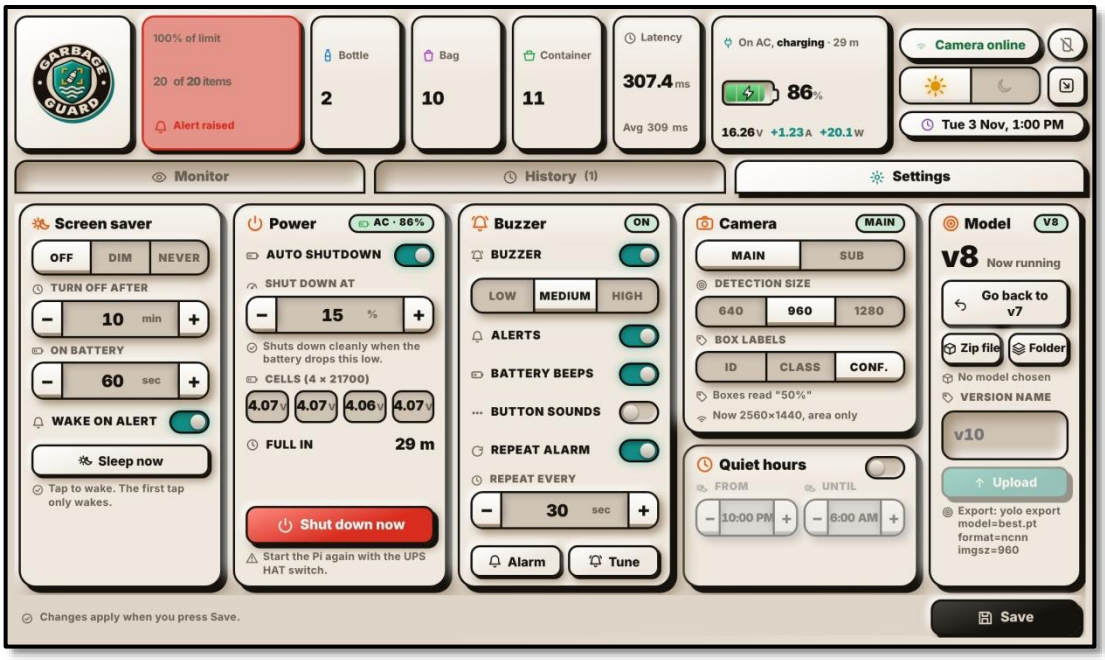



**Figure 60**

*Accumulation Alert Notification on the Android Application (TC_SYS_09)*



## 4.2 Test Results

The results are reported per specific objective, in the order of Section 1.2. Objectives 1 and 2 are covered in this section, and Objective 3, the evaluation under ISO/IEC 25010:2023, under Field Deployment Results and Evaluation Results later in this section. The results also follow the development model in Figure 47: the training runs in Table 10 are the development sprints, each closed by the unit tests in Table 14, while Tables 15 to 20 report the system and acceptance tests of the V-Model at-end.

### *Objective 1: Detection Performance of the YOLO26n Model*

#### Training history

Ten training runs were carried out: v1, v3, v4, v5, v6, v6b, v7, v8, v9 and v10. The dataset for v2 was assembled but not trained, because v3 replaced it. Table 10 lists every run with the change that defined it and its scores on its own test set.

**Table 10**

*Training Runs of the YOLO26n Model and Their Test Results*

| Run | Main change from the previous run | Test images | Precision | Recall | mAP@0.5 |
|---|---|---|---|---|---|
| v1 | Three public datasets, first proper train, validation and test split | 310 | 0.893 | 0.636 | 0.703 |
| v2 | Second bottle source added (assembled, not trained) | – | – | – | – |
| v3 | More bottle variety and TACO food containers | 357 | 0.828 | 0.655 | 0.708 |
| v4 | Class names generalized, more public data for all classes | 377 | – | – | 0.687 |
| v5 | 188 hand-labelled creek images added | 641 | 0.863 | 0.685 | 0.761 |
| v6 | Creek images increased to 934, training moved to a local RTX 2070 | 397 of 509 | 0.831 | 0.729 | 0.783* |
| v6b | Styrofoam labels repaired and capped at 400 images | 441 | 0.824 | 0.683 | 0.754 |
| v7 | Test partition frozen at 20% of the data | 807 | 0.852 | 0.685 | 0.745 |
| v8 | Relabelled creek set, new bag images, field captures, trained at 960px | 807 | 0.829 | 0.674 | 0.728 |
| v9 | 57 more prototype camera frames (1280x720, October 28) and a few more creek photographs, trained at 960px | 807 | 0.793 | 0.702 | 0.730 |

| Run | Main change from the previous run | Test images | Precision | Recall | mAP@0.5 |
|---|---|---|---|---|---|
| v10 | 27 photographs and prototype camera frames from the third data session added to the v9 data, trained at 960px | 807 | 0.846 | 0.672 | 0.744 |

*Note.* The scores of v1 to v6b are not directly comparable with each other or with v7 and later, because each of those runs had a different test set. v7 to v10 use the same frozen test set of 807 images. v8, v9 and v10 are shown at their training size of 960 pixels. *The v6 score is not valid: its data loader silently skipped all 1,092 styrofoam images in every split.

The first run, v1, combined three public datasets into the three classes and gave a mAP@0.5 of 0.703, with bags almost perfectly detected and bottles the weakest class. v3 added bottles from more varied scenes and a small number of food containers from TACO. Bottle recall rose from 0.429 to 0.492, but the overall score barely moved (0.708), which showed that more public data was giving smaller and smaller returns.

v4 renamed the classes from plastic-specific labels to bottle, bag and food container and took every usable image from TACO. The overall score fell to 0.687, because paper and carrier bags from TACO made the bag class less consistent. Public data had stopped helping, which led the researchers to collect their own creek images.

v5 was the first run with real creek images: 188 photographs taken and labelled by the researchers. The overall score rose to 0.761, and the two classes that received creek images, bottles and food containers, were the two that improved. This confirmed that images matching the deployment site mattered more than the amount of public data.

v6 increased the creek images to 934 and scored 0.783, but only 397 of its 509 test images loaded. The investigation found that every styrofoam image had been dropped because its label files mixed box and polygon lines, so the v6 score was not valid. v6b repaired those labels and capped the styrofoam source at 400 images. It scored 0.754 on a complete test set, which was a lower but honest result.

The rebuild for v6b also showed that rebuilding the dataset moved some test images into training. For v7 the test partition was therefore fixed once at 20% of the data, as the adviser required, and never rebuilt. v7 scored a precision of 0.852, a recall of 0.685 and a mAP@0.5 of 0.745 on 807 test images. It served as the reference for every run after it.

v8 replaced the old creek images with a relabelled set, added new bag images and images captured by the prototype itself, and was trained at 960 pixels to match the detection size on the Raspberry Pi. On the same frozen test set it scored slightly below v7 (mAP@0.5 of 0.728 against 0.745) and clearly below it on the creek-only images (0.497 against 0.593), while bags at the creek stayed at about 0.17 for both models. The likely reason is that v8 had fewer images resembling the old creek test photographs, since many of them were left out to keep the test set separate. The frozen test set is made of older phone photographs and does not show the sharper 1280x720 picture the prototype now uses, so the models were also compared on frames from the prototype's own camera (Table 13).

v9 added 57 frames from the prototype's camera and a few more creek photographs to the v8 data and was also trained at 960 pixels. On the frozen test set it scored a mAP@0.5 of 0.730 against 0.728 for v8, and on the creek-only photographs 0.530 against 0.497. Its precision on the frozen test set was 0.793, lower than the 0.829 of v8 and below the 80% target, while its recall was higher (0.702 against 0.674). A difference this small in mAP@0.5 is within what one training run can vary by, and the creek-only set holds only 46 bag boxes, so the researchers did not report v9 as better than v8. To settle the question on pictures that neither model had seen, the researchers held a third data session and kept half of it out of training as a test set.

The third data session took place at the creek in Barangay 870, Pandacan, on November 4, 2026 (Figure E9 in Appendix E). It produced 58 pictures: 31 frames saved by the prototype and 27 photographs taken with a phone. Pictures taken within 30

seconds of each other were treated as one burst, because they show almost the same scene, and every burst was placed whole on one side. This gave 28 pictures for training and 30 for testing, with frames from the prototype and phone photographs on both sides. Any older image taken within 60 seconds of a test picture was also kept out of training. Of these, 27 training pictures and 19 test pictures were labelled and used. The other 11 test pictures were not labelled, so the session 3 test set in this chapter holds 19 pictures with 361 boxes.

v10 is v9's data with the 27 labelled training pictures from the third session added. It was trained with the same settings as v9: 960 pixels, a batch of 16, up to 120 epochs and the same random seed. On the frozen test set it scored a precision of 0.846, a recall of 0.672 and a mAP@0.5 of 0.744. Its mAP@0.5 is close to v8 (0.728) and v9 (0.730), within what one training run can vary by. Its precision is clearly higher than v9's (0.846 against 0.793) and is back above the 80% target, while its recall is lower (0.672 against 0.702). In plain terms, v10 marks a few fewer items than v9, but it is right more often when it does.

On the test half of the third session, v10 reached a mAP@0.5 of 0.404 at 960 pixels, against 0.337 for v9, 0.204 for v8 and 0.149 for v7 (Table 13). Its precision there was 0.530 and its recall 0.367, against 0.330 and 0.266 for v8. This comparison favors v10 for a clear reason: v10 was trained on pictures from the same spot and the same day, and the other models were not. The test pictures themselves were never used in training, so the result shows how well v10 handles new moments at this creek, but it does not show how it would do at a different site. For a camera that stays fixed at one spot, this is the situation that matters. v10 is the reported model for the rest of this objective.

**Results of the reported model**

Table 11 gives the per-class results of v8, v9 and v10 on the frozen test set of 807 images and on its 180 creek images. The v7 results stay in Tables 10 and 12.

**Table 11**

*Per-Class Detection Results on the Frozen Test Set and on the Creek Images*

| Test set | Class | v8 P | v8 R | v8 mAP@0.5 | v9 P | v9 R | v9 mAP@0.5 | v10 P | v10 R | v10 mAP@0.5 |
|---|---|---|---|---|---|---|---|---|---|---|
| Full (807) | All classes | 0.829 | 0.674 | 0.728 | 0.793 | 0.702 | 0.730 | 0.846 | 0.672 | 0.744 |
| Full (807) | Bottles | 0.772 | 0.556 | 0.629 | 0.745 | 0.617 | 0.643 | 0.828 | 0.571 | 0.664 |
| Full (807) | Bags | 0.854 | 0.803 | 0.816 | 0.789 | 0.807 | 0.804 | 0.831 | 0.787 | 0.818 |
| Full (807) | Food containers | 0.861 | 0.662 | 0.739 | 0.845 | 0.682 | 0.742 | 0.881 | 0.657 | 0.750 |
| Creek (180) | All classes | 0.567 | 0.499 | 0.497 | 0.613 | 0.508 | 0.530 | 0.689 | 0.423 | 0.536 |
| Creek (180) | Bottles | 0.601 | 0.505 | 0.509 | 0.671 | 0.547 | 0.563 | 0.834 | 0.450 | 0.590 |
| Creek (180) | Bags | 0.248 | 0.261 | 0.174 | 0.287 | 0.261 | 0.204 | 0.312 | 0.152 | 0.194 |
| Creek (180) | Food containers | 0.851 | 0.732 | 0.807 | 0.881 | 0.716 | 0.823 | 0.921 | 0.666 | 0.825 |

*Note.* P is precision and R is recall. All three models were scored at 960 pixels, the size they were trained at, with the same script on the same computer. The creek-only bag rows rest on 46 boxes.

Precision (Equation 1) is the measure read against the Table 3 scale, following the 80% target in Section 1.3, with recall and mAP@0.5 reported beside it. On the full test set v10 reached a precision of 84.6%, which falls in the Good band (80% to 89.99%) and meets the 80% target. v8 also reached the Good band (82.9%), and v9 fell just short of it (79.3%, Fair). On the creek images alone the precision of v10 was 68.9%, in the Poor band (below 70%), although it was still the highest of the three models there (56.7% for v8 and 61.3% for v9). The difference between the two sets is expected, because the creek images are harder: items are smaller, partly submerged and mixed with leaves and other debris. Nunkhaw et al. (2025) reported the same kind of gap between controlled and field conditions for floating waste in urban canals.

Recall was lower than precision in every class. In plain terms, when the model marks an item it is usually right, but it misses some items altogether. Since missed items are not counted, the system is more likely to count too few items than too many, so an alert may come later than it would with a perfect detector, but rarely too early. Bags in the older creek photographs were the weakest result, with a recall of 0.152 on 46 boxes, lower than the 0.261 of v8 and v9. On the newer pictures of the third session, however,

v10 found bags far more often (recall of 0.427, Table 13), so the weakness is strongest on the older photographs. Food containers were the strongest class, with a mAP@0.5 of 0.825 on the creek images.

The precision of 84.6% on the full test set is close to the 85.9% overall accuracy that Nežerka et al. (2023) reported for convolutional recognition of waste fragments, although the two measures differ and their task was classification of single fragments rather than detection in a moving scene.

Figures 61 to 63 show the training curves, the precision-recall curve and the normalized confusion matrix of the v10 model. The precision-recall curve and the confusion matrix were made on the validation set, not on the frozen test set.

**Figure 61**

*Training and Validation Curves of the v10 Model*

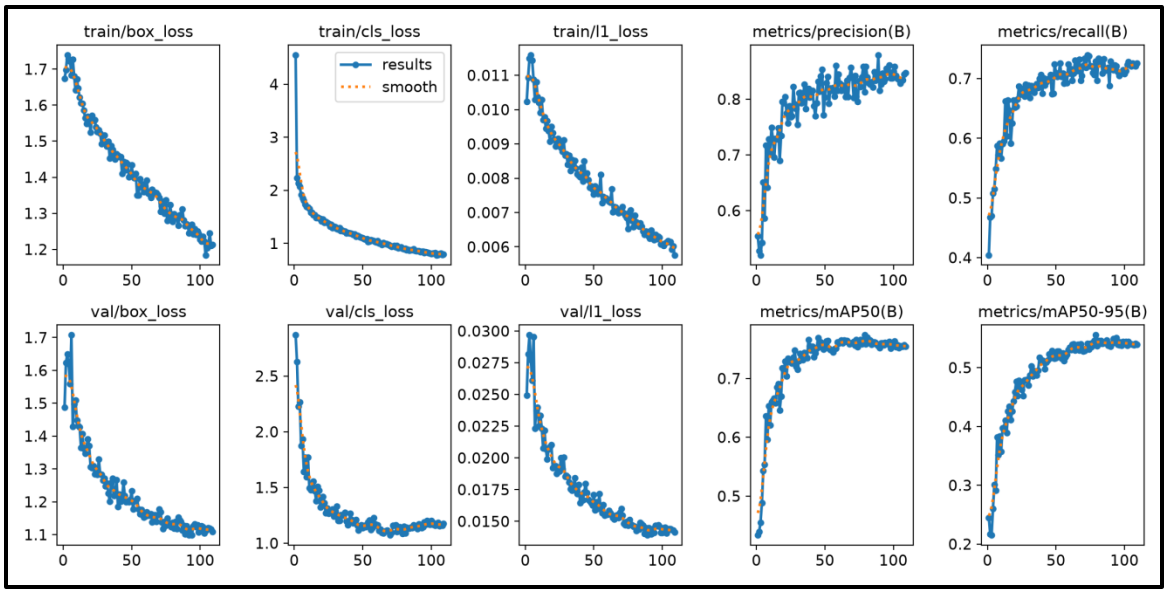


**Figure 62**

*Precision-Recall Curve of the v10 Model on the Validation Set*

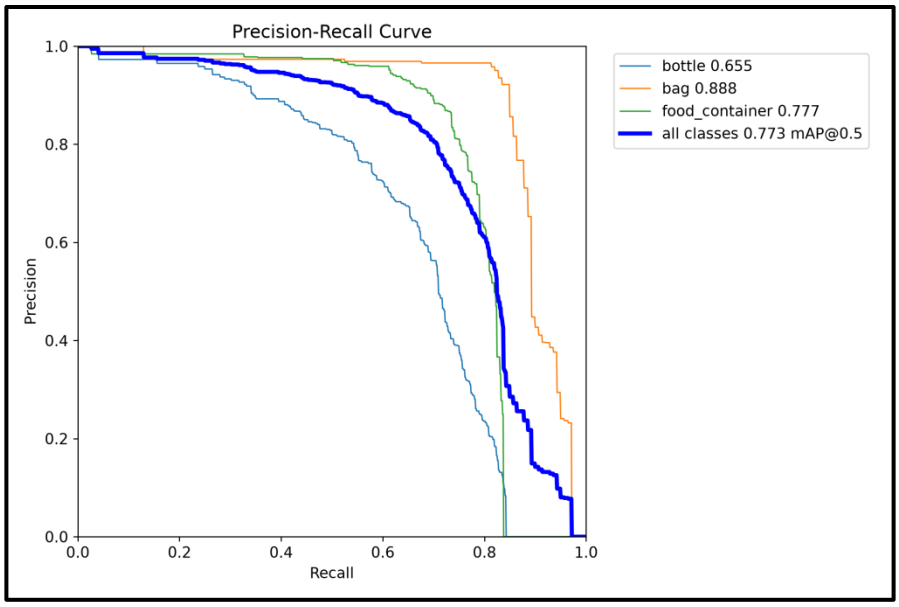
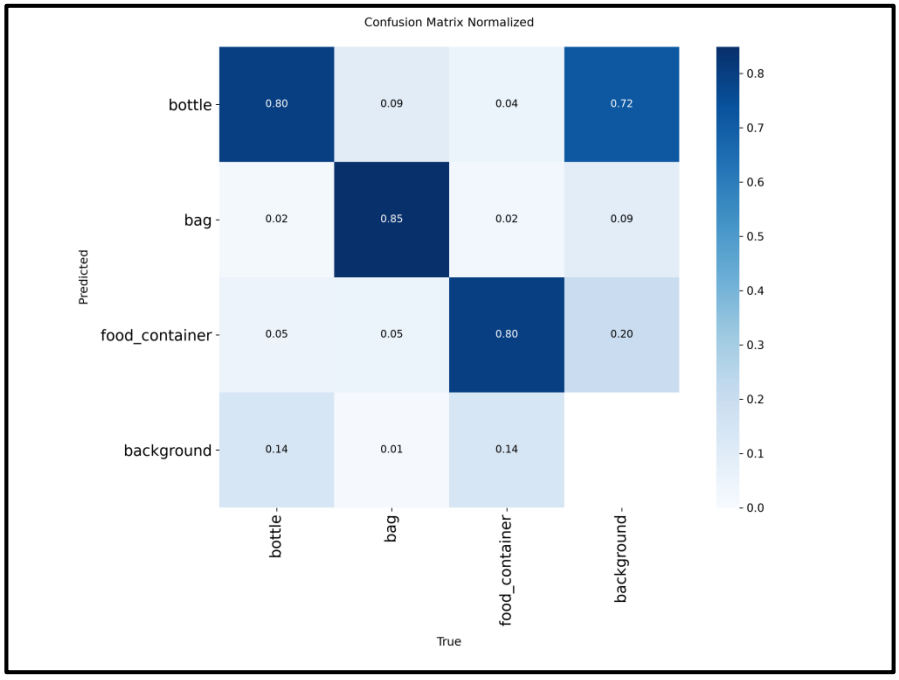



**Figure 63**

*Normalized Confusion Matrix of the v10 Model on the Validation Set*




The training curves in Figure 61 show the training losses falling steadily throughout the run. The validation mAP@0.5 rose quickly over the first 40 epochs and

levelled off at about 0.76 from epoch 50 onward. The validation classification loss stopped falling at about epoch 70 and then rose slightly, a sign that longer training would not have helped. Training stopped early at epoch 109 of 120, after 30 epochs without improvement. The best epoch was 79, and training took about 2.9 hours on the same laptop used for v7 and v8. On the validation set the best epoch reached a precision of 0.854, a recall of 0.698 and a mAP@0.5 of 0.773. These are higher than the frozen test scores in Table 11 because the validation images come from the same sources as the training images. v10 also has a different validation set from v8, so the validation scores of the two runs cannot be compared directly. The confusion matrix in Figure 63 shows that 85% of bags, 80% of bottles and 80% of food containers were classified correctly. Bottles and food containers were the classes most often missed, with 14% of each predicted as background. Of the false detections on background, 72% were labelled as bottles and 20% as food containers, so a false alarm is most likely to appear as a bottle.

Table 12 compares v7 to v10 on the same frozen test set, measured on the same computer with the same script at both image sizes. Table 11 shows the models at their training size of 960 pixels, while Table 12 also shows them at 640 pixels, the size v7 was trained at.

**Table 12**

*mAP@0.5 of v7 to v10 on the Frozen Test Set and the Creek-Only Photographs*

| Test set | Class | Image size | v7 | v8 | v9 | v10 |
|---|---|---|---|---|---|---|
| Full (807) | All classes | 640px | 0.745 | 0.724 | 0.725 | 0.728 |
| Full (807) | All classes | 960px | 0.715 | 0.728 | 0.730 | 0.744 |
| Full (807) | Bottles | 960px | 0.626 | 0.629 | 0.643 | 0.664 |
| Full (807) | Bags | 960px | 0.782 | 0.816 | 0.804 | 0.818 |
| Full (807) | Food containers | 960px | 0.737 | 0.739 | 0.742 | 0.750 |
| Creek (180) | All classes | 640px | 0.593 | 0.540 | 0.554 | 0.538 |
| Creek (180) | All classes | 960px | 0.563 | 0.497 | 0.530 | 0.536 |
| Creek (180) | Bags | 960px | 0.232 | 0.174 | 0.204 | 0.194 |

*Note.* The creek-only bag row rests on 46 boxes. All values come from the same evaluation script, run on the same computer.

On the older phone photographs the four models are close. At 960 pixels v10 has the highest mAP@0.5 on the full test set (0.744) and in every class, but its lead over v8 and v9 is about 0.02 or less, which is within ordinary run-to-run variation. On the creek-only photographs v7 is ahead only at 640 pixels, the size it was trained at.

Table 13 shows the scores on pictures from the creek that the models were not trained on. The first rows are the 59 frames from the prototype's camera taken on October 28. These frames are crowded, with about 44 labelled items in each, and many of the items are very small. The remaining rows are the test half of the third session.

**Table 13**

*mAP@0.5 on Pictures from the Prototype's Camera and on the Third-Session Test Half*

| Test set | Class | Image size | v7 | v8 | v9 | v10 |
|---|---|---|---|---|---|---|
| Prototype frames (59) | All classes | 640px | 0.019 | 0.108 | – | – |
| Prototype frames (59) | All classes | 960px | 0.056 | 0.170 | – | – |
| Prototype frames (59) | Bags | 960px | 0.054 | 0.234 | – | – |
| Session 3 test (19) | All classes | 640px | 0.088 | 0.184 | 0.298 | 0.300 |
| Session 3 test (19) | All classes | 960px | 0.149 | 0.204 | 0.337 | 0.404 |
| Session 3 test (19) | Bottles | 960px | 0.242 | 0.287 | 0.356 | 0.397 |
| Session 3 test (19) | Bags | 960px | 0.085 | 0.185 | 0.400 | 0.445 |
| Session 3 test (19) | Food containers | 960px | 0.120 | 0.141 | 0.255 | 0.369 |

*Note.* v9 and v10 are not scored on the 59 prototype frames because both were trained on them. Only v10 was trained on pictures from the third session; the test pictures themselves were not used by any model. The session 3 test set holds 19 labelled pictures with 361 boxes. The test shrinks each whole picture to the image size, while the prototype crops the frame to the monitored area first, so the live system sees more detail than this test.

On the 59 frames from the prototype's camera, v8 scored clearly higher than v7 (0.170 against 0.056 mAP@0.5 at 960 pixels). v8 was trained on frames from the same

creek spot on another day and v7 was not. The same pattern appears in the third session: the models that had seen more pictures from this spot scored higher, and v10, the only one trained on pictures from that day, scored highest in every class, with a mAP@0.5 of 0.404 against 0.204 for v8. This is fair for a camera fixed at one spot, but it means the result may not carry over to a new site. Every model scored lower at 640 pixels than at 960 pixels on these pictures, by 0.02 to 0.10, which supports keeping the detection size at 960 pixels on the Raspberry Pi. The scores are low for every model because the pictures are crowded and hold many small, partly submerged items. The models were scored on still images, so these results do not show how the system behaves live, such as flicker or false alarms from water and shadows.

A new model is installed from the Model card in the Settings tab, as a zip file or a folder under a version name. The Raspberry Pi checks it before use and keeps the running model if the check fails, and the card goes back to the previous model in one tap. Before v10 was trained, the researchers set a rule that a new model is put on the unit only if it gains at least 0.03 mAP@0.5 on camera pictures it has not seen, or clearly improves bag recall, and runs well live. On the third-session test half v10 met the first two conditions, with a mAP@0.5 0.200 higher than v8 and a bag recall of 0.427 against 0.267. On November 6, v10 ran on the unit for six hours. The live feed ran at 15 frames per second or more, the average processing time was 275ms per frame, and v10 made no false detections that described the floating barrier or other fixed objects as an item. The Raspberry Pi therefore runs v10 as its main model, with v8 kept as the fallback.

Table 14 gives the unit test results.

**Table 14**

*Unit Testing Results*

| Test Case ID | Objective | Expected Result | Actual Result | Successful Trials / 10 |
|---|---|---|---|---|
| TC_UNIT_01 | Evaluate classification precision, recall and mAP@0.5 | Precision of at least 0.80, with recall and mAP@0.5 indicated for each class | On the same 807 test images (Table 11): v10, the reported model, precision 0.846, recall 0.672, mAP@0.5 0.744, pass. For reference, v8 0.829, 0.674, 0.728, pass; v9 0.793, 0.702, 0.730, below the 0.80 precision target. | One evaluation run (not counted in trials) |
| TC_UNIT_02 | Validate the accumulation counter logic | Alert raised once at the 20-item threshold and not again until the count falls to 15 | The 19 automated counter tests passed. In 10 simulated runs on the Raspberry Pi, the alert came once at the threshold and not again until the count fell to the re-arm level. | 10 |
| TC_UNIT_03 | Verify the performance efficiency logging | Latency is calculated without the hardware crashing | Latency summaries were written every 100 frames. A recorded video was played through the Raspberry Pi 10 times without a crash. The average latency was about 180ms on the sub-stream with v7 and 250 to 300ms with v8 and later models. | 10 |

### *Objective 2: Operation of the Raspberry Pi 5 Prototype*

Table 15 gives the results of the system test plan in Table 5, run on the unit in the field.

**Table 15**

*System Testing Results*

| Test Case ID | Objective | Actual Result | Successful Trials / 10 |
|---|---|---|---|
| TC_SYS_01 | Real-world object detection | Boxes appeared on the visible bottles, bags and food containers, and nothing was boxed on sunken items or leaves. | 10 |
| TC_SYS_02 | Real-time accumulation alert | The Accumulation Alert was raised when the count passed the threshold. | 10 |
| TC_SYS_03 | Real-time processing latency | Latency stayed steady over the run, with no crash or overheating. The average was 275 to 350ms (Table 19). | 10 |

| Test Case ID | Objective | Actual Result | Successful Trials / 10 |
|---|---|---|---|
| TC_SYS_04 | System flow (end to end) | Every step happened in order, from switching on to the notification on the application (see the system flow test below). | 10 |
| TC_SYS_05 | Live dashboard | The Monitor tab kept updating the boxes, labels, count and frame rate. | 10 |
| TC_SYS_06 | Region of interest | The item was counted inside the monitored area and ignored outside it. | 10 |
| TC_SYS_07 | Stable tracking and count | The item kept the same ID number while it drifted in and out of the area, and the count went up by one and then back down. | 10 |
| TC_SYS_08 | Event storage | Both the alert and the Capture were stored in History with the time, counts and snapshot. | 10 |
| TC_SYS_09 | Alert delivery | The dashboard showed the alert, the buzzer sounded and the phone received a notification about 0.1 to 2 seconds later on the local network. Over Wi-Fi through the cloud the notification took 5 seconds or more. | 10 |
| TC_SYS_10 | Threshold setting | The alert came at the new threshold value each time the count was repeated. | 10 |

During the October 31 session, most detections were bottles, bags and food containers correctly labelled. The floating barrier across the creek was at first detected as an item. Redrawing the monitored area so that it excluded the barrier removed these false detections, which is the purpose of the region of interest tested in TC_SYS_06. On November 6 the unit made no false detections that described the barrier as an item.

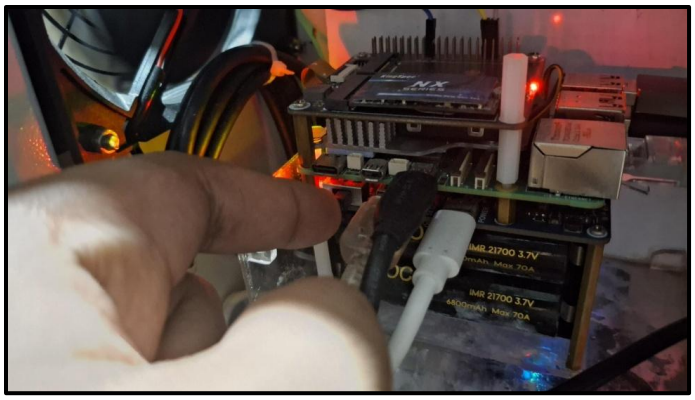
**System Flow Test**

The system flow in Figure 31 was run on the bench on the evening of November 6, 2026, with the camera pointed at bottles on the floor instead of at the creek. The accumulation limit was lowered to 3 items so that a few bottles would reach it. The creek sessions used a limit of 15 to 25 (Tables 16 and 17). Each step is shown below as it appeared on the unit.

Step 1, the unit is switched on. The unit starts when the power switch of the UPS HAT inside the enclosure is pressed (Figure 64).

**Figure 64**

*Switching On the Unit with the UPS HAT Power Switch*

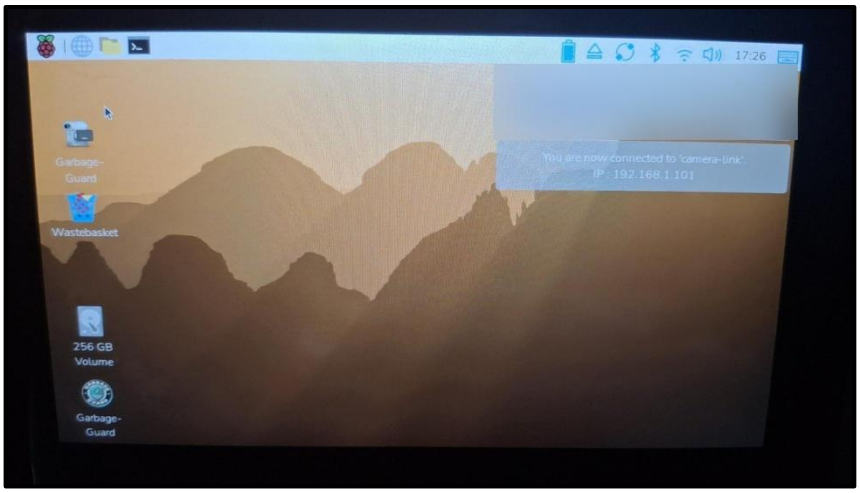



*Note.* Bench test on November 6, 2026. The finger rests on the power switch of the UPS HAT inside the enclosure.

Step 2, the Raspberry Pi starts. The desktop appears with the Garbage-Guard shortcut, and the notice at the top right shows that the link to the camera is connected (Figure 65). The shortcut opens the dashboard.

**Figure 65**

*Raspberry Pi Desktop After Start-Up*

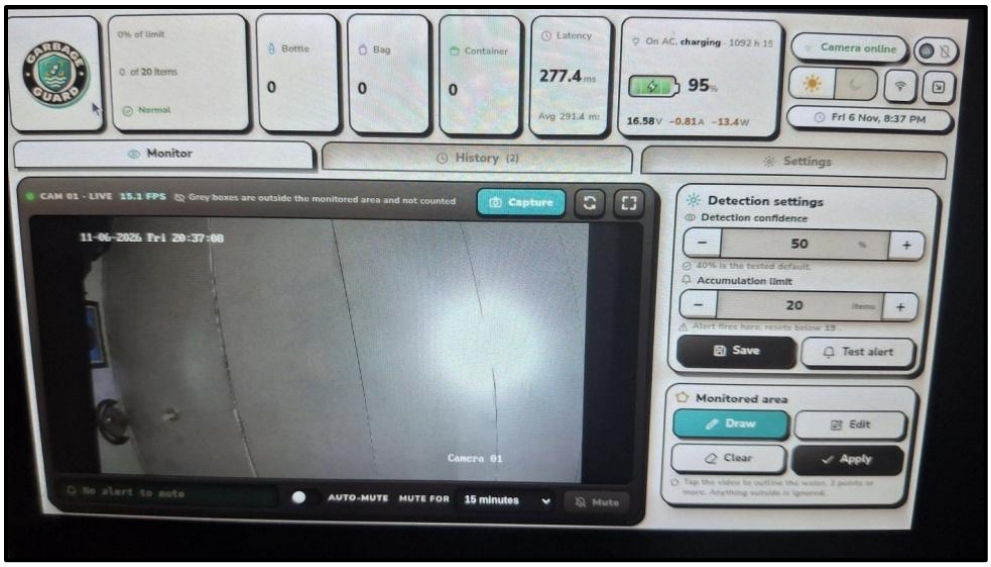



*Note.* Bench test on November 6, 2026. The desktop shows the Garbage-Guard shortcut and the notice that the camera link is connected. The Raspberry Pi clock had not yet taken the network time, so it reads 17:26. The notice about the Wi-Fi network is blurred.

Step 3, the camera feed is shown. The Monitor tab shows the live picture from the camera with the frame rate, the count of items inside the monitored area, the three class counters and the latency (Figure 66). With nothing in the area the count is 0 of 20.

**Figure 66**

*Monitor Tab with the Live Camera Feed and No Items in the Area*

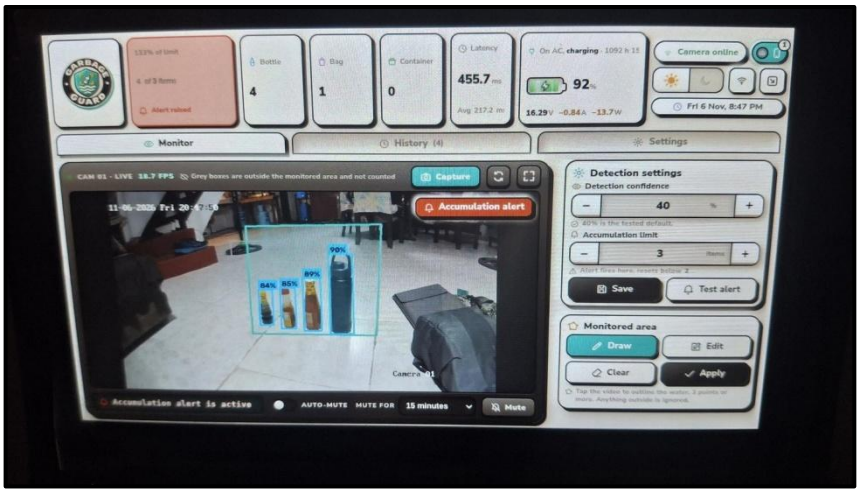



*Note.* Bench test on November 6, 2026, 8:37 PM. The feed runs at 15.1 frames per second with a latency of 277ms, and the count is 0 of 20 items.

Steps 4 to 6, items are placed, the count reaches the limit and the alert is raised. With four bottles inside the monitored area, each is boxed with its class and confidence, between 84% and 90%. The count of 4 passed the limit of 3, the counter turned red and read 133% of the limit with the words Alert raised, and the Accumulation alert banner appeared over the picture, with the message Accumulation alert is active below it (Figure 67).

**Figure 67**

*Accumulation Alert on the Touchscreen with the Limit Set to 3*

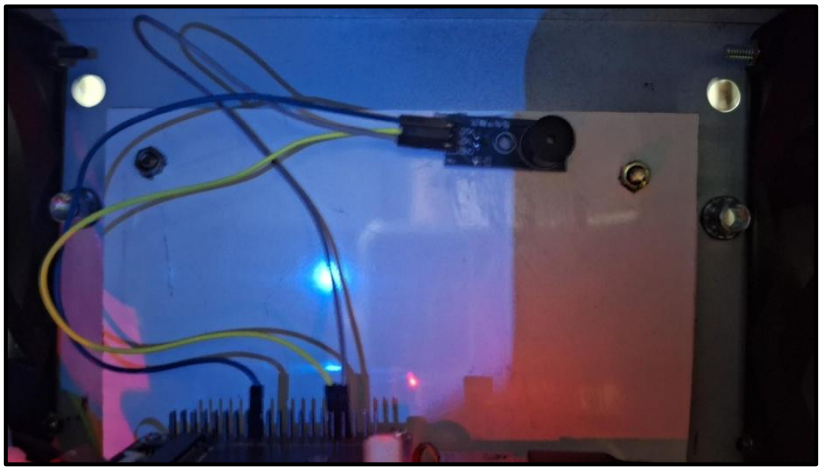



*Note.* Bench test on November 6, 2026, 8:47 PM, with four bottles on the floor and the accumulation limit lowered to 3 items. The monitored area is outlined in cyan.

Step 7, the buzzer sounds. The same event sounds the buzzer in the enclosure (Figure 68). The buzzer sounded in all 10 trials of TC_SYS_09 (Table 15).

**Figure 68**

*Buzzer Module Inside the Enclosure*

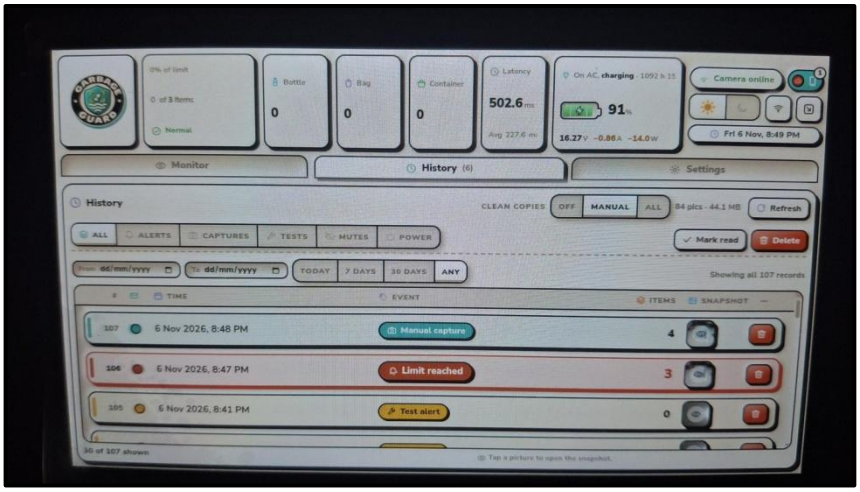


*Note.* The buzzer module is wired to the GPIO pins of the Raspberry Pi. A photograph cannot show the sound, which was heard in every trial of TC_SYS_09.

Step 8, the record is saved. The alert was stored in the History tab as record 106, Limit reached, at 8:47 PM with the number of items and a snapshot (Figure 69).

**Figure 69**

*History Tab with the Saved Limit-Reached Record*

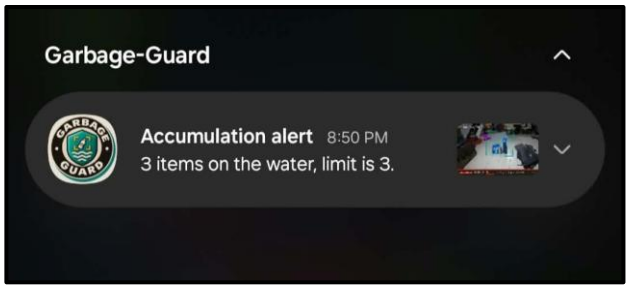



*Note.* Bench test on November 6, 2026, 8:49 PM. Record 106 is the alert at 8:47 PM with 3 items. Record 107 is a manual capture of 4 items, and record 105 is the test alert sent at 8:41 PM.

Step 9, the application is notified. A notification reached the phone with the number of items, the limit and the snapshot (Figure 70). The Monitor tab of the application then showed the same alert and the same counts as the touchscreen (Figure 71).

**Figure 70**

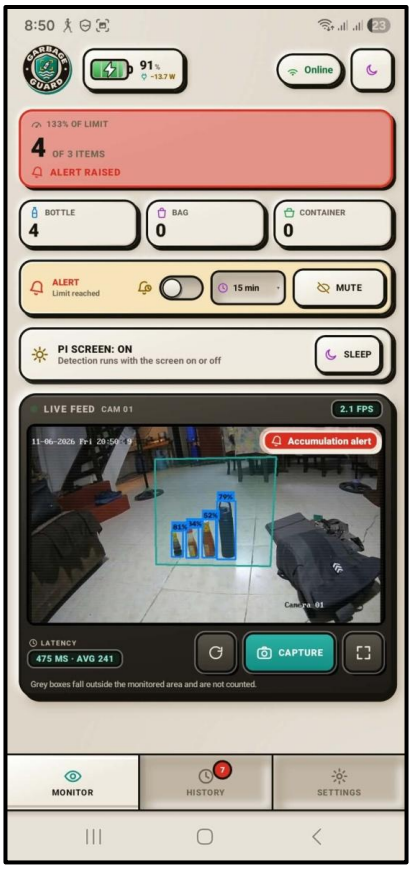
*Accumulation Alert Notification on the Android Phone*




*Note.* November 6, 2026, 8:50 PM. The notification carries the snapshot of the camera picture.

**Figure 71**

*Monitor Tab of the Android Application During the Alert*




*Note.* November 6, 2026, 8:50 PM. The application shows the same alert and class counts as the touchscreen.

### *Field Deployment Results*

The unit was operated on six days: October 17, October 31 and November 4, which were test sessions, and November 6, 7 and 9, the three days of the pilot deployment under the permit in Appendix D. Table 16 lists each session. The camera hung about 13ft above the water on November 6. The team lowered it to about 9ft on November 7 and to about 7ft on November 9.

**Table 16**

*Field Deployment Log*

| Date | Time | Hours | Model and detection size | Camera stream | Threshold | Weather | Crashes or overheating |
|---|---|---|---|---|---|---|---|
| October 17, 2026 | 1:00 to 2:00 PM | 1 | v7 | Sub-stream, 640px, 15fps | 5 | Cloudy, 30°C | None; nothing was detected |
| October 31, 2026 | 1:00 to 3:00 PM | 2 | v8, 960px | Main, 1280x720, 15fps | 15 | Sunny, 35°C | None |
| November 4, 2026 | 10:00 AM to 1:00 PM | 3 | v9 | Main, 1280x720, 15fps | 20 | Sunny, 34°C | None |
| November 6, 2026 | 10:00 AM to 4:00 PM | 6 | v10 | Main, 1280x720, 15fps | 20 | Sunny, 33°C | None |
| November 7, 2026 | 10:00 AM to 4:00 PM | 6 | v10 | Main, 1280x720, 15fps | 20 | Sunny, 31°C | None |
| November 9, 2026 | 10:00 AM to 4:00 PM | 6 | v10 | Main, 1280x720, 15fps | 20 | Sunny, 30°C | None |

Across the six sessions the unit ran for 24 hours without a crash, a freeze or overheating. The detection threshold was 5 in the first session with v7, which detected nothing, and 15 to 20 in the later sessions. The highest detection confidence the team noted on the sheet rose from about 30% with v8 to about 50% with v9 and about 60% with v10 on November 6. On November 7 and November 9, with v10 and the monitored area in use, the sheet records an average confidence of 70 to 75% and 75 to 80%, with no issues.

The latency is measured from the moment a decoded camera frame reaches the program to the moment detection and tracking of that frame are finished, as defined in Equation 5. It does not include the camera's own encoding and network delay, which

added about one second between the scene and the touchscreen. During the October 31 session the dashboard showed an average of about 309ms (0.31s) per frame with v8 at 960 pixels, which falls in the Excellent band (below 0.5s). On November 6 the average with v10 was 275ms (0.28s), fluctuating between 250 and 300ms, and over the three deployment days the average stayed between 275 and 350ms (0.28 to 0.35s), also in the Excellent band, while the live feed ran at 15 frames per second or more. In bench tests with the main stream at 1280x720, the median latency was 186 to 191ms with about five frames processed per second, and the processor stayed at 68°C without throttling. Running a remote screen-sharing session on the Raspberry Pi at the same time raised the median to about 405ms, so all reported latencies were measured without screen sharing. Li et al. (2024) reported 33 frames per second for a very small YOLO model on an edge device. Garbage-Guard processes fewer frames per second, which is adequate for this task because floating waste moves slowly and the count is smoothed over 15 frames, about three seconds.

**Table 17**

*Threshold Calibration Record*

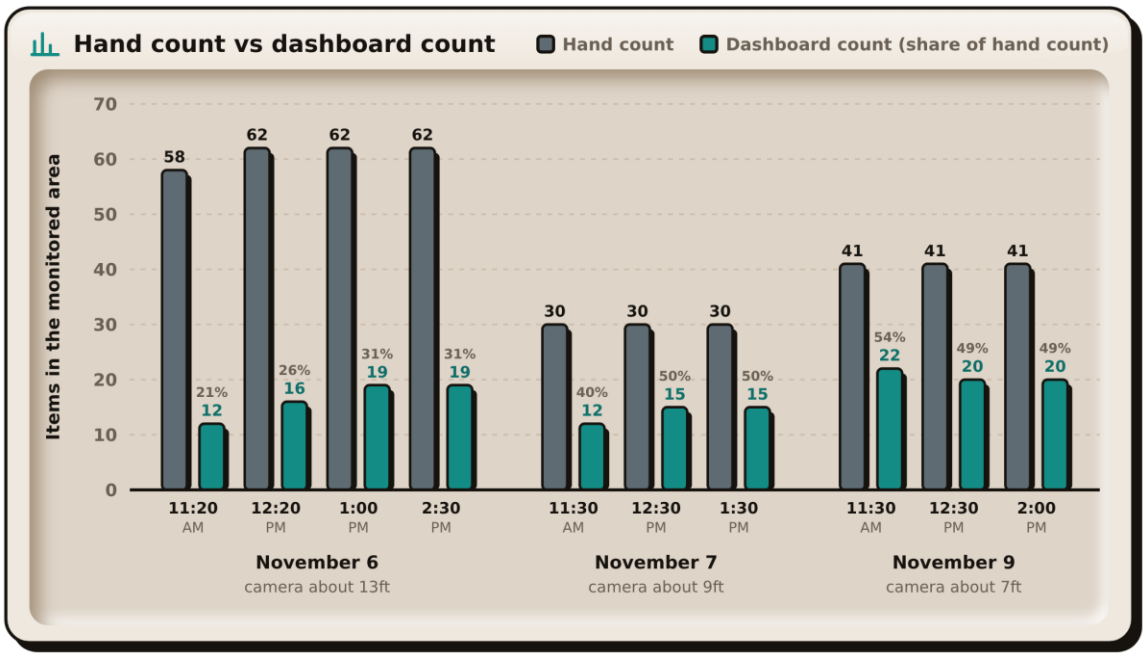
| Date and time | Hand count in the monitored area | Dashboard count | Difference | Threshold before and after adjustment |
|---|---|---|---|---|
| November 6, 11:20 AM | 58 (32 food containers, 11 bottles, 15 bags) | 12 | 46 | 20 to 15 |
| November 6, 12:20 PM | 62 (35 food containers, 12 bottles, 15 bags) | 16 | 46 | 15 to 20 |
| November 6, 1:00 PM | 62 (35, 12, 15) | 19 | 43 | 20 to 20 |
| November 6, 2:30 PM | 62 (35, 12, 15) | 19 | 43 | 20 to 20 |
| November 7, 11:30 AM | 30 (12 food containers, 6 bottles, 12 bags) | 12 | 18 | 20 to 15 |
| November 7, 12:30 PM | 30 (12, 6, 12) | 15 | 15 | 15 to 15 |
| November 7, 1:30 PM | 30 (12, 6, 12) | 15 | 15 | 15 to 15 |

| Date and time | Hand count in the monitored area | Dashboard count | Difference | Threshold before and after adjustment |
|---|---|---|---|---|
| November 9, 11:30 AM | 41 (18 food containers, 8 bottles, 15 bags) | 22 | 19 | 20 to 25 |
| November 9, 12:30 PM | 41 (18, 8, 15) | 20 | 21 | 25 to 20 |
| November 9, 2:00 PM | 41 (18, 8, 15) | 20 | 21 | 20 to 20 |

Note. The difference is the hand count minus the dashboard count. On November 6, with the camera about 13ft above the water, the dashboard counted 21%, 26%, 31% and 31% of the items counted by eye, which agrees with the field team's notes of about 20 to 30% detected. The team recorded that most of the trash was submerged or half submerged. Changing the threshold from 20 to 15 and back did not change what the dashboard counted, because the shortfall came from items the model did not detect and not from where the alert was set. After the camera was lowered to about 9ft, the dashboard counted 40%, 50% and 50% of the items on November 7, and with the camera at about 7ft it counted 54%, 49% and 49% on November 9. About half of the items were still missed. On November 7 the threshold was lowered to 15 at the first reading, when the count was 12. At the next two readings the count reached 15 and the alarm buzzed. On November 9 the count of 22 reached the threshold of 20 and the alarm buzzed, and the team raised the threshold to 25. The count then stayed at 20 to 22, so they lowered it back to 20, and at the last reading the count of 20 reached it and the alarm sounded again. The threshold was left at 20, the default. The team's note on the sheet says that no trash was added to the creek until 4:00 PM and that the system's detection stayed the same, so they ended the day.

**Figure 72**

*Hand Count and Dashboard Count at Each Calibration Reading*




*Note.* Data from Table 17. The percentage above each dashboard bar is the dashboard count divided by the hand count at that reading.

Figure 72 plots the readings of Table 17. The dashboard count stayed below the hand count at every reading. With the camera at about 13ft it reached 21 to 31% of the hand count, and after the camera was lowered it reached 40 to 54%, so the gap narrowed but about half of the items were still missed.

***Evaluation Results***

This section reports the acceptance test plan set during requirement gathering (Tables 6 to 8 in Chapter III): the developer-side test cases and indicators first, then the end-user usability survey.

### Developer-side evaluation

**Table 18**

*Developer-Side Internal Acceptance Results*

| Indicator Code | Evaluation Statement (Table 7) | Result | Table 3 Rating |
|---|---|---|---|
| DEV-FS1 | Accurate detection of the three classes, ignoring sunken debris and organic matter | 74.7% (295 of 395 labels correct over three days, Equation 3, TC_ACC_01); 85.0% on November 9 | Fair |
| DEV-FS2 | Accumulation Alert raised when the threshold is reached | The alert was raised in 10 of 10 trials, a success rate of 100% (Equation 4, TC_SYS_02) | Excellent |
| DEV-PE1 | Latency below 3.0 seconds per frame | Average 275 to 350ms (0.28 to 0.35s) per frame | Excellent |
| DEV-PE2 | Continuous operation without crashing or overheating | No crash, freeze or overheating in 24 hours over six days | Excellent |

Table 19 gives the results of the acceptance test cases in Table 6, run on the final build during the field deployment.

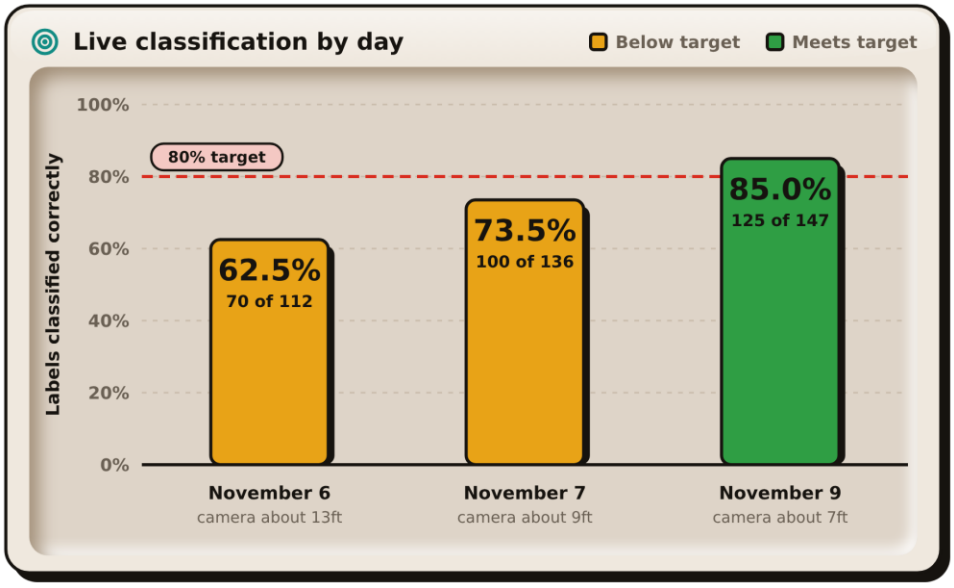
**Table 19**

*Acceptance Testing Results*

| Test Case ID | Objective | Actual Result | Successful Trials / 10 |
|---|---|---|---|
| TC_ACC_01 | Functional Suitability (classification) | 295 of 395 labels correct over the three days (74.7%, Equation 3), in the Fair band and below the 80% target. By day, 70 of 112 on November 6 (62.5%, camera at about 13ft), 100 of 136 on November 7 (73.5%, about 9ft) and 125 of 147 on November 9 (85.0%, about 7ft). Only the last day, in the Good band, reached the target. | Three counts of 112, 136 and 147 labels (not counted in trials) |
| TC_ACC_02 | Functional Suitability (accumulation detection) | The field sheet notes that the alarm buzzed each time the dashboard count reached the threshold on November 7 and 9 (Table 17), and the alert was raised in 10 of 10 trials of TC_SYS_02 (Table 15). | 10/10 Trials successful |
| TC_ACC_03 | Performance Efficiency (latency and stability) | Average latency of 275 to 350ms (0.28 to 0.35s) over the three deployment days with no crash, in the Excellent band and well inside the Table 6 target of 3.0s or faster. | 10 /10 Trials successful |

**Figure 73**

*Live Classification Accuracy by Deployment Day*




*Note.* Data from TC_ACC_01 in Table 19. The figures in each bar give the labels classified correctly out of the labels checked that day.

Figure 73 shows the daily results of TC_ACC_01 against the 80% target. Classification rose each time the camera was lowered, and only November 9, with the camera at about 7ft, reached the target.

**End-user evaluation**

Four end users answered the usability survey on November 6, 2026, after using the touchscreen and the application at the creek: two Estero Rangers (DENR Garbage Collectors assigned at the Barangay), a barangay kagawad and a barangay secretary. The weighted mean of each statement was computed with Equation 6 in Chapter III and read against Table 9.

**Table 20**

*End-User Usability Survey Results*

| Indicator Code | Evaluation Statement (Table 8) | Weighted Mean | Interpretation |
|---|---|---|---|
| USR-U1 | Information on the monitor is easy to read and understand | 4.50 | Excellent |
| USR-U2 | Bounding boxes clearly show where the garbage is | 4.25 | Excellent |
| USR-U3 | The Accumulation Alert is clear and noticeable | 4.25 | Excellent |
| USR-U4 | The system gives helpful insights for monitoring | 4.75 | Excellent |
| USR-U5 | The application matches the touchscreen | 4.25 | Excellent |
| | Overall | 4.40 | Excellent |

All five statements fall in the Excellent band, with an overall weighted mean of 4.40. The system gave helpful insights for monitoring (USR-U4) was rated highest at 4.75. Statements USR-U2, USR-U3 and USR-U5 were tied for lowest at 4.25, still in the Excellent band. Only one rating, a 3 for the clarity of the Accumulation Alert, was below 4. In their written comments, three respondents called the system a good and helpful tool for the barangay and the Estero Rangers in monitoring garbage in waterways. One respondent suggested improving the detection of plastic bags, bottles and containers so that cleaning can be faster.

### *System Capabilities and Limitations*

#### Capabilities

- Detects bottles, bags and food containers with a precision of 84.6% (v10, the reported model) on the frozen test set of 807 images, meeting the 80% target (Section 4.2).
- Counts only the items inside the monitored area. Redrawing the area removed false detections of the floating barrier at the creek (Section 4.2).
- Raises one Accumulation Alert per build-up, with smoothing over 15 frames and a re-arm level that prevents repeated alarms (TC_UNIT_02).

- Processes frames in about 0.28 to 0.35 seconds on the Raspberry Pi 5 (Excellent band), with the picture about one second behind the scene (Section 4.2).
- Stores every event with a snapshot, shows it on the touchscreen and the application, and reaches the application away from the local network through Firebase.
- Runs on its UPS battery, shows the remaining charge and shuts down safely before the battery runs out.
- The prototype has the capability to change the threshold, detection size, camera stream and model from the dashboard, without editing code.

**Limitations**

- Bags in the older creek photographs are rarely detected: v10 has a recall of 0.152 on 46 bag boxes. On the third-session test pictures its bag recall is 0.427. Piles with many bags will still be undercounted.
- Recall is lower than precision in every class, so the count tends to be too low rather than too high. More than half of the bottles in the creek photographs are missed (a recall of 0.450).
- Precision on creek images is lower than on the full test set: 68.9% against 84.6% for v10, which is in the Poor band of Table 3.
- Most false detections are labelled as bottles. Fixed objects in the water, such as the floating barrier, must be kept outside the monitored area.
- The enclosure is no longer IP65 after the cut-outs for the screen, fans and cables, so the unit has to be placed under cover.
- The two case fans run from the USB ports of the power extension, so they stop when mains power fails and the unit runs on its battery.

- The built-in Wi-Fi of the Raspberry Pi 5 did not work during testing, so internet access depends on a USB Wi-Fi adapter mounted outside the enclosure.
- The camera clock resets when the camera loses power, so the date stamped in the video can be wrong. The records use the Raspberry Pi's clock and are not affected.
- The third-session test pictures come from the same spot and the same day as the 27 third-session pictures v10 was trained on, so they show how v10 does at this creek, not at a new site. The test set holds only 19 labelled pictures.
- The results come from one site and one training run for each model, so differences of a few points, such as the 0.016 mAP@0.5 between v10 and v8 on the frozen test set, may be ordinary run-to-run variation.
- The bag result on the creek-only photographs rests on 46 boxes.
- The scores were measured on still photographs, not on live behavior such as flicker and false alarms on water and shadows.
- With the camera about 13ft above the water, most items in the creek were submerged or half submerged, and the dashboard counted only 21 to 31% of the items counted by eye on November 6 (Table 17). After the camera was lowered to about 9ft on November 7 and about 7ft on November 9, it counted 40 to 54%, so about half of the items were still missed. Live classification rose each time the camera was lowered: 62.5% at about 13ft on November 6, 73.5% at about 9ft on November 7 and 85.0% at about 7ft on November 9 (TC_ACC_01). Only the last day reached the 80% target, and the researchers attribute the rise to the lower camera position.

## V. CONCLUSIONS

The study developed Garbage-Guard, a fixed monitoring unit that detects floating garbage in creeks with a convolutional neural network, YOLO26n, counts how much of it has built up in a monitored stretch of water, and alerts the people responsible once the count reaches a set level. The unit runs on a Raspberry Pi 5 with a Hikvision HiWatch HWI-B140H camera, a touchscreen dashboard and an Android application, and it was operated at a creek in Barangay 870, Pandacan, Manila. From the results in Chapter IV, the researchers reached the following conclusions:

1. The first objective was met on the frozen test set. Of the ten training runs, v7 to v10 were scored on the same test set of 807 images, fixed at 20% of the data. On it, YOLO26n, exported to NCNN, detects bottles, bags and food containers with a precision of 84.6% (v10, the reported model), above the 80% target, and the earlier v7 (85.2%) and v8 (82.9%) met it as well. Recall was lower than precision in every class; food containers were the strongest class and bags the weakest, with a creek recall of 0.152 on 46 boxes. The creek-only precision of 68.9% for v10 shows that the model is weaker under field conditions than its headline figure suggests, the same kind of gap Nunkhaw et al. (2025) reported for floating waste in urban canals. The training history also shows what moved the result: images taken at the deployment site improved the model more than any amount of public data, and adding pictures from the third data session doubled the score on new pictures from the same spot (a mAP@0.5 of 0.404 against 0.204 for v8). This agrees with the point raised by Jia et al. (2023) that floating-litter detection suffers from a lack of data that matches real waterways.
2. The second objective was met. A Raspberry Pi 5 can run detection, ByteTrack tracking, area-limited counting, smoothing, alerting, event storage, the dashboard and the cloud link on one board, from a battery, without a separate computer. The accumulation counter passed all 19 automated tests, all ten system test cases met

their expected result in 10 of 10 trials (Table 15), and the unit ran for 24 hours over six days at the creek without a crash, a freeze or overheating (Table 16). The monitored area does more than save processing time. It is also the operator's tool for keeping fixed objects such as the floating barrier out of the count, and redrawing it removed the false detections of the barrier on October 31, so drawing it carefully is part of installing the unit.

3. Because recall is lower than precision, the system is more likely to count too few items than too many. In practice this means an alert will tend to come late rather than early, and false alarms are rare. The researchers concluded that the threshold should be set from an on-site hand count rather than left at the default, which is what the field calibration showed: the dashboard counted 21 to 31% of the items counted by eye on November 6 and 40 to 54% after the camera was lowered, and the threshold ended at 20.
4. The third objective was met for Performance Efficiency. At about 0.28 to 0.35 seconds per frame the Raspberry Pi 5 is well inside the Excellent band, and floating waste moves slowly enough that five frames per second, smoothed over 15 frames, is sufficient. Li et al. (2024) reached 33 frames per second with a very small YOLO model on an edge device, but for a counting task on slow water the extra speed would not change the alert. Functional Suitability in the field fell below the target overall: live classification was 74.7% over the three days (Fair, against the 80% target), although it rose each time the camera was lowered and reached 85.0% on November 9 with the camera at about 7ft, down from about 13ft on November 6. Usability was rated Excellent (4.40) by the end users.
5. The research gap set out in Section 2.4 and Table 2 was addressed. None of the systems reviewed combined a detector trained by the operator, running at the edge at creek level, with counting limited to a drawn area of water, one identity per item across frames, and a threshold alert backed by a saved snapshot. Garbage-

Guard puts these together in one unit and was shown to work at a real creek under a barangay permit.

Several lessons came out of the work that may help later researchers. A test set has to be fixed once and never rebuilt, or the scores of later runs cannot be compared; this was the adviser's requirement from v7 onward, and the v6 and v6b problems showed why. Detecting small items far from the camera depended more on how the picture reached the model than on the model itself: cropping the full-resolution main stream to the monitored area kept bottles and bags large enough to detect, while a higher camera resolution only made the Raspberry Pi fall behind. A labelled snapshot with a clean copy beside it turned out to be useful twice, first as evidence for each alert and then as new training data.

The prototype has limits that the results make plain. It works in daylight only and covers one stretch of creek from one camera about 5ft to 7ft from the items. Bags at the creek are rarely detected, so piles made mostly of plastic bags will be undercounted. The count says how many items are present, not how much space they cover, so a few large items and many small ones can give the same reading. The enclosure lost its weather rating once it was cut for the screen, fans and cables, so the unit must stand under cover with only the camera outside. The two USB case fans draw power from the USB ports of the power extension, so they stop when mains power fails and the unit runs on its battery. The built-in Wi-Fi of the Raspberry Pi 5 could not join a network during testing, so internet access depends on a USB Wi-Fi adapter mounted outside the box, and the camera's clock resets whenever it loses power.

The end users who took part in the survey described the system as a good and helpful tool for the barangay and the Estero Rangers in monitoring garbage in waterways, and one asked for better detection of plastic bags, bottles and containers so that cleaning is faster.

For future research, the researchers recommend the following. The bag class needs more creek images of plastic bags in different states of floating and sinking, and

the creek portion of the test set should grow so that each class is judged on more than a few dozen boxes. As the panel recommended, later models should also detect more classifications, for example, organic waste such as dead animals and plant debris. The clean copies saved with every alert can be labelled and used to retrain the model as a routine sprint of the development model in Figure 47, with each new model tested on the frozen set before it replaces the one on the unit. Later work may extend operation to night with an infrared or low-light camera, serve more than one camera from a unit or use wireless cameras in place of the 40m cable, reach farther with a higher-resolution or longer-lens camera, and estimate the area of water covered by waste through instance segmentation, which Kataoka et al. (2024) compared with bounding boxes. A unit with its own connection, such as a USB LTE modem or a LoRa radio (Te et al., 2024), would not depend on site Wi-Fi, and pairing the count with a water-level sensor, as in the flood monitoring systems of Te et al. (2024) and Arante et al. (2025), could give an earlier flood warning. A deployment of several weeks across the dry and rainy seasons and at more than one creek, with charts of counts per hour, day and week built from the stored records, would show how the system holds up over time.

Barangays and LGUs that adopt the system should place units upstream of culverts, bridges and drainage outfalls, where floating waste is most likely to collect. The camera should be mounted as low and as close to the water as the site allows, since lowering it from about 13ft to about 7ft raised live classification from 62.5% to 85.0% (TC_ACC_01, Figure 73). The threshold at each site should be set from a hand count during installation and reviewed after the first few alerts.

Garbage-Guard is therefore a workable early-warning tool for barangays and LGUs that must decide when a stretch of creek needs clearing. It replaces a periodic walk along the bank with a continuous count and a dated record of every build-up. In doing so it supports UN Sustainable Development Goals 6, 9 and 11 (United Nations, 2025).

## VI. ACKNOWLEDGMENT

The researchers wish to sincerely thank all those who assisted in the completion of this study. The guidance, patience and support of the following individuals shaped this work from the first proposal to the final prototype.

To Dr. Jonel R. Macalisang, the researchers' thesis adviser, for helping shape the content and providing the technical direction for the study. The comments helped the researchers examine their work more closely and provide supporting evidence for every claim. Most importantly, this guidance helped the researchers keep the project focused on its end goal.

To Prof. Ma. Ian P. De Los Trinos, the professor of the Project Development subject and a member of the panel, for helping the researchers with the formatting and organization of the paper. The careful review of each chapter made sure that the document met the format required by TUP-Manila and CIT.

To Prof. Marc Ardie V. Ardiente, the researchers' secondary technical adviser and a member of the panel, for the time and expertise given to the technical aspects of the project, the practical suggestions that solved numerous problems, and the comments at the defense that guided the revisions of this paper.

To the panel, chaired by Prof. Tristan R. Mopas with Prof. Minabelle D. Villafuerte as member, for their comments and suggested revisions at the defense.

To the researchers' families, for their patience, encouragement, and support during the numerous long and demanding days and weeks of this process. They deserve credit for this work and for bearing the pressures of having a family member so preoccupied with research and building something that was at times unreliable and unbearably slow.

To their friends, for the help and companionship they willingly offered and for a good laugh, and for reminding the researchers to not give up and to keep working on this project until it was completed.

## VII. DECLARATION OF CONFLICT OF INTEREST

The researchers declare that they have no financial, personal or professional relationship with any funding agency, company, testing center, sponsor or respondent that could have influenced the design, conduct or reporting of this study. The study received no external funding. The costs of the hardware, software and field deployment of Garbage-Guard were shouldered by the researchers, and no supplier of the components or services named in this paper took part in the testing or evaluation of the system.

The field deployment at the creek site was carried out with the permission of the concerned authorities, and the evaluation survey was answered voluntarily by respondents who were told the purpose of the study. The personal information of the respondents was kept confidential and used only for this study, in compliance with the Data Privacy Act of 2012 (Republic Act No. 10173). The camera recorded the creek surface for garbage detection only, and no image was used to identify any person.

Generative AI tools were used only to assist with language editing and as a coding assistant in parts of the software development. Every output was reviewed, tested and verified by the researchers, who take full responsibility for the content of this paper and for the system it describes.

The researchers affirm that the statements in this declaration are true and correct.

**BRIAN JAY CELESTE**
*Researcher*

**GENESIS WALTER D. REYES II**
*Researcher*

**JADE C. GUNAYAN**
*Researcher*

**KING PERTH G. PEPITO**
*Researcher*

**RODEL S. BANOGBANOG**
*Researcher*

## VIII. REFERENCES

Acot, F. T., Sajorne, R. E., Omar, N.-A. K., Suson, P. D., Rallos, L. E. E., & Bacosa, H. P. (2022). Unraveling macroplastic pollution in rural and urban beaches in Sarangani Bay Protected Seascape, Mindanao, Philippines. *Journal of Marine Science and Engineering, 10*(10), 1532. https://doi.org/10.3390/jmse10101532

Al Noman, Z., Li, F. H., Almukhtar, F., Rahaman, B., Omarov, S. Ray, Shahajan Miah, C., Wang, C. (2022). A computer vision-based lane detection technique using gradient threshold and hue-lightness-saturation value for an autonomous vehicle. International Journal of Electrical and Computer Engineering (IJECE). https://ijece.iaescore.com/index.php/IJECE/article/view/27007

Alzubaidi, L., Zhang, J., Humaidi, A. J., Al-Dujaili, A., Duan, Y., Al-Shamma, O., Santamaría, J., Fadhel, M. A., Al-Amidie, M., & Farhan, L. (2021). Review of deep learning: Concepts, CNN architectures, challenges, applications, future directions. Journal of Big Data, 8, Article 53. https://doi.org/10.1186/s40537-021-00444-8

Arante, H. R. C., Sybingco, E., Roque, M. A., Ambata, L., Chua, A. Y., & Gutierrez, A. N. (2025). *Development of a secured IoT-based flood monitoring and forecasting system using genetic-algorithm-based neuro-fuzzy network*. *Sensors, 25*(13), 3885. https://doi.org/10.3390/s25133885

Balachandran, N., & Talavera, O. (2023). *Apparatus and Method for Waste Monitoring and Analysis* (US11788877B2). Google Patents. https://patents.google.com/patent/US11788877B2

Campbell, D. P., Gormley, M., Kelly, D., Xue, Y., & Stewart, C. (2022). *Drainage monitoring apparatus and system* (WO2022254204A1). Google Patents. https://patents.google.com/patent/WO2022254204A1/en

Chen, L., Li, G., Xie, W., Tan, J., Li, Y., Pu, J., Chen, L., Gan, D., & Shi, W. (2024). A survey of computer vision detection, visual SLAM algorithms, and their applications in energy-efficient autonomous systems. Energies, 17(20), 5177. https://doi.org/10.3390/en17205177

Chordiya, S. B., Das, S., Garg, R., Jain, V., Kumar, A., Nagpure, S. G., Nagvekar, Y. D., Parlikar, A. S., Regulwar, G. B., & Saini, D. K. (2020, September 9). *AU2020102191A4 - WCM-Smart City: Waste Collection and Management under Smart City Using IoT - Google Patents*. https://patents.google.com/patent/AU2020102191A4/en?q=(WCM-Smart+City:+WASTE+COLLECTION+AND+MANAGEMENT+UNDER+SMART+CITY+USING+IOT)&oq=WCM-Smart+City:+WASTE+COLLECTION+AND+MANAGEMENT+UNDER+SMART+CITY+USING+IOT

Circuitrocks. (n.d.-a). *Raspberry Pi 5 quad-core Cortex-A76 4GB/8GB/16GB* [Product page]. Retrieved October 23, 2026, from https://circuit.rocks/products/raspberry-pi-5-2-4ghz-quad-core-arm-cortex-a76-4gb-8gb-16gb

Circuitrocks. (n.d.-b). *Passive buzzer module for Arduino/AVR/PIC* [Product page]. Retrieved October 23, 2026, from https://circuit.rocks/products/buzzer-5v-breadboard-friendly

Corpuz, J. C. G. (2025). Wastewater management in the Philippines: A call for localized solutions and policy integration. *Environmental Health Insights, 19*. https://doi.org/10.1177/11786302241311529

Das, T., Das, N., Zuthi, M. F. R., Pal, S. K., Kraft, E., Thomas, H., & Kuehlewindt, S. (2025). *Plastic waste in marine ecosystems: Identification techniques and policy interventions*. Water, Air, & Soil Pollution, 236, 478. https://www.researchgate.net/publication/391870234_Plastic_Waste_in_Marine_Ecosystems_Identification_Techniques_and_Policy_Interventions

Den Hartog, E. D., & Voncken, R. G. H. (2017). *System for monitoring water drainage* (WO2017089181A1). Google Patents. https://patents.google.com/patent/WO2017089181A1/en

Development Asia. (2024, February 28). *Optimizing Municipal Waste Collection in the Philippines through a Smart App*. Retrieved March 8, 2026, from https://development.asia/case-study/optimizing-municipal-waste-collection-philippines-through-smart-app

Ella, D. M., Lim, D. M., Lu, B., & Cabauatan, R. (2022). *The impact of solid waste management to the economic growth in selected OECD countries and Philippines*. Journal of Economics, Finance and Accounting Studies, 4(1). https://doi.org/10.32996/JEFAS.2022.4.1.20

Garcia, A., Saez, Y., Harris, I., Huang, X., & Collado, E. (2025). *Advancements in air quality monitoring: a systematic review of IoT-based air quality monitoring and AI technologies*. Artificial Intelligence Review, 58(9), 275. https://doi.org/10.1007/s10462-025-11277-9

Gryech, I., Assad, C., Ghogho, M., & Kobbane, A. (2024). *Applications of machine learning & Internet of Things for outdoor air pollution monitoring and prediction: A systematic literature review*. *Engineering Applications of Artificial Intelligence, 137*, 109182. https://doi.org/10.1016/j.engappai.2024.109182

JB Hi-Fi. (n.d.). SanDisk High Endurance microSDXC 64GB memory card [Product page]. Retrieved October 19, 2026, from https://jbhifi.com.au/products/sandisk-high-endurance-microsdxc-64gb-memory-card

Jia, T., Vallendar, A. J., de Vries, R., Kapelan, Z., & Taormina, R. (2023). Advancing deep learning-based detection of floating litter using a novel open dataset. Frontiers in Water, 5, 1298465. https://doi.org/10.3389/frwa.2023.1298465

Kang, J., Tariq, S., Oh, H., & Woo, S. S. (2022). A survey of deep learning-based object detection methods and datasets for overhead imagery. *IEEE Access, 10*, 20118–20134. https://doi.org/10.1109/ACCESS.2022.3149052

Kataoka, T., Yoshida, T., & Yamamoto, N. (2024). Instance segmentation models for detecting floating macroplastic debris from river surface images. Frontiers in Earth Science, 12, 1427132. https://doi.org/10.3389/feart.2024.1427132

Kim, H., Lee, S., Yu, G., & Kang, J. (2019, January 4). *KR101985436B1 - Cleaning unit using the deep-learning - Google Patents*. https://patents.google.com/patent/KR101985436B1/en?q=(floating+trash+detection+deep+learning)&oq=floating+trash+detection+using+deep+learning

KingSpec. (n.d.). M.2 NVMe SSD in series [Product page]. Retrieved October 19, 2026, from https://www.kingspec.com/product/m2-nvme-ssd-in-series.html

Li, B., & Wang, F. (2020, June 23). *CN111767822B - Garbage detection method, related equipment and device - Google Patents*. https://patents.google.com/patent/CN111767822B/en?oq=CN111767822B

Li, H., Yang, S., Zhang, R., Yu, P., Fu, Z., Wang, X., Kadoch, M., & Yang, Y. (2024). Detection of floating objects on water surface using YOLOv5s in an edge computing environment. Water, 16(1), 86. https://doi.org/10.3390/w16010086

Liu, N., Liu, P. J., Su, M. R., Zhao, B. H., Liu, Y., Wang, W., & Yuan, Z. F. (2022). *Detection and monitoring of waste disposal in an environmental location using artificial intelligence* (US11308603B2). Google Patents. https://patents.google.com/patent/US11308603B2

MacAfee, E. A., & Löhr, A. J. (2023). *Multi-scalar interactions between mismanaged plastic waste and urban flooding in an era of climate change and rapid urbanization*. WIREs Water. https://doi.org/10.1002/wat2.1708

Magyar, D., Cserép, M., Vincellér, Z., & Molnár, A. D. (2023, March 25). Waste Detection and Change Analysis based on Multispectral Satellite Imagery. *arXiv*. https://doi.org/10.48550/arXiv.2303.14521

MakerLab. (n.d.-a). *Official Raspberry Pi 5 Active Cooler* [Product page]. Retrieved October 23, 2026, from https://www.makerlab.ph/products/official-raspberry-pi-5-active-cooler

MakerLab. (n.d.-b). *PKCell lithium-ion 18650/21700 battery 3.7V* [Product page]. Retrieved October 23, 2026, from https://makerlab.ph/products/pkcell-lithium-

ion-18650-21700-battery-3-7v-true-rated-18650-2200mah-3000mah-3350mah-4500mah-rechargeable-battery-for-power-bank-flashlight-mini-fan

MakerLab. (n.d.-c). *USB 5V to 12V DC-DC converter cable jack step-up power module* [Product page]. Retrieved October 23, 2026, from https://makerlab.ph/products/dc-dc-converter-cable-usb-5v-to-12v-dc-jack-step-up-power-module

MakerLab. (n.d.-d). *7 inch HDMI LCD for Raspberry Pi* [Product page]. Retrieved October 23, 2026, from https://makerlab.ph/products/7-inch-hdmi-lcd-for-raspberry-pi

MakerLab. (n.d.-e). *1.5m HDMI to micro HDMI cable* [Product page]. Retrieved October 23, 2026, from https://makerlab.ph/products/hdmi-to-micro-hdmi-cable

MakerLab. (n.d.-f). *Pulihua CAT 5E RJ45 UTP internet cable* [Product page]. Retrieved October 23, 2026, from https://makerlab.ph/products/ethernet-cable-computer-network-cord

Meijer, L. J. J., van Emmerik, T., van der Ent, R., Schmidt, C., & Lebreton, L. (2021). More than 1000 rivers account for 80% of global riverine plastic emissions into the ocean. *Science Advances, 7*(18), eaaz5803. https://doi.org/10.1126/sciadv.aaz5803

Multi-Line. (n.d.). HWI-B140H [Product page]. Retrieved October 19, 2026, from https://www.multi-line.com.ph/product/hwi-b140h/

Murakami. (n.d.). *Hikvision (HWI-B140H) 4 MP fixed bullet network camera* [Product page]. Retrieved October 23, 2026, from https://murakami.com.ph/product/hikvision-hwi-b140h/

Murat, A. A., & Kiran, M. S. (2025). A comprehensive review on YOLO versions for object detection. *Engineering Science and Technology, an International Journal, 70*, 102161. https://doi.org/10.1016/j.jestch.2025.102161

Narayana, T. L., Venkatesh, C., Kiran, A., Babu, J. C., Kumar, A., Khan, S. B., Almusharraf, A., & Quasim, M. T. (2024). *Advances in real time smart monitoring of environmental parameters using IoT and sensors*. Heliyon, 10(7), e28195. https://doi.org/10.1016/j.heliyon.2024.e28195

Nežerka, V., Zbíral, T., & Trejbal, J. (2023). Machine-learning-assisted classification of construction and demolition waste fragments using computer vision: Convolution versus extraction of selected features. *Expert Systems with Applications*. https://www.sciencedirect.com/science/article/abs/pii/S0957417423020705

Ng, C. H., Mistoh, M. A., Teo, S. H., Galassi, A., Ibrahim, A., Sipaut, C. S., Foo, J., Seay, J., Taufiq-Yap, Y. H., & Janaun, J. (2023). *Plastic waste and microplastic issues*

*in Southeast Asia*. Frontiers in Environmental Science, 11, 1142071. https://doi.org/10.3389/fenvs.2023.1142071

Nunkhaw, M., & Miyamoto, H. (2024). An image analysis of river-floating waste materials by using deep learning techniques. Water, 16(10), 1373. https://doi.org/10.3390/w16101373

Nunkhaw, M., Chitwatkulsiri, D., & Miyamoto, H. (2025). Enhancing river waste detection with deep learning and preprocessing: A case study in the urban canals of the Chao Phraya River. Water, 17(22), 3193. https://doi.org/10.3390/w17223193

PC Express. (n.d.). *SanDisk 64GB High Endurance microSD (SDSQQNR-064G-GN6IA) memory card* [Product page]. Retrieved October 23, 2026, from https://pcx.com.ph/products/sandisk-64gb-high-endurance-microsd-sdsqqnr-064g-gn6ia-memory-card

Pound Sterling Live. (n.d.). *U.S. dollar to Philippine peso history: 2026*. Retrieved October 23, 2026, from https://www.poundsterlinglive.com/history/USD-PHP-2026

Robles, K. P. V., & Monjardin, C. E. F. (2025, June 26). Assessment and Monitoring of Groundwater Contaminants in Heavily Urbanized Areas: A Review of Methods and Applications for Philippines. *Water*, *17*(13), 1903. https://doi.org/10.3390/w17131903

Roboflow. (n.d.). *Roboflow Annotate* [Computer software]. Retrieved October 31, 2026, from https://roboflow.com/annotate

Samolovac, J. (2026, July 10). *What is hybrid project management?* Plaky. https://plaky.com/learn/project-management/hybrid-project-management/

Shao, Z., Li, Y., Liu, B., Quan, Y., & Liang, Y. (2023). *Method and system for flood early warning in smart cities based on the Internet of Things* (US11721191B1). Google Patents. https://patents.google.com/patent/US11721191B1

Shopee. (n.d.). Shopee Philippines [Online marketplace]. Retrieved October 19, 2026, from https://shopee.ph

Te, M. C. L., Bautista, J. A. T., Dimacali, S. M. E. V., Lood, A. V. M., Pangan, M. G. M., & Chua, A. Y. (2024). *A smart IoT urban flood monitoring system using a high-performance pressure sensor with LoRaWAN*. *HighTech and Innovation Journal, 5*(4), 918–936. https://doi.org/10.28991/HIJ-2024-05-04-04

The Pi Hut. (n.d.-a). Active Cooler for Raspberry Pi 5 [Product page]. Retrieved October 19, 2026, from https://thepihut.com/products/active-cooler-for-raspberry-pi-5

The Pi Hut. (n.d.-b). Raspberry Pi 5 [Product page]. Retrieved October 19, 2026, from https://thepihut.com/products/raspberry-pi-5

Torres, J. M., Adrian, E. S., & Cook, R. C. (2024). *Flood warning system* (US12094320B2). Google Patents. https://patents.google.com/patent/US12094320B2

UNEP. (2023, May 16). *Turning off the Tap: How the world can end plastic pollution and create a circular economy*. UNEP. Retrieved March 10, 2026, from https://www.unep.org/resources/turning-off-tap-end-plastic-pollution-create-circular-economy

United Nations. (2025). *The Sustainable Development Goals report 2025*. United Nations Statistics Division. https://unstats.un.org/sdgs/report/2025/

Victoriano, J. M., Delos Santos, M. L. C., Vinluan, A. A., & Carpio, J. T. (2022, February 12). Predicting Pollution Level Using Random Forest: A Case Study of Marilao River in Bulacan Province, Philippines. *International Journal of Computing Sciences Research*, *3*(1), 151-162. https://doi.org/10.48550/arXiv.2202.06066

Waveshare. (n.d.-a). PCIe TO 2-CH M.2 HAT+ [Product page]. Retrieved October 19, 2026, from https://www.waveshare.com/PCIe-TO-2-CH-M.2-HAT-Plus.htm

Waveshare. (n.d.-b). UPS HAT (E) [Product page]. Retrieved October 19, 2026, from https://www.waveshare.com/ups-hat-e.htm

Wikimedia Commons. (n.d.). Wikimedia Commons [Media repository]. Wikimedia Foundation. Retrieved October 20, 2026, from https://commons.wikimedia.org

World Bank. (2022, March 21). *Philippines: Plastics Circularity Opportunities Report*. World Bank Group. Retrieved March 10, 2026, from https://www.worldbank.org/en/country/philippines/publication/market-study-for-philippines-plastics-circularity-opportunities-and-barriers-report-landing-page

Wu, P., Wang, T., Wang, G., & Zhang, X. (2019, November 28). *CN110884791A - Vision Garbage Classification System and Classification Method Based on TensorFlow - Google Patents*. https://patents.google.com/patent/CN110884791A/en?q=(Ai+trash+detection)&oq=Ai+trash+detection

Yaseen, M. (2024, August 28). What is YOLOv8: An In-Depth Exploration of the Internal Features of the Next-Generation Object Detector. *arXiv*. arXiv:2408.15857v1. https://arxiv.org/html/2408.15857v1

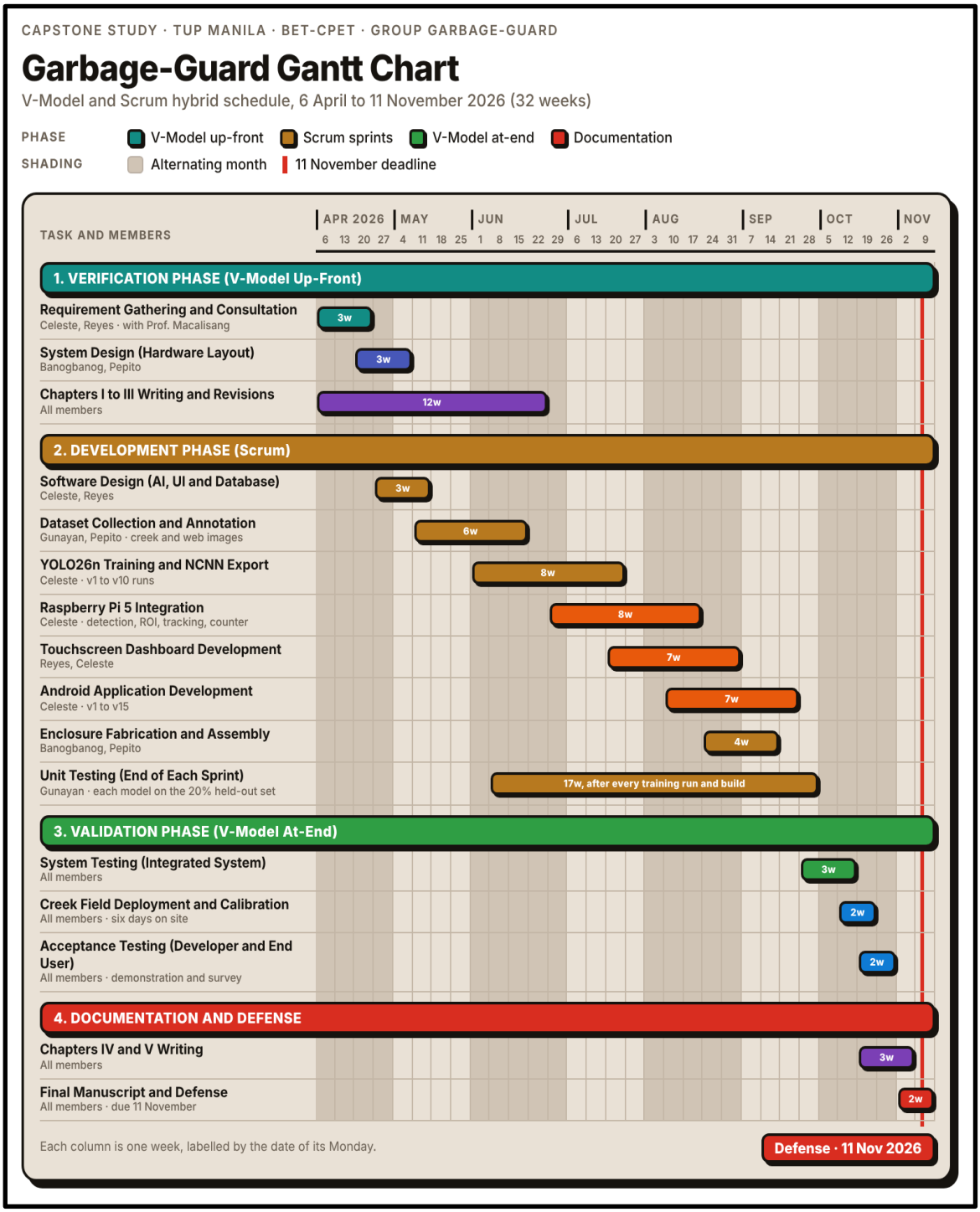
Zeng, F., Pang, C., & Tang, H. (2024). *Sensors on Internet of Things systems for the sustainable development of smart cities: A systematic literature review*. Sensors, 24(7), 2074. https://doi.org/10.3390/s24072074

Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., & Wang, X. (2022). ByteTrack: Multi-object tracking by associating every detection box. In Computer Vision – ECCV 2022 (Lecture Notes in Computer Science, Vol. 13682). Springer. https://doi.org/10.1007/978-3-031-20047-2_1

# IX. APPENDICES

## Appendix A

## Gantt Chart



# Appendix B

## Bill of Materials

| No. | Component | Qty | Unit Price | Amount |
|---|---|---|---|---|
| 1 | Raspberry Pi 5 single-board computer, 8GB | 1 | ₱12,000.00 | ₱12,000.00 |
| 2 | Raspberry Pi 5 Active Cooler (official) | 1 | ₱600.00 | ₱600.00 |
| 3 | Hikvision HiWatch HWI-B140H 4MP bullet network camera | 1 | ₱3,000.00 | ₱3,000.00 |
| 4 | Waveshare UPS HAT (E) for Raspberry Pi | 1 | ₱2,000.00 | ₱2,000.00 |
| 5 | 21700 lithium-ion cell, 3.7V 4500mAh | 4 | ₱200.00 | ₱800.00 |
| 6 | USB 5V to 12V step-up cable with DC barrel jack | 1 | ₱135.00 | ₱135.00 |
| 7 | Erlvsoiis WF-ER1800M dual-band USB Wi-Fi adapter | 1 | ₱489.00 | ₱489.00 |
| 8 | Vention USB 3.0 Type A male-to-female extension cable | 1 | ₱129.00 | ₱129.00 |
| 9 | 7-inch HDMI LCD (C), 1024x600 IPS, capacitive touch | 1 | ₱2,500.00 | ₱2,500.00 |
| 10 | Micro-HDMI to HDMI cable, 1.5m | 1 | ₱100.00 | ₱100.00 |
| 11 | SanDisk High Endurance 64GB microSD card | 1 | ₱2,000.00 | ₱2,000.00 |
| 12 | KingSpec 256GB M.2 2242 NVMe solid-state drive | 1 | ₱3,000.00 | ₱3,000.00 |
| 13 | Waveshare PCIe to 2-CH M.2 HAT+ | 1 | ₱2,000.00 | ₱2,000.00 |
| 14 | Passive buzzer module | 1 | ₱50.00 | ₱50.00 |
| 15 | 2-in-1 CAT5e UTP Ethernet and CCTV power cable, 40m | 1 | ₱500.00 | ₱500.00 |
| 16 | Metal waterproof junction box, 25cm x 30cm x 15cm | 1 | ₱1,500.00 | ₱1,500.00 |
| 17 | USB case fan | 2 | ₱200.00 | ₱400.00 |
| 18 | AC extension unit | 1 | ₱500.00 | ₱500.00 |
| 19 | Raspberry Pi 5 27W power adapter | 1 | ₱600.00 | ₱600.00 |
| | | | **Total** | ₱32,303.00 |

*Note.* Prices are in Philippine pesos as listed by the online stores on October 23, 2026.

# Appendix C

## Testing Permit

October 21, 2026
HON. MARIA THERESA B. MEMPIN
Barangay 870
Pandacan, Manila

**Subject: Request for Permission to Conduct a Field Test of the GarbageGuard Floating Trash Detection System**

Dear Hon. Mempin:

Good day! We are fourth-year students of the Technological University of the Philippines-Manila, developing our capstone project, GarbageGuard: a camera-based system that detects floating solid waste in creeks and sends an alert when trash accumulates. It aims to help barangays and local offices know when a creek needs clearing before it clogs and contributes to flooding.

We respectfully request permission to conduct a short field test of our prototype along the creek in your barangay on October 21, 2026 from 1:00–3:00 pm.

We will gladly share our findings with the barangay after the test. Attached are our school endorsement and project summary. You may reach us 976-620-8735 or genesis071805@gmail.com.

Thank you very much for your time and consideration.

Respectfully yours,

Reyes, Genesis Walter D. G. Reyes

Banogbanog, Rodel S.

Celeste, Brian Jay

Gunayan, Jade C.

Pepito, King Perth G.

Noted by:

PROF. MA IAN DELOS TRINOS
Professor

DR. JONEL R. MACALISANG
Thesis Adviser

DR. TRISTAN R. MOPAS
Department Head

PROF. GHAZALI R. SISON
College Dean

Approved by:

MARIA THERESA B. MEMPIN
Barangay Chairwoman 10/22/2024

# Appendix D

## Deployment Permit

November 6, 2026

**HON. MARIA THERESA B. MEMPIN**
Punong Barangay
Barangay 870
Pandacan, Manila

**Subject: Request for Permission to Conduct Pilot Deployment of Our System**

Dear Hon. Mempin:

Greetings!

We are fourth-year students of Technological University of the Philippines-Manila, currently developing a system entitled **"GarbageGuard Floating Trash Detection System"** as part of our academic project.

Having successfully completed the initial testing and evaluation of our system, we would like to respectfully request permission from your office to conduct a pilot deployment within Barangay 870.

The purpose of this deployment is to evaluate the system's functionality and performance in an actual environment, gather relevant observations, and identify possible improvements before its final implementation. We believe that conducting this deployment within your barangay would provide valuable insights into the system's effectiveness and potential benefits to the community.

In connection with this request, we would also like to seek your approval and assistance regarding the following deployment requirements:

**1. System Installation Location**
We respectfully request permission to install the main system inside the Barangay Hall. The system will serve as the central unit for monitoring and operating the camera.

**2. Camera Installation**
The camera will be positioned near the creek within the barangay to monitor the designated area. A connecting wire will run from the system inside the Barangay Hall to the camera's location. We would like to request permission to route and secure the wire outside the Barangay Hall and along the necessary path toward the creek.

**3. Installation and Setup Assistance**
We respectfully request the cooperation of the barangay officials in identifying a suitable location for the camera and determining an appropriate and safe route for the connecting wire.

**4. Proposed Deployment Schedule**

We would like to request permission to conduct the installation, setup, and pilot deployment on the following dates:

- **November 6, 2026 (Friday):** 10:00 AM – 4:00 PM
- **November 7, 2026 (Saturday):** 10:00 AM – 4:00 PM
- **November 9, 2026 (Monday):** 10:00 AM – 4:00 PM

**5. Permission to Conduct Pilot Deployment**

We seek authorization to install, operate, and evaluate the system within the designated locations for the agreed deployment period.

We assure your office that the installation and deployment will be conducted responsibly and in coordination with the concerned barangay officials. We are also willing to discuss the installation arrangements and comply with the necessary guidelines and requirements of the barangay.

We sincerely hope for your favorable consideration of this request. Your support and cooperation would greatly contribute to the successful completion of our academic project and the further development of our system.

Thank you for your time and consideration.

Respectfully yours,

Reyes, Genesis Walter D.

Banogbanog, Rodel S.

Celeste, Brian Jay

Gunayan, Jade C.

Pepito, King Perth G.

Noted by:

PROF. MA IAN DELOS TRINOS
Professor

DR. JONEL R. MACALISANG
Thesis Adviser

DR. TRISTAN R. MOPAS
Department Head

PROF. GHAZALI R. SISON
College Dean

Approved by:

MARIA THERESA B. MEMPIN
Barangay Chairwoman 11/6/2020

## Appendix E

## Documentation of the Study

This appendix records the development and testing of Garbage-Guard in photographs, from the approval of the topic, through the days spent assembling the prototype, tests at creeks, the measurement of the camera cable, and the three days of deployment, with the usability survey on the first. Each figure is followed by an account of what was done at that stage and what the team learned from it.

**Figure E1**

*Topic Approval at the Proposal Defense*




Figure E1 shows the researchers and their adviser, from left to right: Genesis Walter D. Reyes II, Brian Jay Celeste, Rodel S. Banogbanog, King Perth G. Pepito, Dr. Jonel R. Macalisang, and Jade C. Gunayan at the proposal defense where the topic “Garbage-Guard: Development of Computer Vision for Creeks Garbage Detection Using Convolutional Neural Network” was approved. The title slide behind them names the study and its proponents. The approval set the direction for everything that followed: a camera-based system that detects floating garbage in a creek with a convolutional neural network. Taken on May 4, 2026.

**Figure E2**

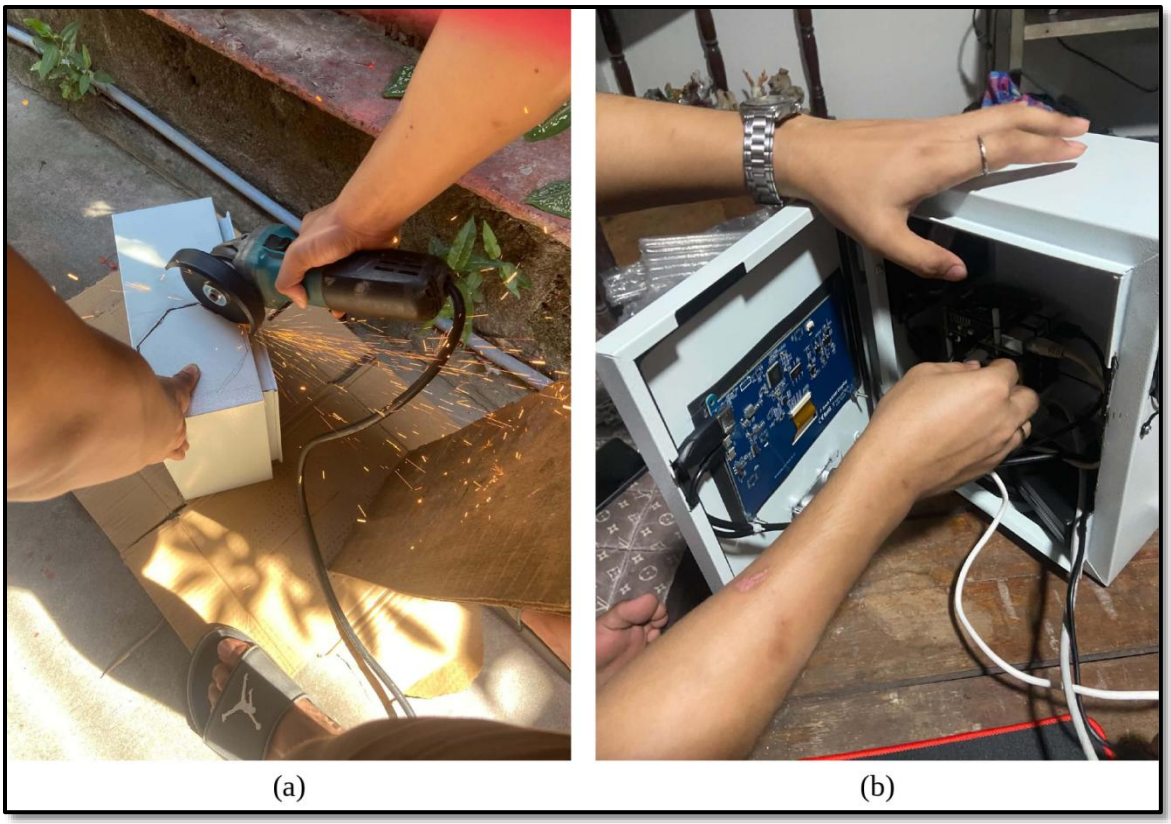
*First Day of Prototype Assembly*




Figure E2 covers the first day of building the prototype. A rectangular metal box was chosen as the casing, and in (a) a member cuts an opening in its upper front with an angle grinder so that the touchscreen could be mounted there. Panel (b) shows the box with the opening finished. Most of the components had not arrived yet, so preparing the casing was the only work possible that day. Taken on October 3, 2026.

**Figure E3**

*Second Day of Prototype Assembly*




On the second day, shown in Figure E3, holes were cut in both upper sides of the box for the cooling fans, as in (a). Most of the remaining parts arrived the same day, which made it the most productive of the three: the team planned the layout, wired the components and arranged them inside the casing. Panel (b) shows the touchscreen already fitted behind the front opening while the wiring continues inside. Taken on October 9, 2026.

**Figure E4**

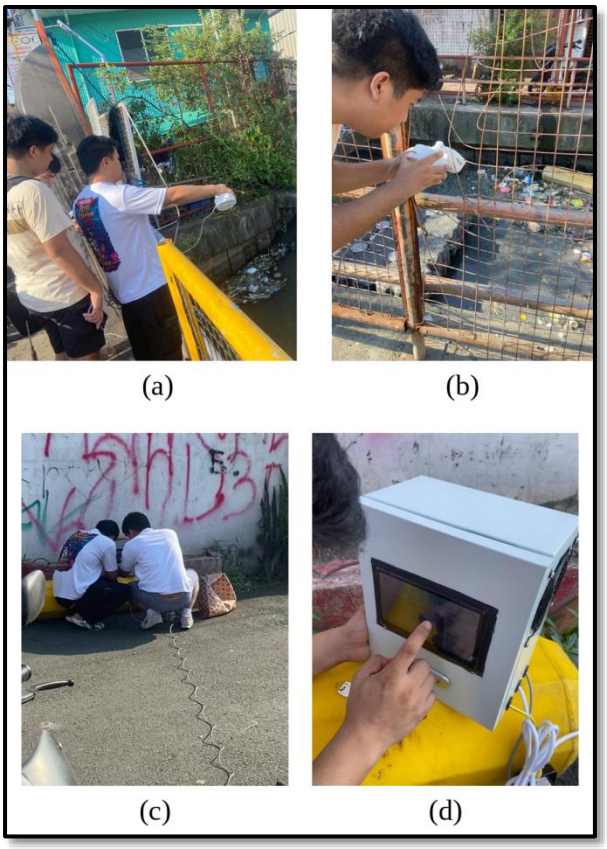
*Third Day of Prototype Assembly*




Figure E4 shows the third day, which picked up where the second had stopped. Some of the wiring was corrected and the last components were added, as in (a), until every part sat in its place inside the casing. By the end of the day the system was complete enough for its first test, and (b) shows the assembled unit with its screen running. Taken on October 10, 2026.

**Figure E5**

*First Independent Test for Data Gathering*




The first test, in Figure E5, was an independent run at another creek, held to see how the system performed and to gather pictures for annotation; (a) to (d) show the team at the creek and the unit being set up and operated there. The system had trouble detecting garbage once the items were some distance from the camera, and the confidence of its detections was low. The team answered this with new training data from the creek itself: creek waste was recorded with the system's own CCTV camera, annotated by class and used to retrain the model. Taken on October 14, 2026.

**Figure E6**

*Researchers with the Barangay Officials*




Figure E6 shows the researchers with barangay officials on the day of the permitted test at a creek in Barangay 870, Pandacan, Manila. The barangay granted the testing permit in Appendix C, and its personnel are the people the system is built for, since Garbage-Guard is meant to tell them when a stretch of creek needs clearing. Taken on October 21, 2026.

**Figure E7**

*Field Test at Barangay 870, Pandacan, Manila*

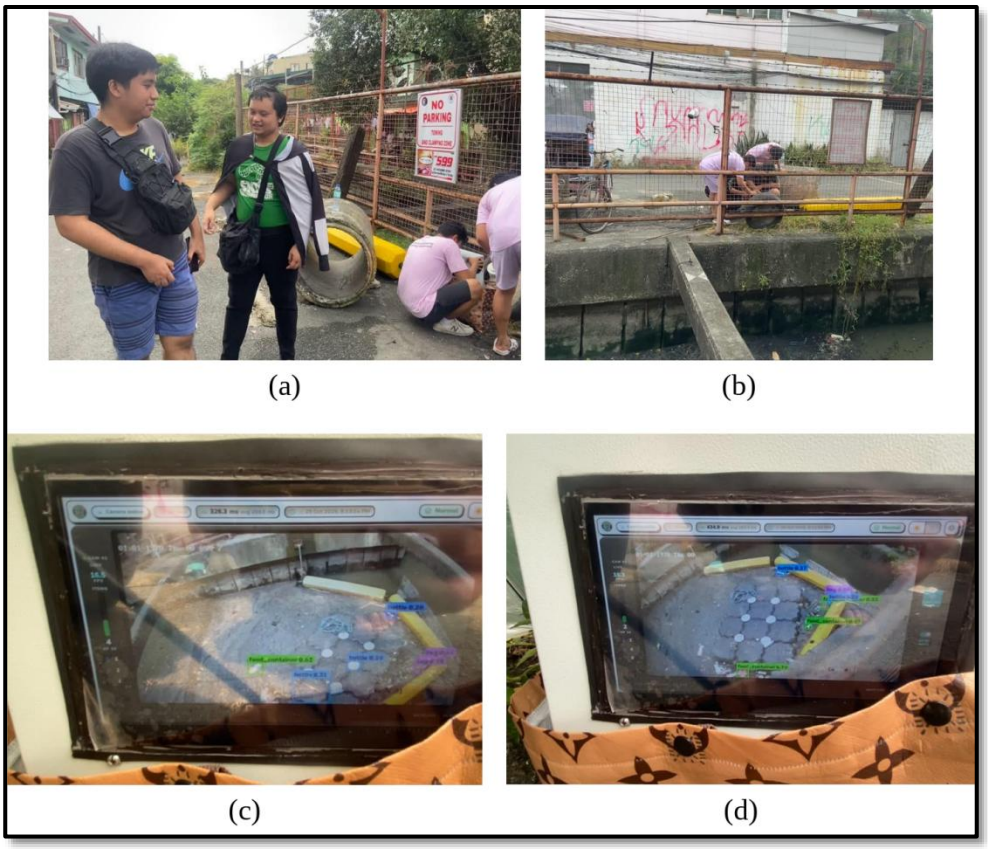
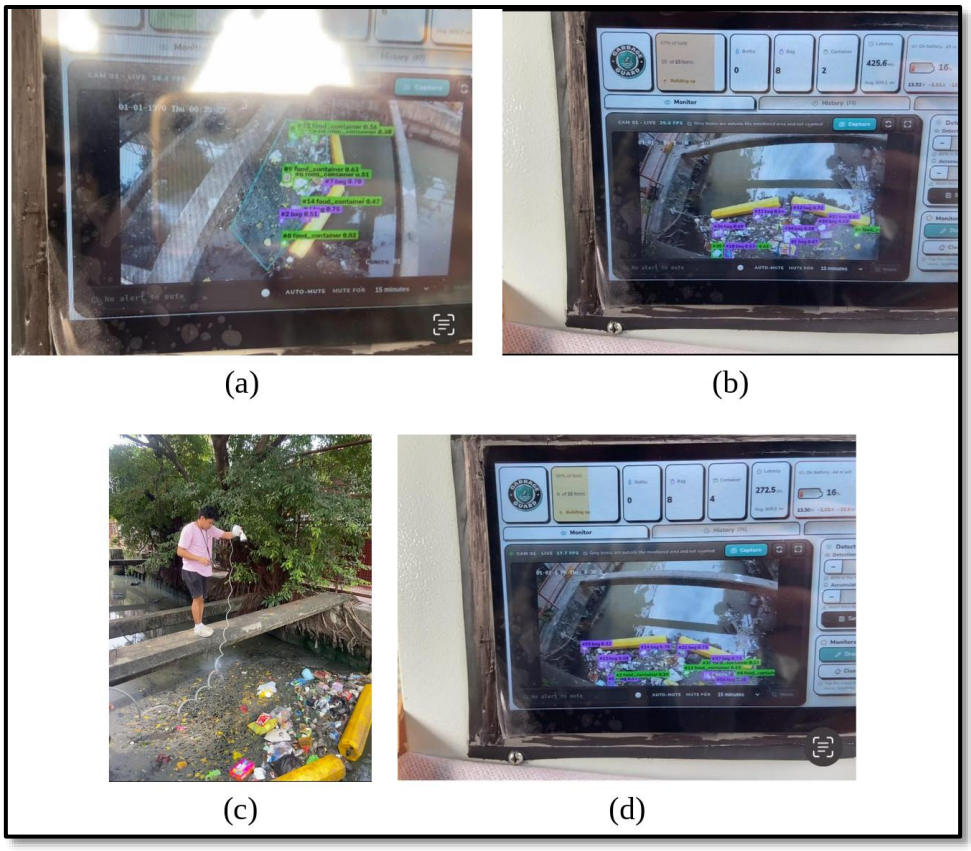



Figure E7 shows the second test, held at a creek in Barangay 870, Pandacan, Manila, under the permit in Appendix C. The retrained model now detected the garbage in the creek and labelled each item with the class it had been annotated with, as the screens in (c) and (d) show. Confidence was still low, however, and detection still needed work. Two changes followed. The model was trained again with the annotation focused on creek images, and the settings of the CCTV camera were changed to give a better picture. Taken on October 21, 2026.

**Figure E8**

*Second Independent Test and First Accumulation Alert*




The third run, shown in Figure E8, was again an independent test at another creek to gather pictures, and detection improved markedly. The model found the garbage according to the annotated classes, and for the first time the count reached the configured threshold of 20 and set off the alarm. Panels (a), (b) and (d) show the touchscreen dashboard during the test and (c) a member at the creek. Some problems remained, but they were manageable. The next steps were to annotate more images and to raise the confidence of the detections. Taken on October 27, 2026.

**Figure E9**

*Third Independent Test and Image Collection for Model v10*

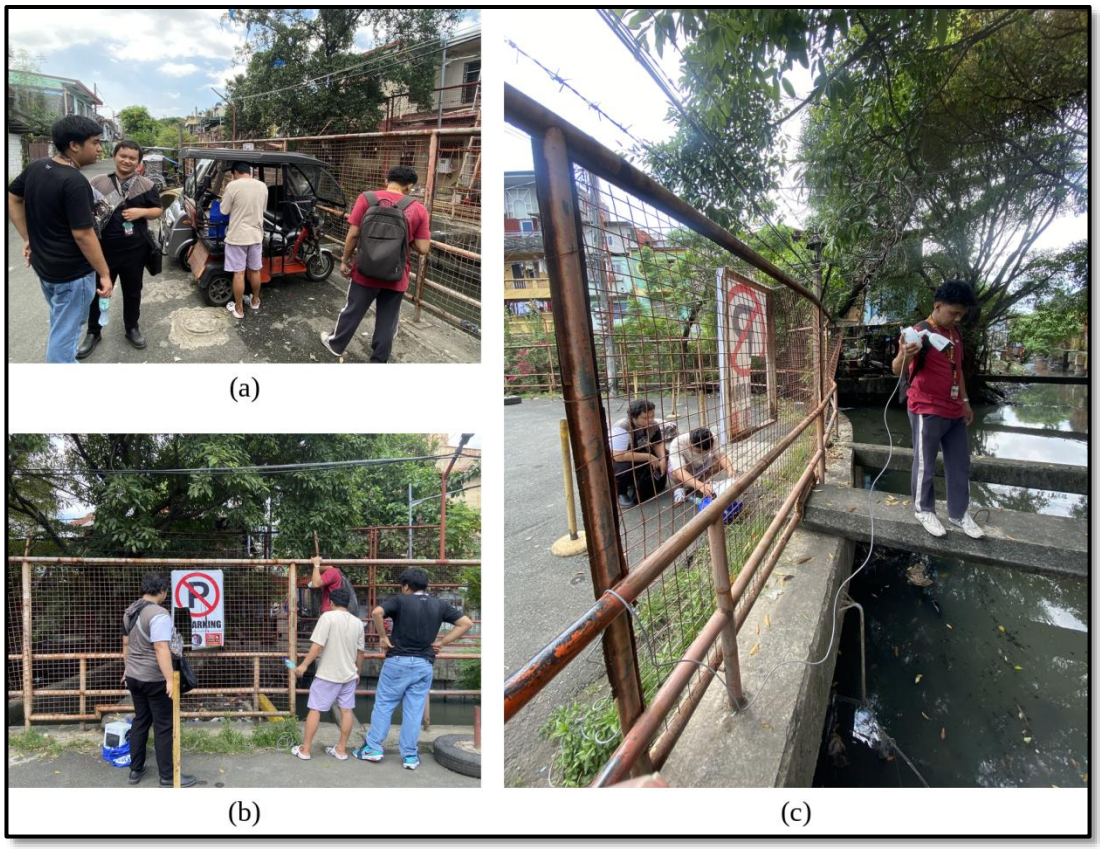



Figure E9 shows the fourth run, an independent test held back at the creek in Barangay 870, Pandacan, Manila. Panel (a) shows the team arriving at the bridge, (b) the team at the railing setting up, and (c) a member on the ledge beside the bridge holding the camera over the water while two others work at the railing. The session was also used to take new pictures of the creek, to be annotated for model v10. Taken on November 4, 2026.

**Figure E10**

*Measuring the Camera Cable from the Barangay Building to the Bridge*




Figure E10 shows the camera cable being measured on the same day, from the barangay building to the bridge where the camera is to be mounted. Panels (a) and (b) show the cable laid out along the street. The 40m 2-in-1 Ethernet and power cable covered the distance with about 15m left over, so one cable is enough to link the unit in the barangay building to the camera on the bridge. Taken on November 4, 2026.

**Figure E11**

*First Day of Deployment Test*

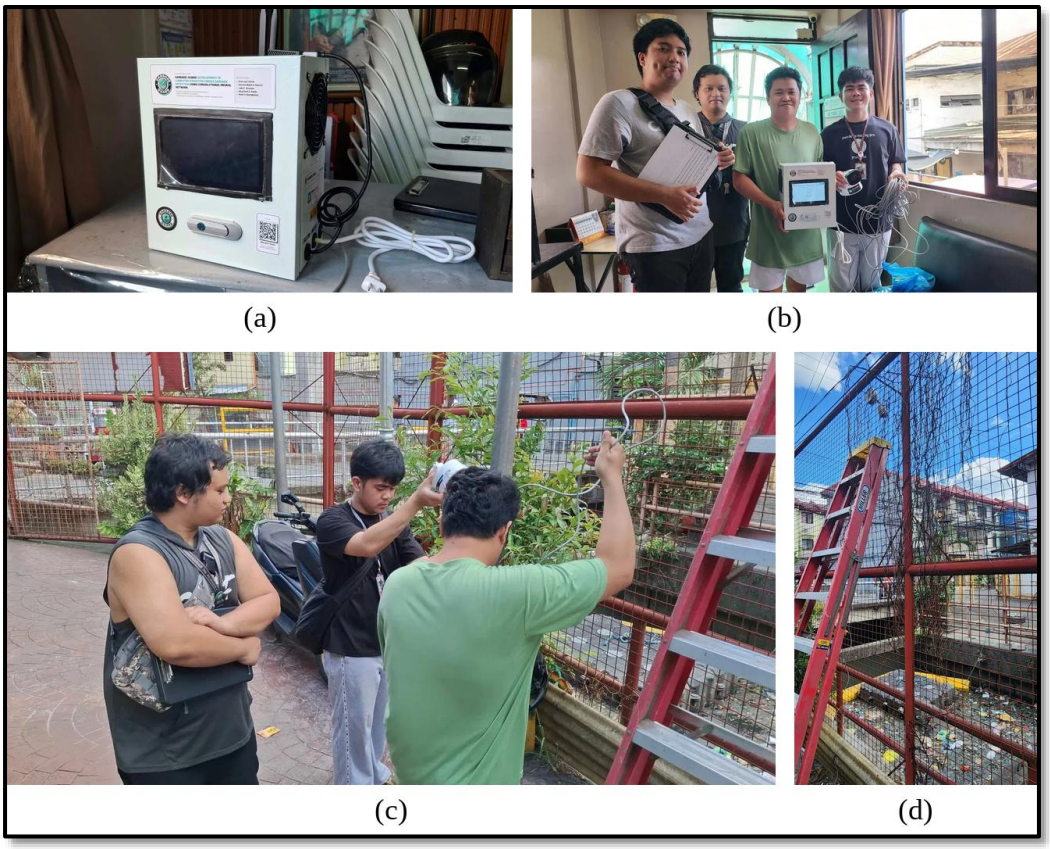



Figure E11 shows the first day of the deployment test at the barangay. Panel (a) shows the prototype set up on a table, with its touchscreen, case fan and power cord, and panel (b) shows the researchers with the unit, the camera and a coil of network cable before the installation began. Panels (c) and (d) show the camera being set up on the fence beside the creek, at a spot 13ft above the water. A ladder was used to reach the mounting point, and in (d) the trash floating in the creek can be seen from where the camera sits. Taken on November 6, 2026.

**Figure E12**

*Usability Survey with the Barangay Personnel and the Estero Rangers*




Figure E12 shows the usability survey held on the first day of the deployment test with the barangay personnel, the Estero Rangers and the barangay secretary. Each of them used the touchscreen and the Android application at the unit and then filled in the Usability Survey Form in Appendix F; their ratings and comments are in Appendix G. Panel (a) shows the researchers at the unit on the desk in the barangay hall while a respondent looks on. Panel (b) shows a member following the live camera feed on the phone application beside the touchscreen, while the researchers and the respondents gather around the unit and one of them holds the survey form. Taken on November 6, 2026.

**Figure E13**

*Second Day of Deployment Test*




Figure E13 shows the second day of the deployment test at the barangay. The camera was lowered from about 13ft to about 9ft above the water on this day, after the first day's count showed that the dashboard counted only 21 to 31% of the items counted by eye. Panel (a) shows two members crouched together under a red umbrella on the road beside the fence, a member standing with a tablet and the camera on the fence behind them. Panel (b) shows a member adjusting the camera on the fence rail at its new height. With the camera at about 9ft the dashboard counted 40 to 50% of the items counted by eye (Table 17), and live classification rose from 62.5% to 73.5% (TC_ACC_01). Taken on November 7, 2026.

**Figure E14**

*Third Day of Deployment Test*




Figure E14 shows the third day of the deployment test at the barangay. The camera was lowered again, to about 7ft above the water. Panel (a) shows one member standing on the concrete base beside the lamp post and reaching over the fence rail, while another sits on the road beside the unit, which rests on a tyre, with papers and a cable next to it. Panel (b) shows the camera fixed to the fence rail and aimed at the creek, with the floating trash below it and the cable that leaves the camera. With the camera at about 7ft the dashboard counted 49 to 54% of the items counted by eye (Table 17), and live classification reached 85.0%, the only day above the 80% target (TC_ACC_01). Taken on November 9, 2026.

**Figure E15**

*Final Defense*




Figure E15 shows the researchers with members of the panel, their adviser and the barangay representative after the final defense, with the prototype on the table and the title slide behind them. From left to right: Prof. Marc Ardie V. Ardiente, Prof. Minabelle D. Villafuerte, Dr. Jonel R. Macalisang, Rodel S. Banogbanog, Divine Grace De Leon (Estero Ranger of the DENR and representative of the barangay), Jade C. Gunayan, King Perth G. Pepito, Genesis Walter D. Reyes II, Brian Jay Celeste, and Prof Ma. Ian P De Los Trinos. Taken on November 11, 2026.

**Figure E16**

*Completion of the Final Defense at the TUP Plaza*




(a) (b)

Figure E16 shows the five researchers at the university seal in the TUP plaza after completing the final defense, each holding papers from the defense. Panel (a) shows the group standing in front of the seal, and panel (b) is a group selfie taken at the same spot. Taken on November 11, 2026.

## Appendix F

## Usability Survey Form

# Usability Survey Form

For barangay personnel, after they have used the touchscreen and the phone app. One sheet per person.

Name (optional): _____ Position: _____ Date: _____

Encircle one number for each statement: 5 Strongly Agree, 4 Agree, 3 Neutral, 2 Disagree, 1 Strongly Disagree

| Code | Statement | Rating |
|---|---|---|
| USR-U1 | Information displayed on the monitor is easy to read and understand. | 5 4 3 2 1 |
| USR-U2 | Visual bounding boxes on the camera feed clearly showed where the garbage is located on the creek. | 5 4 3 2 1 |
| USR-U3 | The Accumulation Alert is clear, noticeable and effectively informs when the creek needs cleaning. | 5 4 3 2 1 |
| USR-U4 | The system provides helpful insights that would assist local authorities in monitoring creek pollution. | 5 4 3 2 1 |
| USR-U5 | The Android application shows the alerts and the history clearly, and what it shows matches the touchscreen. | 5 4 3 2 1 |

**Comments or suggestions:**

_____

_____

_____

_____

_____

## Appendix G

## End-User Usability Survey Responses

This appendix lists the ratings and comments of the four end users who answered the Usability Survey Form in Appendix F on November 6, 2026. Their weighted means are the ones reported in Table 20 of Chapter IV.

### Table G1

*Ratings and Comments of the Respondents in the End-User Usability Survey*

| Respondent | Position | U1 | U2 | U3 | U4 | U5 | Comment |
|---|---|---|---|---|---|---|---|
| Marlon Ocampo | Estero Ranger | 5 | 4 | 4 | 5 | 4 | ANG SYSTEM AY MAGANDA AT MALAKING TULONG PARA SA BARANGAY AT LALO NA SA AMING ESTERO RANGERS PARA SA PAGMONITOR NG ILOG AT WATERWAYS. |
| Joey L. Garcia | Barangay Kagawad | 4 | 4 | 5 | 5 | 4 | SANA MAGING SUCCESSFUL ANG EXPERIMENT NA ITO PARA MAKATULONG SA ATING KALIKASAN. |
| Anelyn D. Layco | Barangay Secretary | 5 | 4 | 3 | 4 | 4 | Ok naman yung device, suggestion lang na mas dagdagan pa yung na didetect na basura, bukod sa plastic bags, bottles and container. Para mas mabilis na pag lilinis din sa ating estero. |
| Divine Grace De Leon | Estero Ranger | 4 | 5 | 5 | 5 | 5 | MAGANDA ANG PORPOSE NG SYSTEM OR CAMERA PARA SA ESTERO SA AMING BRGY, MAKAKATULONG SA PAGBILIS NG PAGLINIS AT PAGSASAGAWA NG MGA BASURANG NASA ILOG. |
| Weighted mean | | 4.50 | 4.25 | 4.25 | 4.75 | 4.25 | Overall: 4.40 |

*Note.* N = 4. U1 to U5 are the statements USR-U1 to USR-U5 of Table 8, each rated on the five-point scale of Table 9 (5 = Strongly Agree, 1 = Strongly Disagree). Comments are given as the respondents wrote them, in Filipino. The survey was answered on November 6, 2026.

## Appendix H

## User Interface of the System

This appendix presents the user interface of Garbage-Guard: the web dashboard that the Raspberry Pi 5 serves to its own touchscreen and to browsers on the same network, and the Android mobile application. The screens were captured from the current versions of both programs connected to the project's mock server, which serves the same interface as the Pi with test data. For this reason, the camera area shows a test picture in place of the creek.

**Figure H1**

*Web Dashboard Monitor Tab*

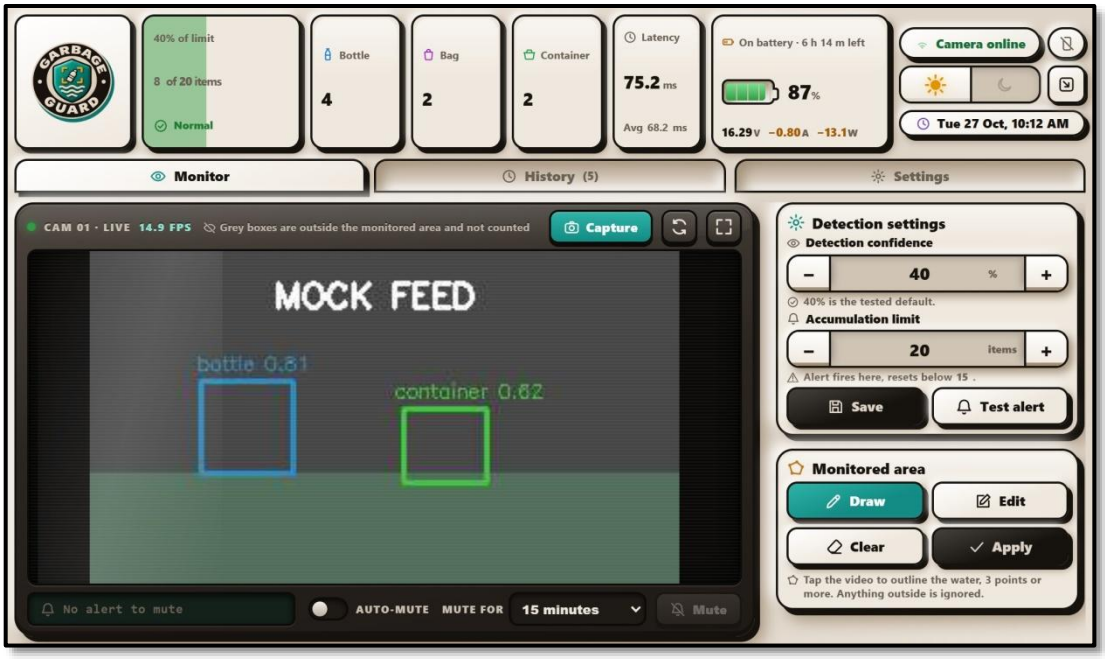



Figure H1 shows the web dashboard's Monitor tab as displayed on the 7-inch touchscreen of the Raspberry Pi 5. The top strip shows the level of accumulation, the number of each garbage class in the monitored area, inference latency, UPS battery status, the camera link and the clock. The live feed draws a box with a confidence score around each item, and the boxes vary in shape and size with the items. The right panel shows the

confidence threshold, the level of accumulation, and controls to limit the monitored area. It also provides controls to enable and disable alerting and muting.

**Figure H2**

*Web Dashboard During an Accumulation Alert*

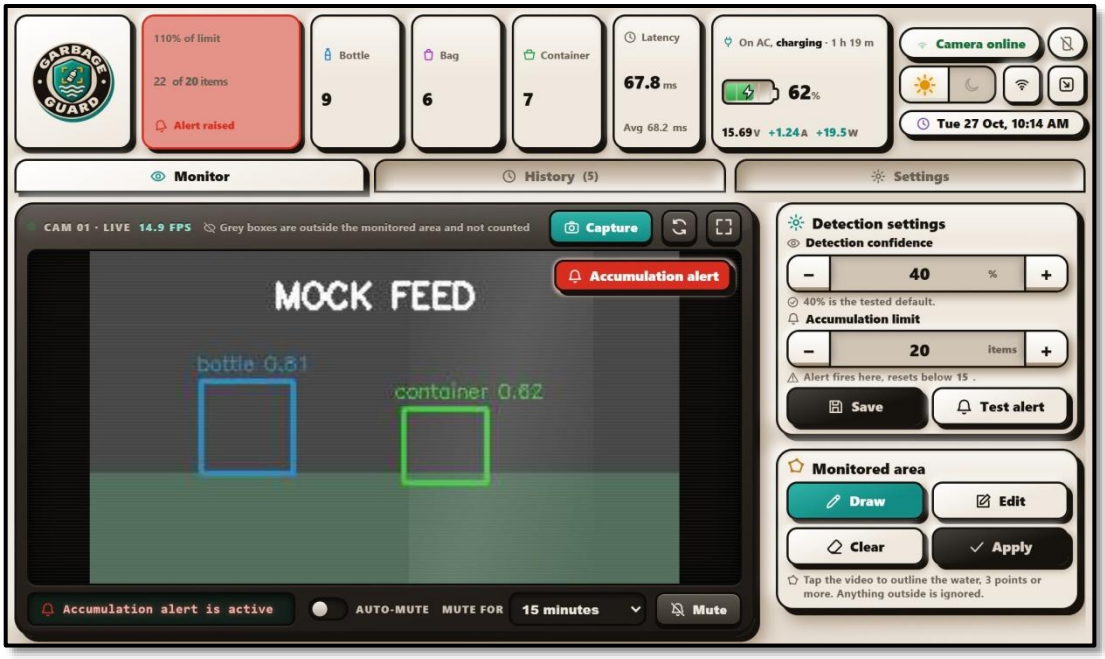



Figure H2 shows an active accumulation alert. The level box is red and states, “Alert raised.” The feed shows an Accumulation Alert tag. The mute bar states the alert is active and can be turned off for a time by the operator. The alert is cleared automatically when the smoothed count is reduced to the re-arm level.

**Figure H3**

*Drawing the Monitored Area on the Web Dashboard*

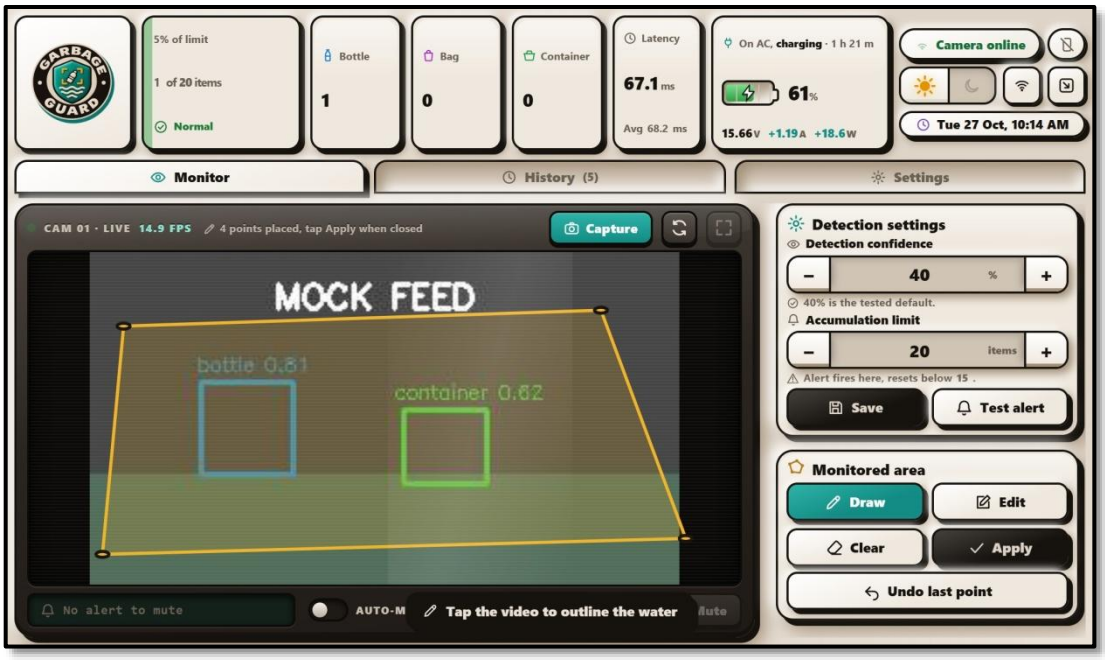



Figure H3 shows the monitored area being drawn. The operator taps the video to place the corners of a polygon over the water, and only items whose center falls inside it are counted, while detections outside it are drawn as thin grey boxes. The area is saved with Apply and can be edited, cleared or undone one point at a time. The area can also be drawn from the fullscreen view shown in Figure H4.

**Figure H4**

*Web Dashboard Fullscreen View*

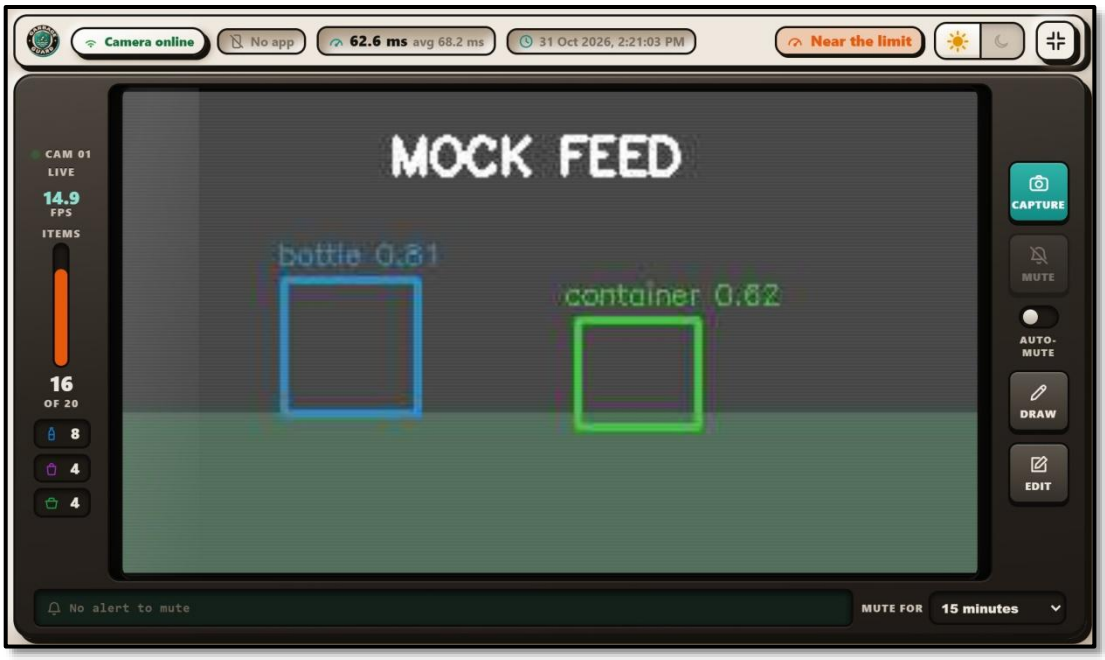



Figure H4 shows the Fullscreen view, which makes the camera picture occupy most of the screen. The side columns keep the item count against the limit, the count per class, the frame rate and the latency, together with the capture, mute and auto-mute keys. The Draw and Edit keys on the right let the operator outline the monitored area without leaving Fullscreen. While drawing, these keys change to Cancel, Undo, Clear and Apply, and a strip across the top of the picture counts the points placed. Two fingers pinch to zoom into the picture and drag to pan it, a Reset zoom key returns to the whole frame, and one finger still places the points of the area. Full-size snapshots opened from the history zoom the same way.

**Figure H5**

*Web Dashboard in Dark Mode*

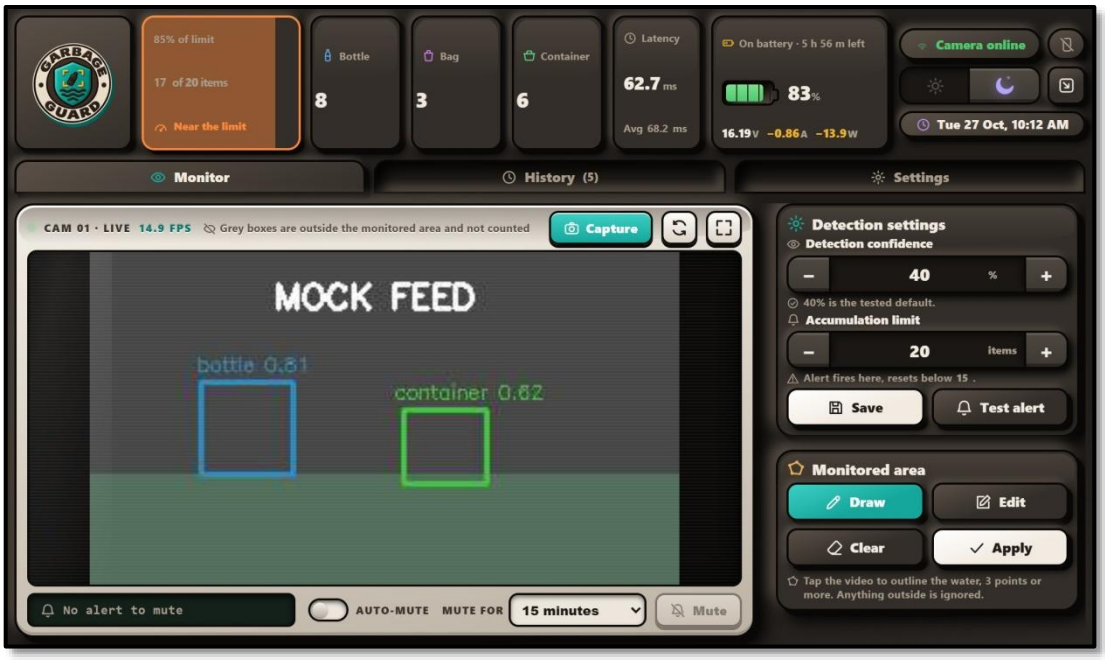



Figure H5 shows the dashboard in dark mode, which is selectable via the sun and moon switch in the top right corner of the screen. Every page follows the theme choice, which makes the touchscreen easier to read in dark mode.

**Figure H6**

*Web Dashboard History Tab*

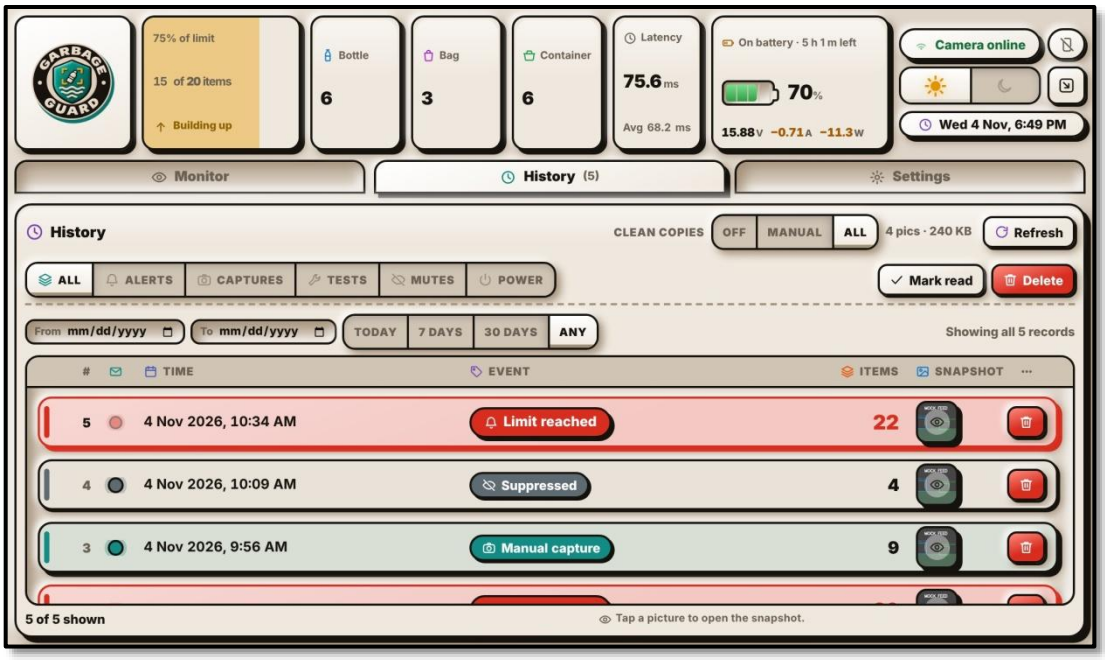



Figure H6 shows the History tab, which lists all events stored in the SQLite database on the Pi. Events include accumulation alerts, spikes, manual captures, test alerts, mutes and power events. Each row carries a record number, counted down from the newest, and a square thumbnail of its snapshot with an eye mark; tapping the thumbnail opens the snapshot. Older rows load as the list is scrolled, so the whole history is reached without changing pages. Events can be filtered by type and by a date range, marked as read and deleted. The Clean copies switch determines whether an unannotated copy of each snapshot is kept.

**Figure H7**

*Snapshot Viewer on the Web Dashboard*

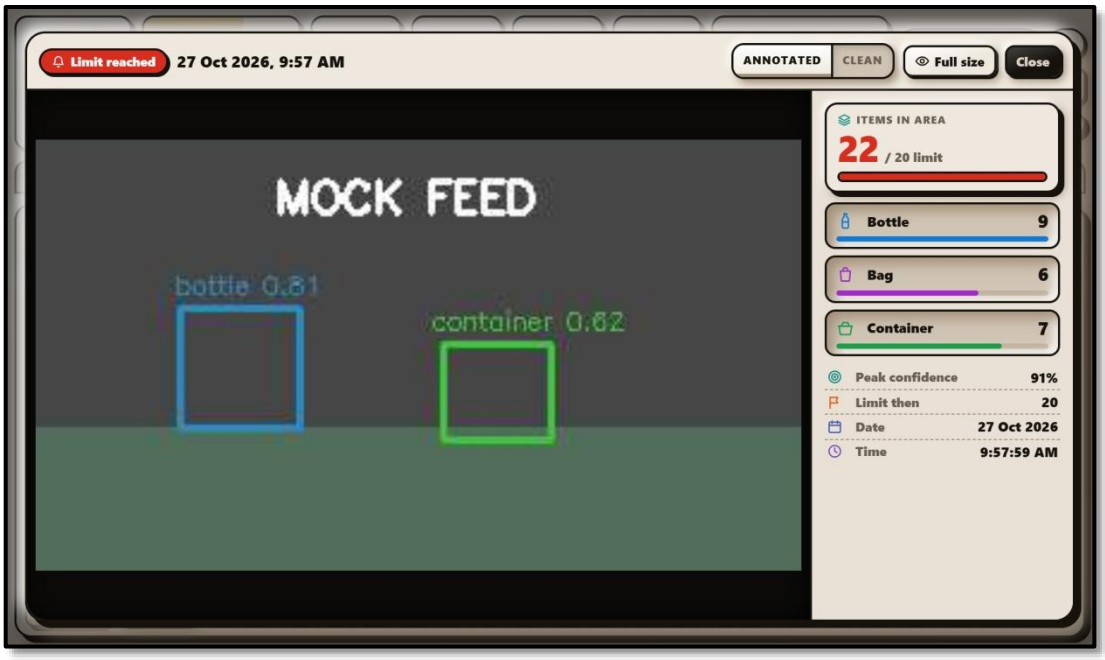



Figure H7 shows a snapshot opened from the history. The saved frame appears on the left, either annotated or as the clean copy, and the panel on the right lists the number of items in the area against the limit, the count per class, the peak confidence, and the date and time of the event.

**Figure H8**

*Delete Confirmation on the Web Dashboard*




Figure H8 shows the alert and snapshot removal confirmation. This keeps a user from deleting a record with an accidental tap.

**Figure H9**

*Web Dashboard Settings Tab*

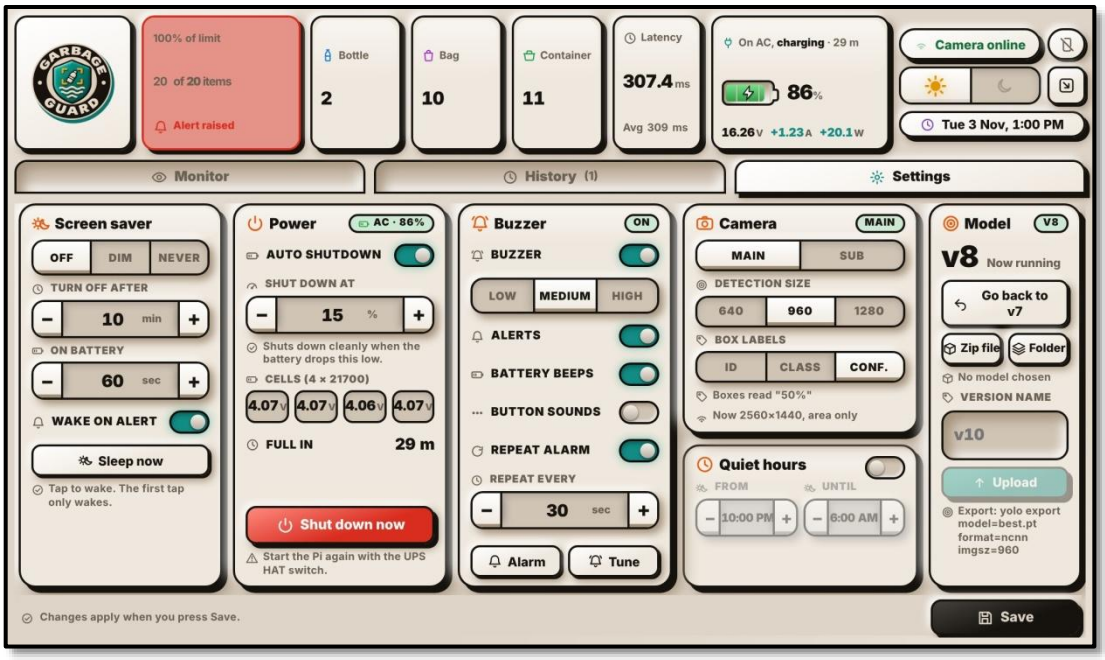



Figure H9 shows the Settings tab, laid out in five columns on the touchscreen. The Screen saver setting determines when the touchscreen turns off or dims. Power sets the battery shutdown level and shows the voltage of each battery cell. The Buzzer setting determines the volume and the events that sound. The Camera card selects the camera stream and the detection size, and its Box labels row chooses what is written beside each detection box: the track ID, the class name, the confidence, or any mix of the three. With only the confidence selected, a box reads “50%.” Quiet hours, a time window during which the buzzer is disabled, sit below the Camera card, with the times shown greyed out while the setting is off. The Model card shows the model that is running, goes back to the previous one with a single button, and installs a new one from a zip file or a folder under a version name.

**Figure H10**

*Battery Details on the Web Dashboard*

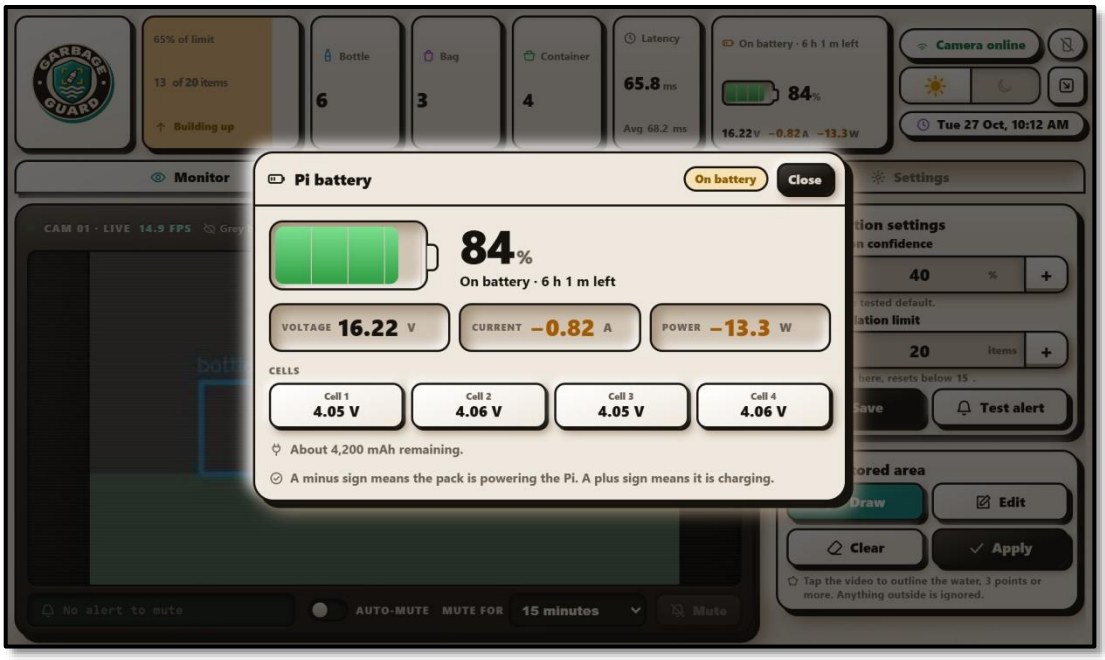



Figure H10 shows the battery details opened from the battery box. It reports the charge, the voltage, current and power of the UPS HAT, the voltage of each of the four cells and the estimated remaining capacity. A minus sign means the pack is powering the Pi, and a plus sign means it is charging.

**Figure H11**

*Connected Phones Panel on the Web Dashboard*

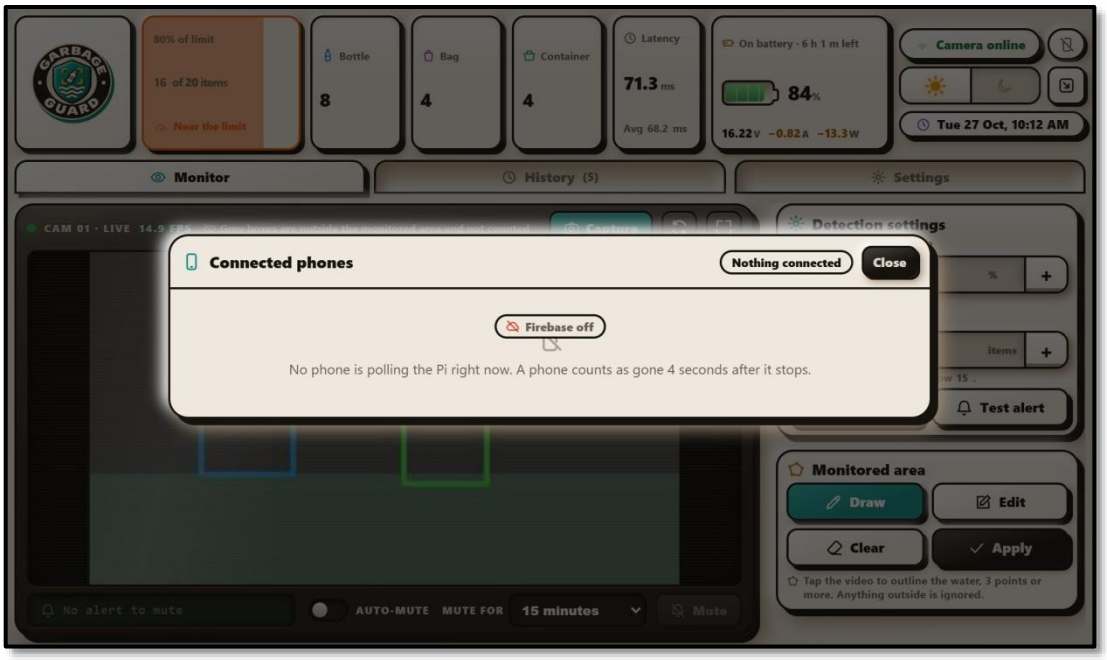



Figure H11 shows the Connected phones panel, which lists the phones that are polling the Pi and whether Firebase sync is on. No phone was connected when this screen was captured.

**Figure H12**

*Network Setup Page of the Raspberry Pi*

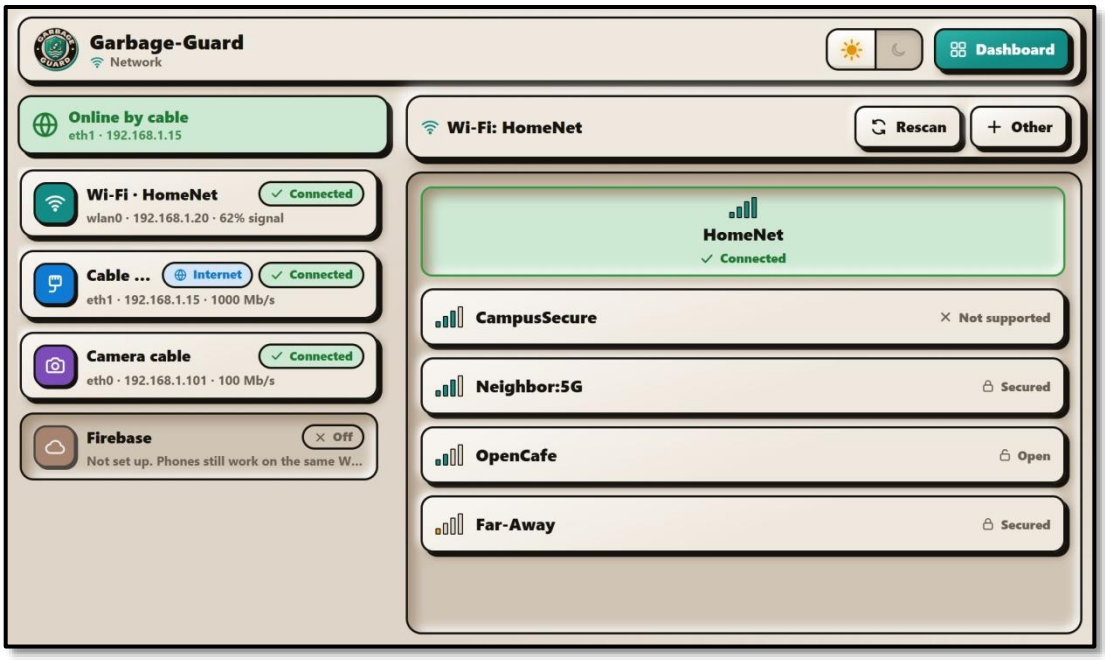
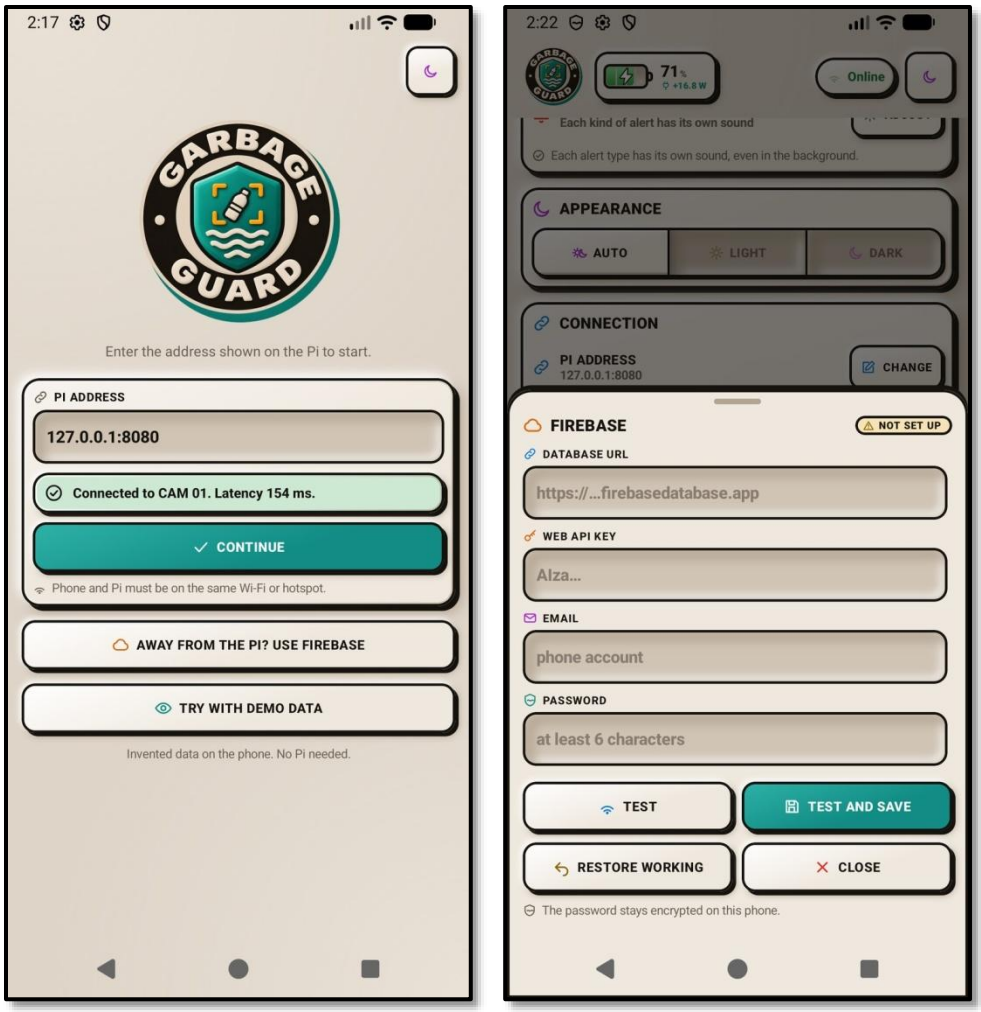



Figure H12 shows the network page used to set up the connection of the Pi. The left column shows the state of the Wi-Fi, the cable link to the internet, the camera cable and Firebase, and the right column lists the Wi-Fi networks in range. Wi-Fi and address settings can only be changed from the Pi's own touchscreen; other devices see the page as view only.

**Figure H13**

*Mobile App Connection Screens*

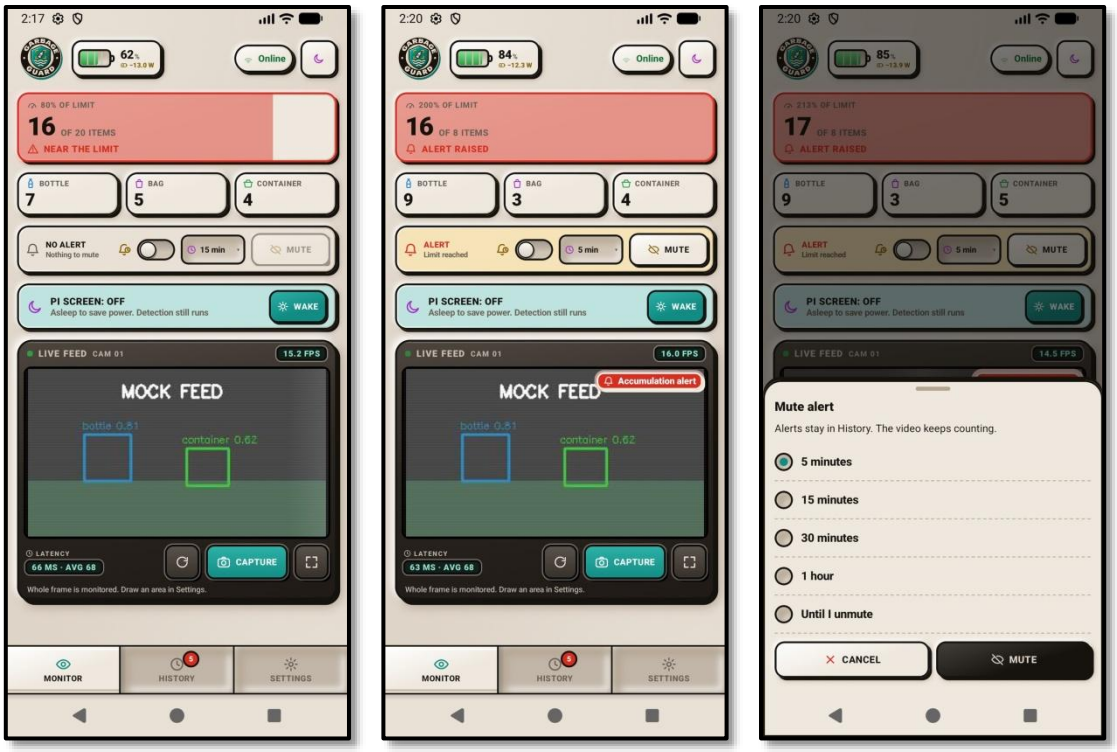



(a) (b)

Figure H13 shows the first screens of the mobile application. In (a) the user enters the address shown on the Pi and taps Test connection; once the Pi answers with its camera name and latency, Continue opens the app. The same screen offers a demo mode with invented data. In (b) the user enters the Firebase database address, web API key and account so the app can still receive alerts and still pictures when the phone is away from the Pi's network.

**Figure H14**

*Mobile App Monitor Screen*




(a) (b) (c)

Figure H14 shows the Monitor screen of the app. In (a) the count is near the limit, so the level card turns red as a warning before an alert is raised. In (b) an accumulation alert is active: the card reads Alert raised, the feed carries the alert tag and the Mute key becomes available. In (c) the user picks how long to mute the alert, from five minutes up to indefinitely, until the user unmutes it. The card below the counts shows whether the Pi's own screen is asleep and lets the user wake it.

**Figure H15**

*Mobile App Fullscreen View*

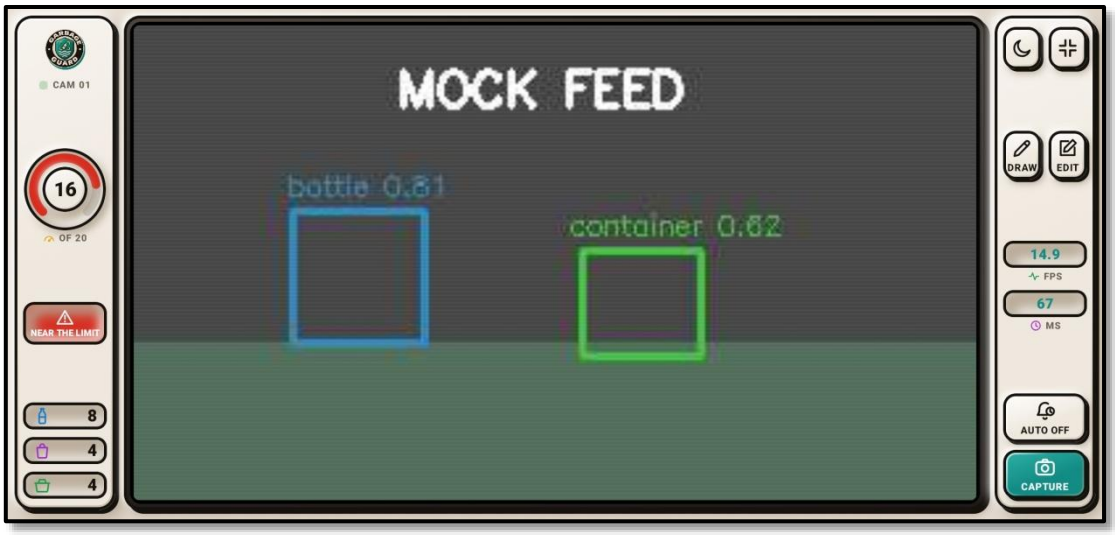
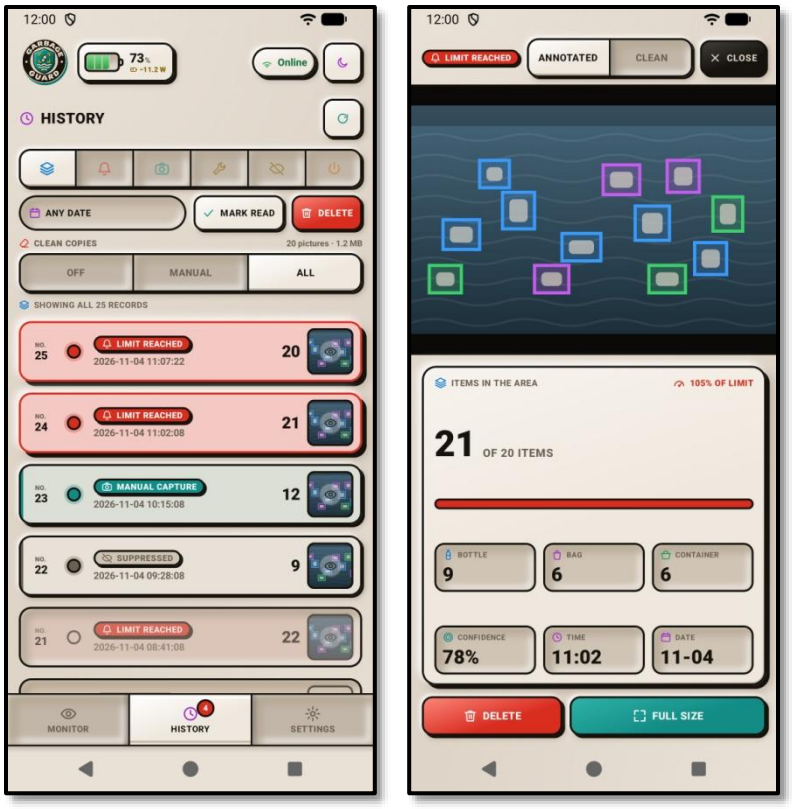



Figure H15 shows the app in Fullscreen, which turns the phone sideways and gives the camera picture the whole display. The count against the limit, the count per class, the frame rate, the latency, and the auto-mute and capture keys stay on the sides. The Draw and Edit keys on the right panel open the same drawing mode used in Settings, and Cancel or Apply returns the user to the Fullscreen feed. As on the dashboard, a two-finger pinch zooms and pans the picture and Reset zoom returns to the whole frame.

**Figure H16**

*Mobile App History Screen*

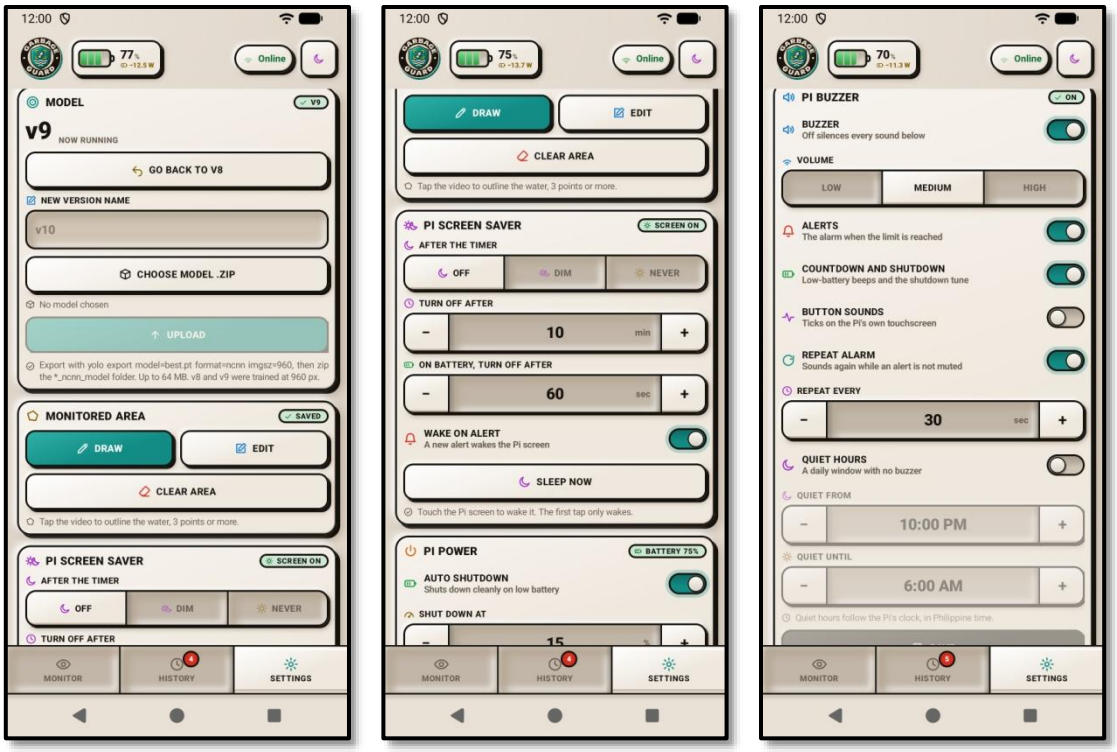



(a) (b)

Figure H16 shows the History screen. In (a) each event carries a record number, counted down from the newest, with its type, time and item count and a snapshot thumbnail with an eye mark; the list can be filtered, marked as read or deleted. In (b) a record is opened by tapping its thumbnail and shows its snapshot, annotated or clean, together with the counts, confidence and time of the event.

**Figure H17**

*Mobile App Settings for Detection, Pi Screen and Buzzer*

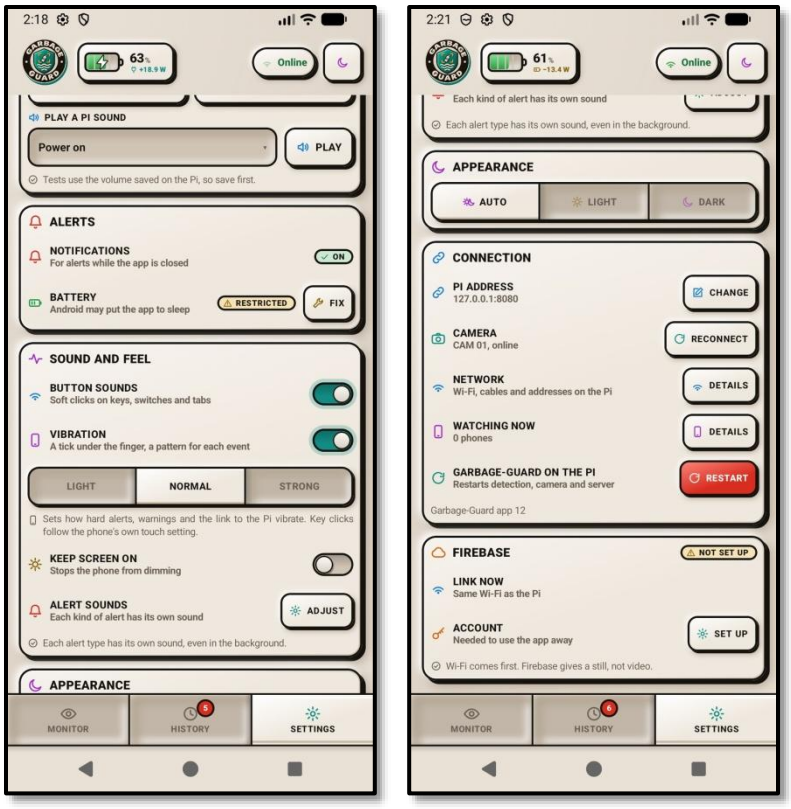



(a) (b) (c)

Figure H17 shows the middle part of the Settings screen. In (a) the Model card shows the model that is running, goes back to the previous one, and uploads a new model as a zip file under a version name. In (b) the user draws the monitored area and sets the screen saver of the Pi. In (c) the user controls the buzzer of the Pi, including the events that sound and the repeating alarm, and sets the quiet hours, whose times stay greyed out while the setting is off.

**Figure H18**

*Mobile App Settings for Alerts, Appearance and Connection*

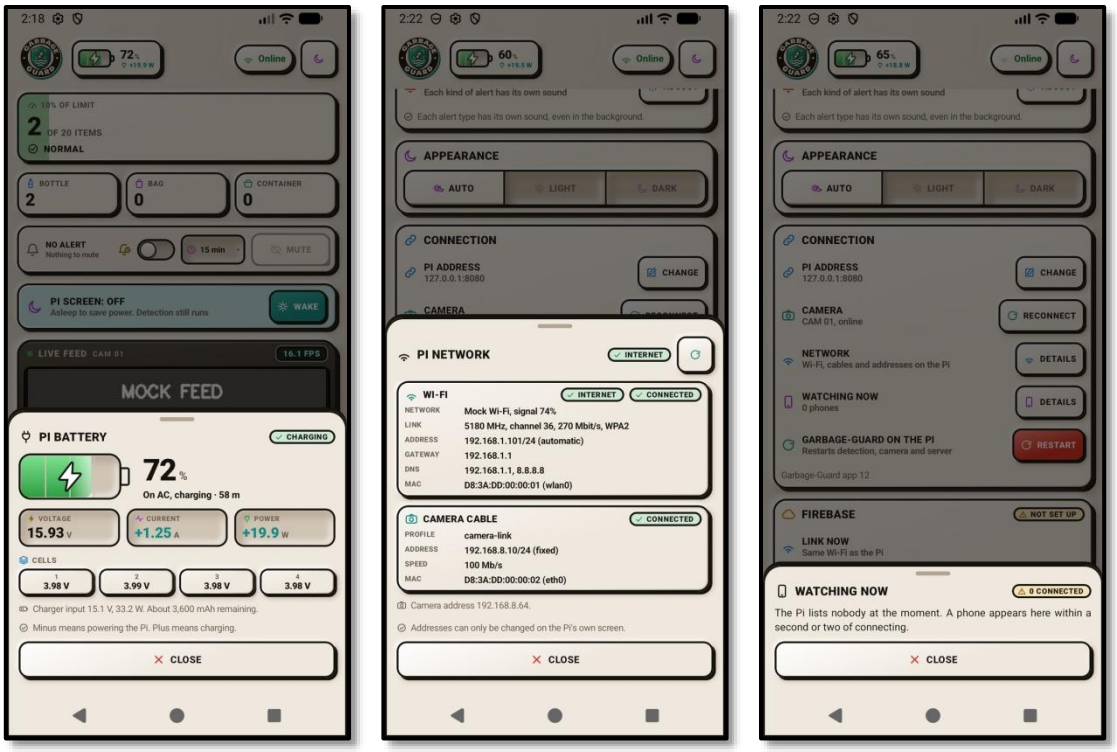



(a) (b)

Figure H18 shows the rest of the Settings screen. In (a) the phone's own alert notifications, button sounds and vibration are set. In (b) the user chooses the theme, checks the connection to the Pi and its camera, opens the network and viewer details, restarts the detection program, and links the app to Firebase.

**Figure H19**

*Mobile App Detail Panels*




(a) (b) (c)

Figure H19 shows the detail panels of the app. In (a) the battery panel gives the charge, voltage, current, power and cell voltages of the UPS HAT. In (b) the network panel lists the Wi-Fi and camera cable addresses of the Pi. In (c) the Watching now panel lists the phones connected to the Pi.

**Figure H20**

*Mobile App Alert Notification*

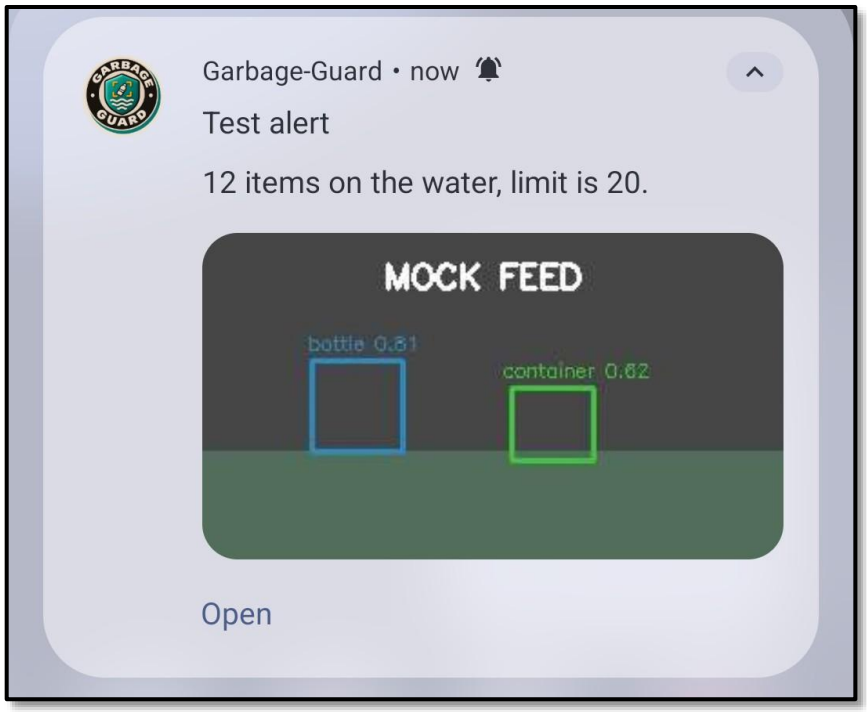



Figure H20 shows an alert notification on the phone. Each type of event has its own sound and vibration, and the notification carries the item count, the limit and the snapshot so the user can judge the situation before opening the app.

**Figure H21**

*Mobile App in Dark Mode*

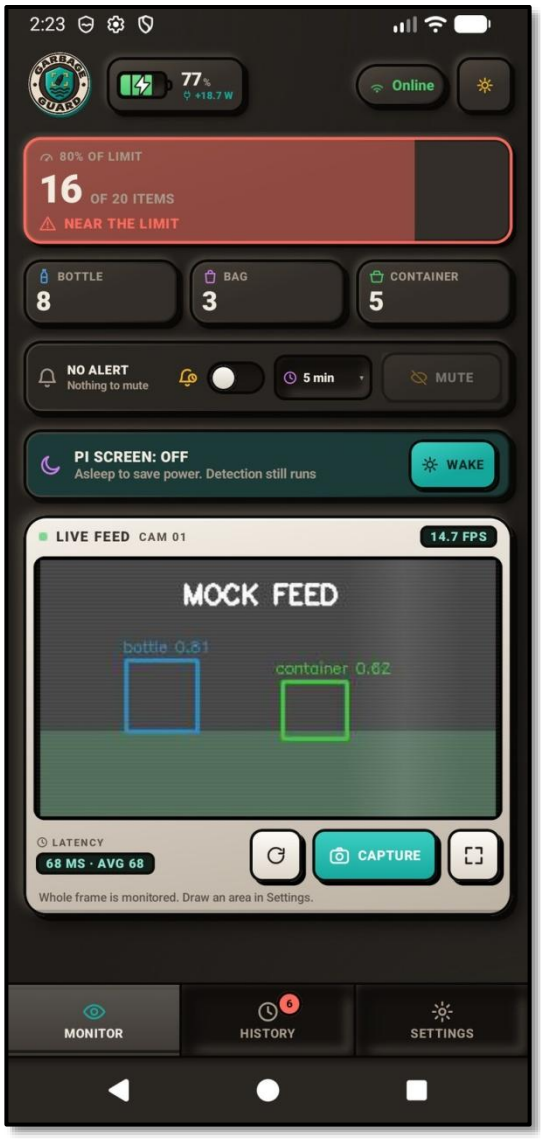



Figure H21 shows the app in dark mode, chosen under Appearance in Settings, which can also follow the phone's own theme.

# Appendix I

## User Manual

**Figure I1**

*Cover and Contents of the User Manual*

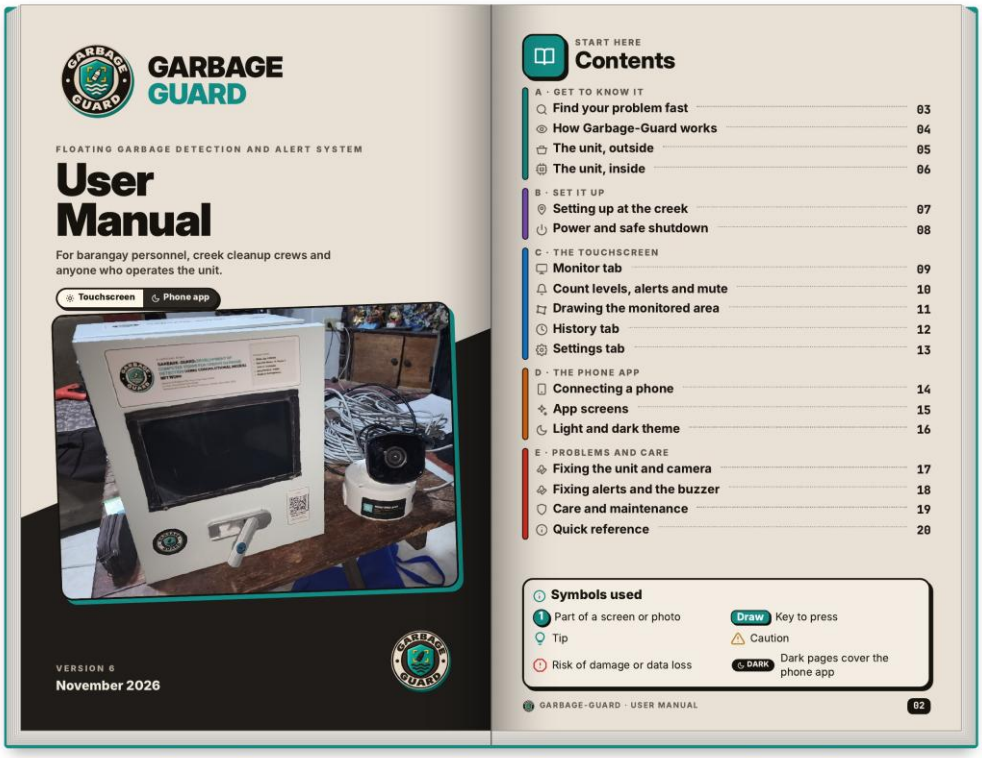



Figure I1 shows the front cover and the contents page of the User Manual. The cover says who the manual is for: barangay personnel, creek cleanup crews and anyone else who operates the unit. The manual is a booklet of 20 pages printed two to a sheet, and each figure in this appendix shows one sheet opened flat. The contents page on the right groups the manual into five parts, from getting to know the unit to fixing problems and caring for it, with a page number beside every topic so a reader can go straight to what is needed. It also explains the symbols used in the manual: a numbered circle marks a part of a screen or photo, a bulb marks a tip, and a caution sign marks a risk of damage or data loss. Pages about the phone app use a dark theme so they are easy to tell apart.

**Figure I2**

*Problem Finder and Overview of the System*

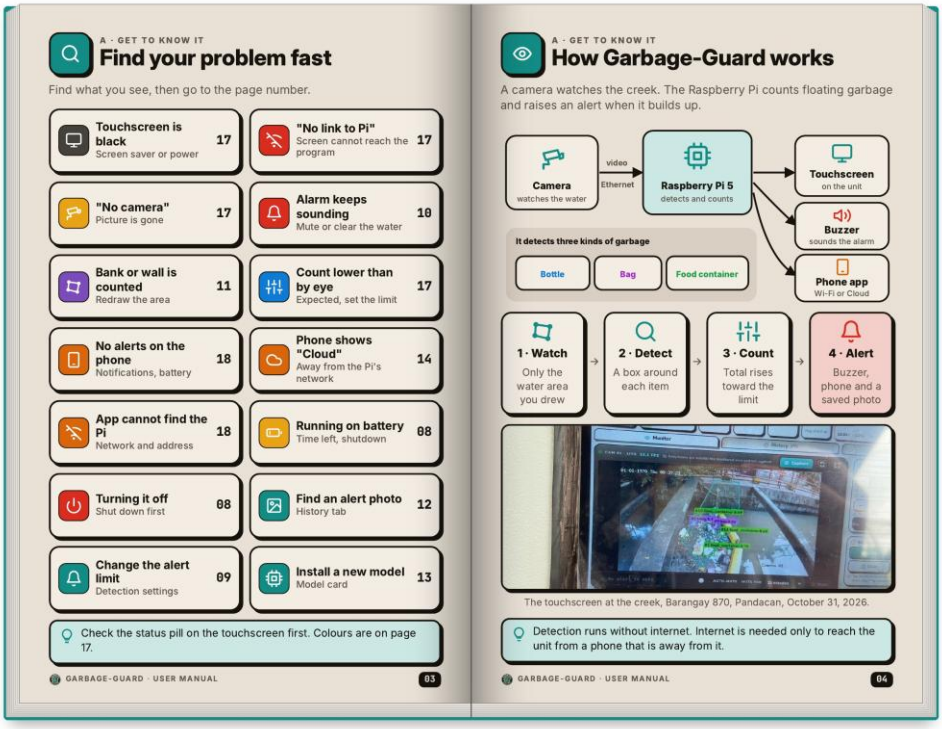



Figure I2 shows the second sheet. The left page is a problem finder: the reader looks for what they see, such as a black touchscreen, a "No camera" message, an alarm that keeps sounding or a phone that shows no alerts, and the number beside it gives the page with the fix. The right page explains in one picture how the system works. The camera watches the creek and sends video over the cable to the Raspberry Pi 5 inside the unit. The Pi detects bottles, bags and food containers, counts them, and raises an alert when the total passes the limit, which sounds the buzzer, sends a notice to the phone app and saves a photo. A note on the page says that detection runs without internet, which is needed only to reach the unit from a phone that is away from it. The photo shows the touchscreen at the creek in Barangay 870, Pandacan, on October 31, 2026.

**Figure I3**

*Outside and Inside of the Unit*

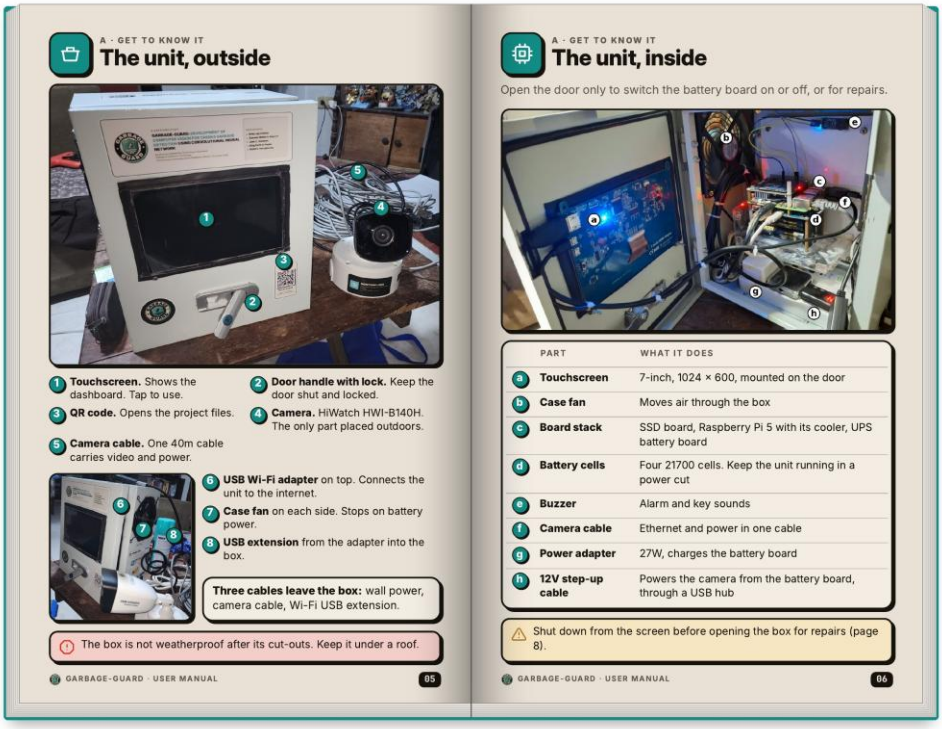



Figure I3 labels the unit from the outside and from the inside. On the left page, numbered circles mark the touchscreen, the door handle with its lock, the QR code that opens the project files, the HiWatch HWI-B140H camera and the single 40m cable that carries both video and power. The smaller photo shows the USB Wi-Fi adapter on top, a case fan on each side and the USB extension that runs from the adapter into the box. Only three cables leave the box: wall power, the camera cable and the Wi-Fi extension. The right page opens the door and lists what is inside in a table, from the touchscreen and case fan to the board stack, the four 21700 battery cells, the buzzer, the 27W power adapter and the 12V step-up cable. The manual warns that the box is not weatherproof after its cut-outs, so it must stay under a roof, and that the unit has to be shut down from the screen before the door is opened.

**Figure I4**

*Setting Up at the Creek and Safe Shutdown*

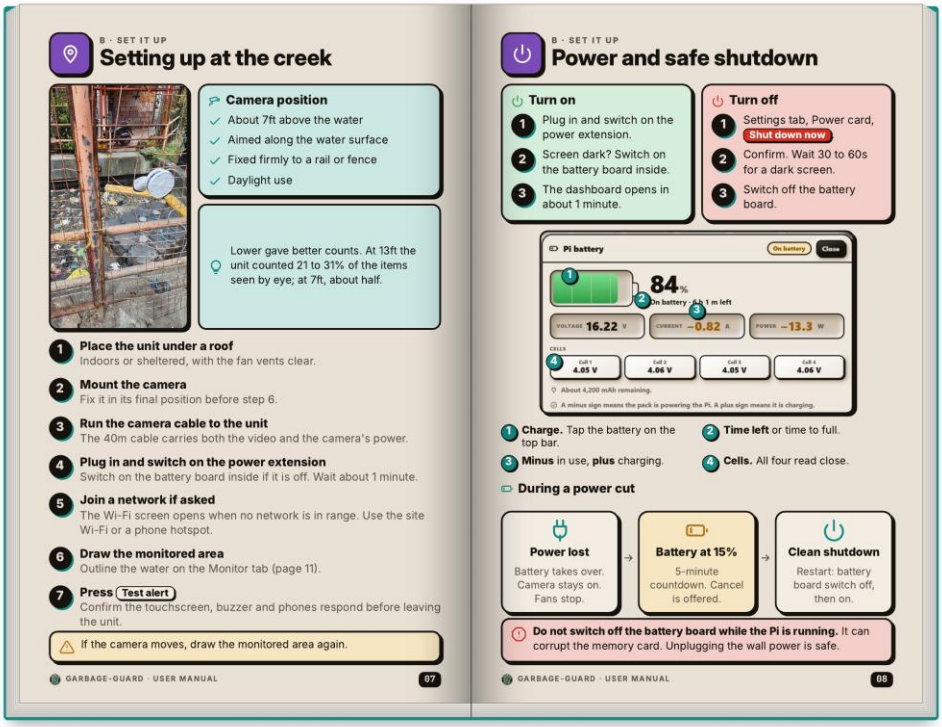



Figure I4 covers setting up and power. The left page gives the camera position: about 7ft above the water, aimed along the water surface and fixed firmly to a rail or fence, for daylight use. It also records what the tests showed, that at 13ft the unit counted 21 to 31% of the items seen by eye and at 7ft about half. Below that are the seven setup steps, from placing the unit under a roof to pressing Test alert before leaving, with a reminder to draw the monitored area again if the camera moves. The right page explains turning the unit on and off and reading the battery indicator on the top bar. It also shows what happens in a power cut: the battery takes over, and at 15% a five-minute shutdown countdown starts that can be cancelled. The page warns that the battery board must not be switched off while the Pi is running, because that can corrupt the memory card. Unplugging the wall power is safe.

**Figure I5**

*Monitor Tab and Alert Controls*

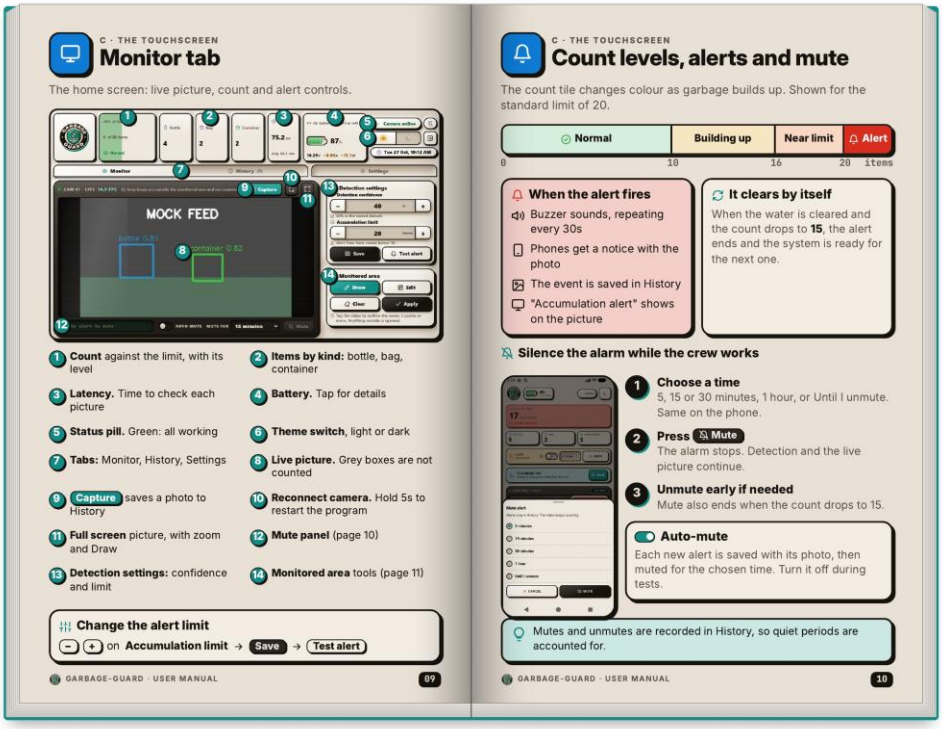



Figure I5 explains the Monitor tab, the home screen of the touchscreen. The left page numbers every control: the count against the limit, the items counted by kind, the latency, the battery, the status pill, the theme switch, the tabs, the live picture, Capture, Reconnect camera, full screen, the mute panel and the detection settings. Grey boxes in the live picture are items outside the drawn area and are not counted. The right page shows how the count tile changes colour as garbage builds up, from Normal to Building up, Near limit and Alert, for the standard limit of 20 items. When the alert fires, the buzzer repeats every 30s, phones get a notice with the photo and the event is saved in History. It clears by itself once the water is cleared and the count drops to 15. The page also shows how to silence the alarm for 5, 15 or 30 minutes, for 1 hour or until it is unmuted, and how auto-mute works.

**Figure I6**

*Monitored Area and History Tab*

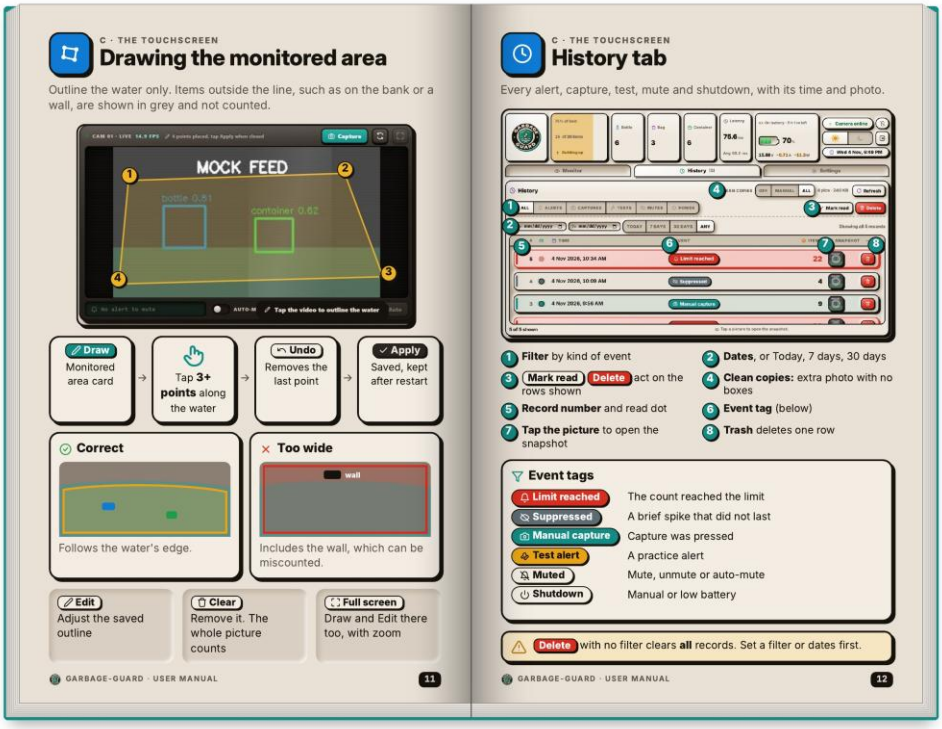



Figure I6 shows two screens the crew will use often. On the left page, the monitored area is drawn by tapping three or more points along the water, then pressing Undo or Apply. A correct outline follows the water's edge, while one that is too wide takes in the wall, which can be miscounted. Edit adjusts the saved outline and Clear removes it, which makes the whole picture count. The right page is the History tab, a record of every alert, capture, test, mute and shutdown with its time and photo. Its numbered controls filter by kind of event and by date, mark rows as read and open the snapshot. A table of event tags explains Limit reached, Suppressed (a brief spike that did not last), Manual capture, Test alert, Muted and Shutdown. The page warns that Delete with no filter clears all records, so a filter or dates should be set first.

**Figure I7**

*Settings Tab and Connecting a Phone*

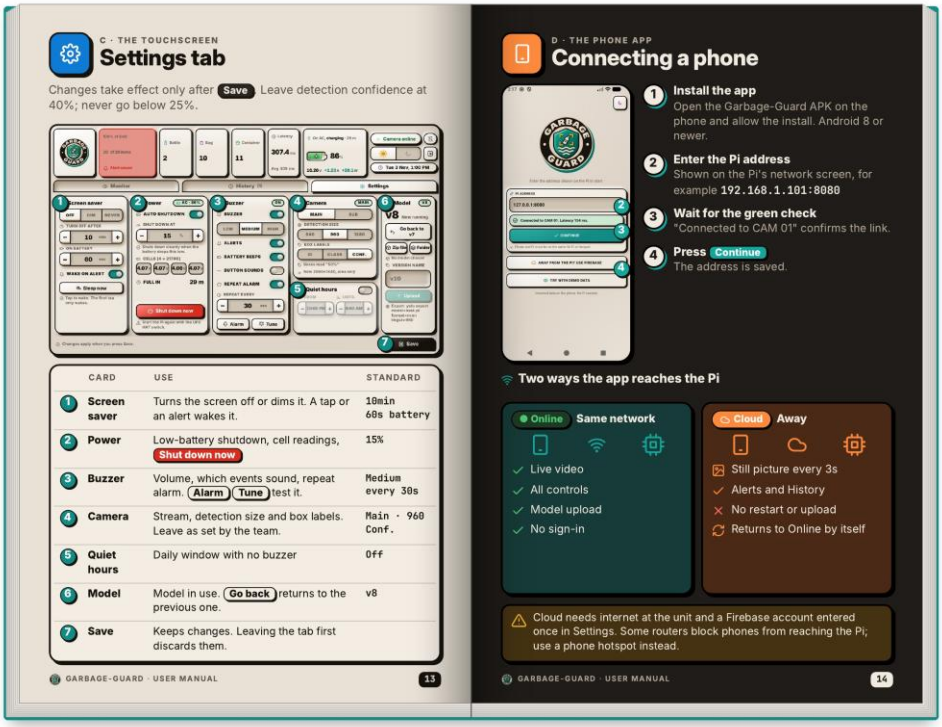



Figure I7 starts with the Settings tab on the left page, where the cards are numbered: screen saver, power, buzzer, camera, quiet hours and the model in use. A table lists the standard value of each, for example a 10min screen saver, shutdown at 15% battery, medium buzzer volume repeating every 30s and detection confidence of 40%. Changes take effect only after Save, and confidence should stay at 40% and never go below 25%. The right page, Connecting a phone, has four steps: install the APK on an Android 8 or newer phone, enter the Pi address shown on the network screen, wait for the green check that says Connected to CAM 01, and press Continue. It then compares the two ways the app reaches the Pi. On the same network it shows live video with all controls. Over the cloud it shows a still picture every 3s with alerts and History only. A note says that cloud needs internet at the unit, and that if a router blocks phones, a phone hotspot should be used instead.

**Figure I8**

*Phone App Screens and Themes*




Figure I8 shows the phone app. It has the same three tabs as the touchscreen, Monitor, History and Settings, and a change made on one appears on the other within seconds. The left page numbers the app controls: count and level, items by kind, the mute row and auto-mute, the Pi screen with its Wake button, capture and full screen, filter and dates, mark read and delete, the model in use with go back and upload, the monitored area drawing, and the Pi screen saver and power settings. It also shows an alert notification, which arrives even when the app is closed, carries the photo and the count, and has its own sound and vibration for each type of event. The right page shows that the touchscreen and the phone each have a light and a dark theme with the same layout. Light suits bright daylight outdoors and dark suits indoors or low light. The theme is chosen under Appearance in Settings as Auto, Light or Dark.

**Figure I9**

*Troubleshooting the Unit, Camera and Alerts*

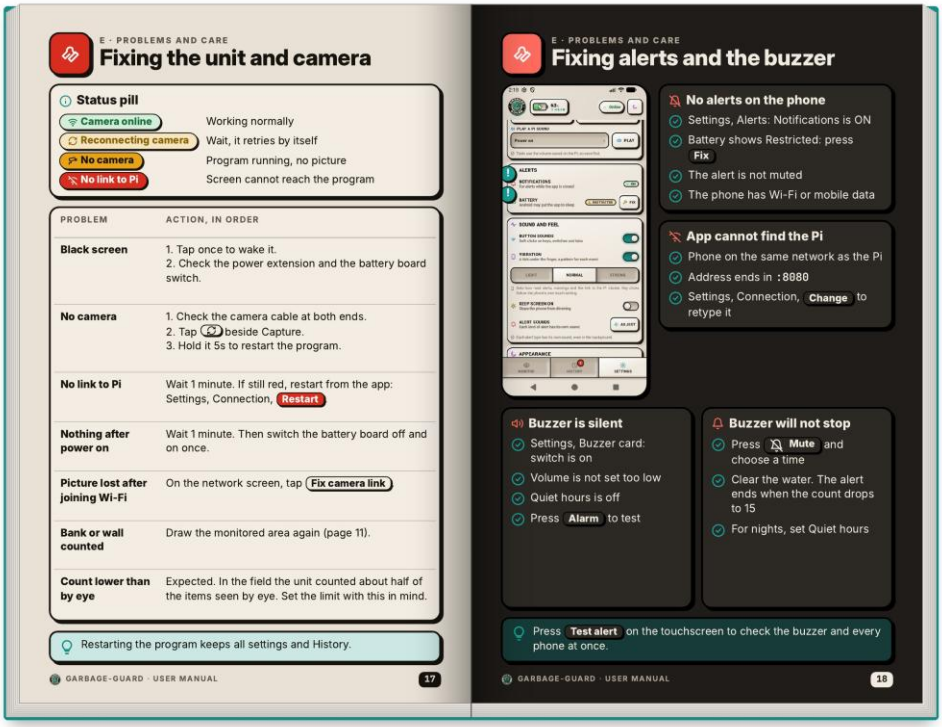



Figure I9 is the troubleshooting sheet. The left page begins with a key to the status pill: Camera online means working normally, Reconnecting camera means wait because it retries by itself, No camera means the program is running but there is no picture, and No link to Pi means the screen cannot reach the program. A table then lists each problem with the actions to take in order, for a black screen, no camera, no link to Pi, a unit that shows nothing after power on, a lost picture after joining Wi-Fi, a bank or wall being counted and a count lower than by eye. The last row says plainly that in the field the unit counted about half of the items seen by eye, so the limit should be set with this in mind. The right page covers alerts and the buzzer: no alerts on the phone, an app that cannot find the Pi, a silent buzzer and a buzzer that will not stop, each with a short checklist.

**Figure I10**

*Care, Maintenance and Quick Reference*

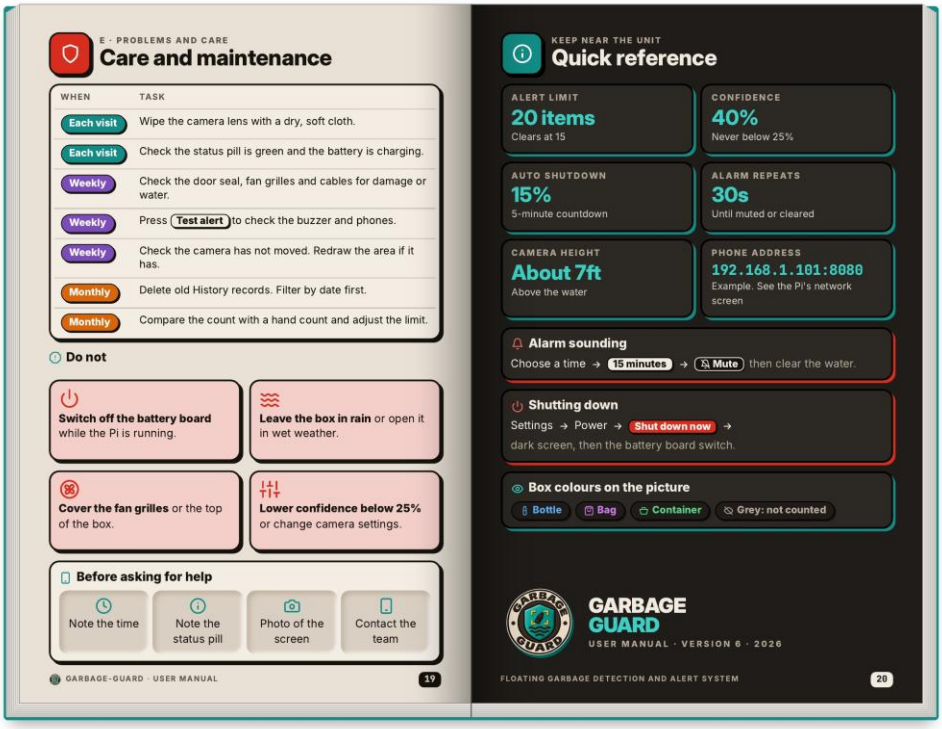



Figure I10 closes the manual with care tasks and a quick reference. The left page is a maintenance schedule. At each visit, wipe the camera lens with a dry, soft cloth and check that the status pill is green and the battery is charging. Weekly, check the door seal, fan grilles and cables, press Test alert, and check that the camera has not moved. Monthly, delete old History records and compare the count with a hand count to adjust the limit. A box of things not to do warns against switching off the battery board while the Pi is running, leaving the box in rain or open in wet weather, covering the fan grilles or the top of the box, and lowering confidence below 25%. The right page is a card to keep near the unit, with the alert limit of 20 items, confidence of 40%, auto shutdown at 15%, an alarm that repeats every 30s, a camera height of about 7ft and the example phone address. It also lists what to note before asking the team for help.

## X. STUDENTS' BIONOTE

**Brian Jay Celeste** is a Computer Engineering Technology student at the Technological University of the Philippines – Manila. He holds the Japanese Language Proficiency Test N3 certificate, won the C-Language Competition in May 2023 and is a Technical Associate of the CIT Student Council. In this study, he is the second project developer, who worked on the system architecture, the YOLO26 detection model and the Android application.

**Genesis Walter D. Reyes II** is a Computer Engineering Technology student at the Technological University of the Philippines – Manila. He finished the STEM strand at St. Joseph's Academy in Las Piñas City, is proficient in C, Python and Java, and attended the Beyond Pixels seminar on Android mobile development in 2024. In this study, he is the first and main project developer, who trained the YOLO26 garbage detection model.

**Jade C. Gunayan** is a Computer Engineering Technology student at the Technological University of the Philippines – Manila. He attended San Rafael Technological Vocational High School from 2021 to 2023 and completed the Civic Welfare Training Service at the university in August 2024. In this study, he conducted the research testing and the statistical analysis that verify the performance of the prototype.

**King Perth G. Pepito** is a Computer Engineering Technology student at the Technological University of the Philippines – Manila. He has been a member of the Institute of Computer Engineering Technologist (ICPET) since August 2024 and was Vice Coordinator of the Ministry of Altar Servers at San Roque Cathedral Parish. In this study, he analyzed the system, set the group's rules and proofread the paper.

**Rodel S. Banogbanog** is a Computer Engineering Technology student at the Technological University of the Philippines – Manila. He holds National Certificate II (NC II) qualifications in Computer Systems Servicing and in Electrical Installation and Maintenance and knows basic C++, Java and Python. In this study, he fabricated the prototype and built the design into a working unit.